CodeDeGrace ICPC Team Notebook

Contents

1 Template 2
1.1 Template . . . 2

2 Data-structures 2
2.1 Segment tree . . . 2
2.2 Segment tree iterativo . . . 2
2.3 Segment tree tricks . . . 2
2.4 Segment tree with structures . . . 3
2.5 Segment tree segmento con mayor suma . . . 3
2.6 Merge Sort Tree . . . 4
2.7 Segment tree lazy . . . 4
2.8 Segment tree lazy some operations . . . 4
2.9 Segment tree lazy arithmetic progression . . . 5
2.10 Segment tree lazy segmento con mayor suma . . . 5
2.11 Segment tree persistent . . . 6
2.12 Segment tree sparse . . . 7
2.13 Segment tree sparse lazy . . . 7
2.14 Segment tree 2D . . . 8
2.15 Segment tree 2D sparse . . . 9
2.16 Fenwick Tree . . . 9
2.17 Sparse Table . . . 9
2.18 SQRT Decomposition . . . 10
2.19 Mo's Algorithm . . . 10
2.20 Rectangular partial sums . . . 11
2.21 Kadane . . . 11

3 Graphs 11
3.1 Dijkstra . . . 11
3.2 Bellman Ford . . . 11
3.3 SPFA . . . 12
3.4 Floyd Warshall . . . 12
3.5 BFS 0-1 . . . 12
3.6 Find edges and vertexes in shortest paths . . . 13
3.7 LCA BFS . . . 13
3.8 DSU . . . 13
3.9 DSU bipartition test . . . 14
3.10 DSU rollback . . . 14
3.11 Dynamic connectivity offline . . . 14
3.12 Kruskal . . . 15
3.13 Prim . . . 15
3.14 Tarjan SCC . . . 16
3.15 Toposort . . . 16
3.16 Bridges . . . 16
3.17 Articulation points . . . 17
3.18 TSP . . . 17
3.19 Counting cycles . . . 17
3.20 Counting paths . . . 18
3.21 Eulerian path . . . 18
3.22 Grafo funcional Floyd . . . 19
3.23 Grafo funcional count steps . . . 19
3.24 Grafo funcional kth step . . . 20
3.25 Find faces grafo plano . . . 20

4 Trees 21
4.1 Diameter . . . 21
4.2 Euler Tour . . . 21
4.3 LCA Euler Tour . . . 22
4.4 LCA and Binary Lifting . . . 22
4.5 Heavy light decomposition . . . 22
4.6 Max clique . . . 23
4.7 Minimum vertex cover . . . 23

5 Flow 24
5.1 Dinic . . . 24
5.2 Push Relabel . . . 25
5.3 Hungarian . . . 26
5.4 Min Cost Max Flow . . . 26
5.5 Blossom . . . 27

6 DP 27
6.1 Divide and Conquer . . . 27
6.2 Knuth . . . 28
6.3 Convex Hull Trick . . . 28
6.4 Convex Hull Trick dinamico . . . 28
6.5 Digit DP . . . 29
6.6 LIS . . . 29
6.7 Recorrer submasks . . . 30

7 Math 30
7.1 Combinatory . . . 30
7.2 Coeficientes binomiales . . . 32
7.3 Power of factorial divisor . . . 33
7.4 Factorial MOD . . . 33
7.5 Matriz . . . 33
7.6 Probability . . . 34
7.7 FFT . . . 36
7.8 Random . . . 36
7.9 Composicion de funciones . . . 37
7.10 Numeros practicos . . . 37

8 Number Theory 37
8.1 Binpow . . . 37
8.2 Inverso modular . . . 37
8.3 Descomposicion prima . . . 37
8.4 Sieve of Eratosthenes . . . 37
8.5 Segmented sieve . . . 37
8.6 Divisores de cada numero . . . 37
8.7 Miller Rabin . . . 37
8.8 Pollard rho . . . 38
8.9 Funcion de Mobius . . . 38
8.10 Funcion de Euler . . . 38
8.11 Mulpow . . . 39
8.12 Euclides . . . 39
8.13 Diofanticas . . . 39
8.14 Chinese remainder theorem . . . 39
8.15 Fibonacci code . . . 39
8.16 Teoria . . . 39

9 Miscelanea 40
9.1 Compresion . . . 40
9.2 Biseccion . . . 41
9.3 Busqueda ternaria . . . 41
9.4 Bits . . . 41
9.5 Sliding window . . . 41
9.6 Game theory . . . 41
9.7 Games on arbitrary graphs . . . 42
9.8 Arbol cartesiano . . . 43
9.9 Combinaciones longitud k . . . 43
9.10 Josefo . . . 43
9.11 Hashing XOR . . . 43
9.12 Python . . . 44

10 C++ 45
10.1 Bitset . . . 45
10.2 Enteros de 128 bits . . . 45
10.3 Imprimir decimales . . . 46
10.4 Deque . . . 46
10.5 Indexed set . . . 46
10.6 Listas . . . 46
10.7 Map . . . 46
10.8 Metodos de ordenamiento . . . 46
10.9 Permutaciones . . . 46

11 Strings 46
11.1 Hashing . . . 46
11.2 Minimal Rotation . . . 47
11.3 String Matching . . . 47
11.4 Trie . . . 47

12 Geometria 47
12.1 Geo Struct . . . 47
12.2 Convex Hull . . . 52
12.3 Intersection Points . . . 52
12.4 Line Segment Intersection . . . 52
12.5 Segment Intersection . . . 53
12.6 Point in Polygon . . . 53
12.7 Polygons Area . . . 53
12.8 Li Chao Tree . . . 53
12.9 Gauss (UNTESTED) . . . 54

1 Template

1.1 Template

```cpp
#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
#define db(x) cerr<< #x<<" "<<x<<endl
#define for0(i,n) for(int i = 0; i < (int)n; i++)
#define for1(i,n) for(int i = 1; i <= (int)n; i++)
#define forlr(i,l,r) for(int i = (int)l; i <= (int)r; i
    ++)
#define forn1(i,n) for(int i = (int)n; i > 0; i--)
#define forn0(i,n) for(int i = (int)(n) - 1; i >= 0; i
    --)
#define forrl(i,l,r) for(int i = (int)r; i >= (int)l; i
    --)
#define pb push_back
#define sz(a) ((int)a.size())
#define all(a) a.begin(), a.end()
#define fi first
#define se second
#define lb lower_bound
#define ub upper_bound
#define pqueue priority_queue
typedef long long ll;
typedef long double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<string> vs;
typedef vector<bool> vb;
typedef vector<pii> vii;
typedef vector<pll> vll;
// #include<ext/pb_ds/assoc_container.hpp>
// using namespace __gnu_pbds;
// using indexed_set = tree<int, null_type, less<int>,
    rb_tree_tag, tree_order_statistics_node_update>;

const ll INFL = 1000000000000000001;
const int INF = 1e9 + 1;
// const ll MOD = 1e9 + 7;

void solver(){

}

int main(){
    ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    // freopen("name.in", "r", stdin);
  // freopen("name.out", "w", stdout);
    int t = 1;
    // cin>>t;
    while(t--){
        solver();
    }

    return 0;
}
```

2 Data-structures

2.1 Segment tree

```cpp
// Implementacion con array 0-index
/*
- Se pueden calcular sumas, maximos y minimos,
    multiplicacion, multiplicacion de
  matrices), and (&), or (|) y xor (^), gcd y cualquier
      operacion que cumpla con
  la propiedad asociativa.
- Se pueden contar minimos/maximos utilizando un par que
     guarde minimo, cantidad
  de apariciones.
*/

struct node{ // Change
    ll val;
};

class segTree {
private:
    int size;
    vector<node> tree;
    node neutro = {LLONG_MAX - 1}; // Change

    node calcOp(node a, node b){ // Change
        if(a.val == neutro.val) return b;
        if(b.val == neutro.val) return a;
        node ans = {a.val + b.val};
        return ans;
    }

    void update(int pos, ll val, int v, int tl, int tr){
        if(tr - tl == 1){
            tree[v] = {val}; // Change
            return;
        }

        int tm = (tl + tr) / 2;
        if(pos < tm) update(pos, val, 2*v + 1, tl, tm);
        else update(pos, val, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

    // [l, r)
    node calc(int l, int r, int v, int tl, int tr){ //
        Change si walking on segment tree
        if(tl >= r || l >= tr) return neutro;
        if(tl >= l && tr <= r) return tree[v]; // si
            walking on seg tree -> tr - tl == 1

        int tm = (tl + tr) / 2;
        node m1 = calc(l, r, 2*v + 1, tl, tm);
        node m2 = calc(l, r, 2*v + 2, tm, tr);
        return calcOp(m1, m2);
    }

    void build(vl& a, int v, int tl, int tr){
        if(tr - tl == 1){
            if(tl < sz(a)) tree[v] = {a[tl]}; // Change
            return;
        }
        int tm = (tr + tl) / 2;
        build(a, 2*v + 1, tl, tm);
        build(a, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

public:
    void init(int n){
        size = 1;
        while(size < n) size *= 2;
        tree.assign(2*size, {0LL});
    }
    void update(int pos, ll val){update(pos, val, 0, 0,
        size);}
    node calc(int l, int r){return calc(l, r, 0, 0, size
        );}
    void build(vector<ll>& a){build(a, 0, 0, size);}
};

void solver(){
    int n, m; cin>>n>>m;
    vl a(n);
    for(int i = 0; i < n; i++) cin>>a[i];

    // Inicializar
    segTree st; st.init(n);
    st.build(a);

    // Updates
    int i; ll v; cin>>i>>v;
    st.update(i, v);

    // Calcular
    int l, r; cin>>l>>r;
    cout<<st.calc(l, r).val<<endl;
}
```

2.2 Segment tree iterativo

```cpp
/*
- 0-index, se usa exactamente igual que el recursivo.
- Un poco mas eficiente que el recursivo.
- El vector que se mande en el build debe tener tamano
    igual al
  parametro que se manda en el init.
- Usar para consultas simples como sum, min, max, xor,
    gcd, etc.
*/

struct segTree{
    int n; ll neutro = LLONG_MAX - 1; // CHANGE neutro
    vl tree;
    inline ll op(ll a, ll b){ return min(a, b); } //
        CHANGE
    void init(int _n){
        n = _n;
        tree.assign(2*n, neutro);
    }
    void build(vl& a){
        for0(i, n) tree[n + i] = a[i];
        forrl(i, 1, n-1) tree[i] = op(tree[i<<1], tree[i
            <<1|1]);
    }
    ll calc(int l, int r){  // [l, r)
        ll resL = neutro, resR = neutro;
        for(l += n, r += n; l < r; l >>= 1, r >>= 1){
            if(l&1) resL = op(resL, tree[l++]);
            if(r&1) resR = op(tree[--r], resR);
        }
        return op(resL, resR);
    }
    void update(int p, ll val){  // set val at position
        p (0 - idx)
        for(tree[p += n] = val; p > 1; p >>= 1) tree[p
            >>1] = op(tree[p], tree[p^1]);
    }
};
```

2.3 Segment tree tricks

```cpp
/*****************************
Nested segments:
Para contar la cantidad de segmentos contenidos en otro
    se pueden recorrer los
segmentos en order descendente de l y ascendente de r,
    se hace una querie de
suma en su rango (ese valor es la cantidad de segmentos
    contenidos en este) y
finalmente se actualiza su valor con 1 en r.
*****************************/

/*****************************
Intersecting Segments:
Para hallar la cantidad de segmentos que se intersectan
    con otro, se recorren
los puntos del arreglo. Al llegar a un extremo l, se
    marca esta posicion en 1.
```

```
Al llegar a un extremo r se marca la posicion en l con
    -1 y en r con 1 y se hace
una querie de suma en (l, r) o [l, r] segun sea el caso,
     ese valor es la
cantidad de segmentos que se intersectan con este. La
    suma total dividido 2 es
la cantidad de intersecciones.
*****************************/

/*****************************
kth one:
En un array de 1s y 0s, hallar la kth posicion donde hay
     un 1. Sirve para ejercicios
donde me pidan activar y desactivar posiciones.
    Consultas sobre el array completo.
Tambien se puede usar un enfoque similar para hallar el
    kth elemento mayor o igual a x
(pero las queries deben ser sobre el array completo y no
     sobre rangos especificos, de
lo contrario usar persistente).
Usar operacion de sumas y: */
int calc(int l, int r, int k, int v, int tl, int tr){
    // propagate(v, tl, tr); // Si es lazy
    if(tl >= r || l >= tr) return -1; // Hay menos de k
        1s
    if(tr - tl == 1) return tl;

    int tm = (tl + tr) / 2;
    if(tree[2*v + 1].val >= k) return calc(l, r, k, 2*v
        + 1, tl, tm);
    k -= tree[2*v + 1].val;
    return calc(l, r, k, 2*v + 2, tm, tr);
}
/*****************************/

/*****************************
- Primer elemento >= x:
  Hallar la primera posicion i >= l con a[i] >= x. Usar
     operacion de maximos y: */
int calc(int l, int r, ll x, int v, int tl, int tr){ //
    Change si walking on segment tree
    // propagate(v, tl, tr); // Si es lazy
    if(tl >= r || l >= tr) return -1; // No hay ninguno
    if(tr - tl == 1) return tree[v].val >= x ? tl : -1;

    int tm = (tl + tr) / 2;
    int ans = -1;
    if(tree[2*v + 1].val >= x) ans = calc(l, r, x, 2*v +
        1, tl, tm);
    if(ans == -1 && tree[2*v + 2].val >= x) ans = calc(l
        , r, x, 2*v + 2, tm, tr);
    return ans;
}
```

2.4 Segment tree with structures

```
/*
- Para problemas en los cuales se necesite que cada nodo
     sea una estructura se
  puede tomar esta implementacion como base. La
     complejidad es casi la misma
  que la de un segment tree normal agregando un factor
     segun la complejidad
  de las operaciones de insert y erase de la estructura
     utilizada.
- Este ejemplo usando set tiene una complejidad temporal
    de O(n*lg(n)) para build
  y O(lg(n)^2) para update. La complejidad espacial
     tambien es O(n*lg(n)).
- Segun esa informacion que se necesite se debe hacer
    binary search, acceder a
  la clase del mapa, etc. Entonces la complejidad de
     calculos es variable, pero
  suele ser O(lg(n)^2).
- Normalmente se usa para problemas similares a los que
     se pueden resolver usando
  merge sort tree, pero que requieren updates. Si bien
     la complejidad es la misma
  el factor constante es un poco mas alto.
*/

struct node{ // Change
    set<ll> val;
};

class segTree{
private:
    int size;
    vector<node> tree;

    node neutro = {{}}; // Change

    node calcOp(node& a, node& b){ // Change
        if(a.val == neutro.val) return b;
        if(b.val == neutro.val) return a;
        node ans = a;
        for(ll x : b.val) ans.val.insert(x);
        return ans;
    }

    void update(int pos, ll val, int v, int tl, int tr){
        if(tr - tl == 1){
            tree[v] = {{val}}; // Change
            return;
        }

        int tm = (tl + tr) / 2;
        if(pos < tm) update(pos, val, 2*v + 1, tl, tm);
        else update(pos, val, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

    // [l, r)
    ll calc(int l, int r, int v, int tl, int tr){
        if(tl >= r || l >= tr) return 0;
        if(tl >= l && tr <= r){
            // Para los calculos se deben usar cosas
                como
            // el tamano del set, binary search sobre el
                 set,
            // etc.
        }

        int tm = (tl + tr) / 2;
        ll m1 = calc(l, r, 2*v + 1, tl, tm);
        ll m2 = calc(l, r, 2*v + 2, tm, tr);
        return m1 + m2;
    }

    void build(vl& a, int v, int tl, int tr){ // O(n)
        if(tr - tl == 1){
           if(tl < sz(a)) tree[v] = {{a[tl]}}; // Change
           return;
        }
        int tm = (tr + tl) / 2;
        build(a, 2*v + 1, tl, tm);
        build(a, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

public:
    void init(int n){
        size = 1;
        while(size < n) size *= 2;
        tree.assign(2*size, {{}});
    }
    void update(int pos, ll val){update(pos, val, 0, 0,
        size);}
    ll calc(int l, int r){return calc(l, r, 0, 0, size)
        ;}
    void build(vector<ll>& a){build(a, 0, 0, size);}
};
```

2.5 Segment tree segmento con mayor suma

```
struct node{
    ll val, pre, suf, sum;
};

class segTree{
private:
    int size;
    vector<node> tree;

    node neutro = {LLONG_MAX - 1, LLONG_MAX - 1,
        LLONG_MAX - 1, LLONG_MAX - 1};

    node calcOp(node a, node b){
        if(a.val == neutro.val) return b;
        if(b.val == neutro.val) return a;
        node ans;
        ans.val = max(max(0LL, a.suf + b.pre), max(a.val
            , b.val));
        ans.pre = max(0LL, max(a.pre, a.sum + b.pre));
        ans.suf = max(0LL, max(b.suf, a.suf + b.sum));
        ans.sum = a.sum + b.sum;
        return ans;
    }

    void update(int pos, ll val, int v, int tl, int tr){
         // O(lg(n))
        if(tr - tl == 1){
            tree[v] = {max(0LL, val), max(0LL, val), max
                (0LL, val), val};
            return;
        }

        int tm = (tl + tr) / 2;
        if(pos < tm) update(pos, val, 2*v + 1, tl, tm);
        else update(pos, val, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

    // O(lg(n))
    // [l, r)
    node calc(int l, int r, int v, int tl, int tr){
        if(tl >= r || l >= tr) return neutro;
        if(tl >= l && tr <= r) return tree[v];

        int tm = (tl + tr) / 2;
        node m1 = calc(l, r, 2*v + 1, tl, tm);
        node m2 = calc(l, r, 2*v + 2, tm, tr);
        return calcOp(m1, m2);
    }

    void build(vl& a, int v, int tl, int tr){ // O(n)
        if(tr - tl == 1){
           if(tl < sz(a)) tree[v] = {max(0LL, a[tl]),
               max(0LL, a[tl]), max(0LL, a[tl]), a[tl
               ]};
           return;
        }
        int tm = (tr + tl) / 2;
        build(a, 2*v + 1, tl, tm);
        build(a, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

public:
    void init(int n){
        size = 1;
        while(size < n) size *= 2;
        tree.assign(2*size, {0LL, 0LL, 0LL, 0LL});
```

```
    }
    void update(int pos, ll val){update(pos, val, 0, 0,
        size);}
    node calc(int l, int r){return calc(l, r, 0, 0, size
        );}
    void build(vl& a){build(a, 0, 0, size);}
};
```

2.6 Merge Sort Tree

```
/*
- Para problemas en los cuales no hay updates y se
    necesiten cosas como contar
  diferentes, contar cantidad de elementos en un rango
     de valores o similares.
- Complejidad temporal de O(n*lg(n)) para build y O(lg(n
    )^2) para update.
  La complejidad espacial tambien es O(n*lg(n)).
- Segun esa informacion que se necesite, normalmente, se
     debe hacer binary
  search. Entonces la complejidad de calculos suele ser
     O(lg(n)^2).
Aplicaciones:
- Hallar la frecuencia de un numero en especifico en un
    rango.
- Hallar la cantidad de elementos mayores o menores a x
    en un rango.
- Hallar la cantidad de elementos entre [x, y] en un
    rango.
*/

class segTree{
private:
    int size;
    vector<vl> tree;
    vl neutro = {{}}; // Change

    vl calcOp(vl& a, vl& b){ // Change
        if(a == neutro) return b;
        if(b == neutro) return a;
        vl ans;
        int i = 0, j = 0;
        while(i < sz(a) || j < sz(b)){
            if(i >= sz(a) || (j < sz(b) && b[j] <= a[i])
                ){
                ans.pb(b[j]);
                j++;
            }
            else{
                ans.pb(a[i]);
                i++;
            }
        }

        return ans;
    }

    ll calc(int l, int r, int v, int tl, int tr){
        if(tl >= r || l >= tr) return 0;
        if(tl >= l && tr <= r){
            // Para los calculos se deben usar cosas
                como
            // el tamano del vector, binary search, etc.
        }

        int tm = (tl + tr) / 2;
        ll m1 = calc(l, r, 2*v + 1, tl, tm);
        ll m2 = calc(l, r, 2*v + 2, tm, tr);
        return m1 + m2;
    }

    void build(vl& a, int v, int tl, int tr){ // O(n)
        if(tr - tl == 1){
            if(tl < sz(a)) tree[v] = {{a[tl]}}; // Change
            return;
        }
        int tm = (tr + tl) / 2;
        build(a, 2*v + 1, tl, tm);
        build(a, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

public:
    void init(int n){
        size = 1;
        while(size < n) size *= 2;
        tree.assign(2*size, {});
    }
    ll calc(int l, int r){return calc(l, r, 0, 0, size)
        ;}
    void build(vector<ll>& a){build(a, 0, 0, size);}
};
```

2.7 Segment tree lazy

```
/*
Segment Tree Lazy Propagation (ejemplo con update assign
     y calc suma).
- Es necesario que se cumpla la propiedad asociativa
    tanto en las operaciones de update como calc.
- Es necesario que se cumpla que calcOp(updateOp(a, x),
    updateOp(b, x)) == updateOp(calcOp(a, b), x),
  es decir, que updateOp sea distributiva relativa a
     calcOp, ejemplo, update de multiplicacion, calc
  suma. En caso de que no se cumpla esa propiedad (
     update assign, calc suma o update suma, calc suma
     ),
  se debe ajustar utilizando la longitud del rango en la
      operacion update.
- Tambien se pueden hacer updates donde cada valor a_i
    se lleva al minimo entre x y a_i (o maximo) y
  hacer consultas de elementos.
*/

class segTree {
private:
    int size;
    vl lazy;
    vl tree;

    ll neutro = LLONG_MAX - 1;

    // Para query suma y update assign
    ll updateOp(ll a, ll b, ll len){
        if(b == neutro) return a;
        return b * len;
    }
    ll calcOp(ll a, ll b){
        if(a == neutro) return b;
        if(b == neutro) return a;
        return a + b;
    }

    void applyUpdOp(ll &a, ll b, ll len){
        a = updateOp(a, b, len);
    }

    void propagate(int v, int tl, int tr){
        if(tr - tl == 1) return;
        int tm = (tr + tl) / 2;
        applyUpdOp(lazy[2*v + 1], lazy[v], 1);
        applyUpdOp(tree[2*v + 1], lazy[v], tm - tl);
        applyUpdOp(lazy[2*v + 2], lazy[v], 1);
        applyUpdOp(tree[2*v + 2], lazy[v], tr - tm);
        lazy[v] = neutro;
    }

    // [l, r)
    void update(int l, int r, ll val, int v, int tl, int
         tr){
        propagate(v, tl, tr);
        if(tl >= r || l >= tr) return;
        if(tl >= l && tr <= r){
            applyUpdOp(lazy[v], val, 1);
            applyUpdOp(tree[v], val, tr - tl);
            return;
        }

        int tm = (tl + tr) / 2;
        update(l, r, val, 2*v + 1, tl, tm);
        update(l, r, val, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

    // [l, r)
    ll calc(int l, int r, int v, int tl, int tr){
        propagate(v, tl, tr);
        if(tl >= r || l >= tr) return neutro;
        if(tl >= l && tr <= r) return tree[v];

        int tm = (tl + tr) / 2;
        ll m1 = calc(l, r, 2*v + 1, tl, tm);
        ll m2 = calc(l, r, 2*v + 2, tm, tr);
        return calcOp(m1, m2);
    }

    void build(vl& a, int v, int tl, int tr){
        if(tr == tl + 1){
            if(tl < sz(a)) tree[v] = a[tl];
            return;
        }
        int tm = (tr + tl) / 2;
        build(a, 2*v + 1, tl, tm);
        build(a, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

public:
    void init(int n){
        size = 1;
        while(size < n) size *= 2;
        lazy.assign(2*size, neutro);
        tree.assign(2*size, 0LL); // Change
    }
    void update(int l, int r, ll val){update(l, r, val,
        0, 0, size);}
    ll calc(int l, int r){return calc(l, r, 0, 0, size)
        ;}
    void build(vl& a){build(a, 0, 0, size);}
};

void solver(){
    int n, m; cin>>n>>m;
    vl a(n);
    for0(i,n) cin>>a[i];

    // Inicializar
    segTree st; st.init(n);
    st.build(a);

    // Updates
    int l, r; ll val; cin>>l>>r>>val;
    st.update(l, r, val);

    // Calcular
    int l, r; cin>>l>>r;
    cout<<st.calc(l, r)<<endl;
}
```

2.8 Segment tree lazy some operations

```
/*****************************
Para query suma y update suma */
    ll updateOp(ll a, ll b, ll len){
```

```
        if(b == neutro) return a;
        if(a == neutro) return b*len;
        return a + b*len;
    }
    ll calcOp(ll a, ll b){
        if(a == neutro) return b;
        if(b == neutro) return a;
        return a + b;
    }
/*****************************/

/*****************************
Para query minimo y update suma */
    ll updateOp(ll a, ll b, ll len){
        if(b == neutro) return a;
        if(a == neutro) return b;
        return a + b;
    }
    ll calcOp(ll a, ll b){
        if(a == neutro) return b;
        if(b == neutro) return a;
        return min(a, b);
    }
/*****************************/

/*****************************
Para arrays de 1s y 0s y updates de invertir los valores
     (calc de suma) */
    int updateOp(int a, int len){
        return len - a;
    }

    void propagate(int v, int tl, int tr){
        if(tr - tl == 1 || !lazy[v]) return;
        int tm = (tr + tl) / 2;
        lazy[2*v + 1] = !lazy[2*v + 1];
        lazy[2*v + 2] = !lazy[2*v + 2];
        applyUpdOp(tree[2*v + 1], tm - tl);
        applyUpdOp(tree[2*v + 2], tr - tm);
        lazy[v] = 0;
    }

    // [l, r)
    void update(int l, int r, int v, int tl, int tr){
        propagate(v, tl, tr);
        if(tl >= r || l >= tr) return;
        if(tl >= l && tr <= r){
            lazy[v] = 1;
            applyUpdOp(tree[v], tr - tl);
            return;
        }

        int tm = (tl + tr) / 2;
        update(l, r, 2*v + 1, tl, tm);
        update(l, r, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }
/*****************************/

/*****************************
- Si se llega a pedir update de diferentes tipos, toca
    tener cuidado con la propagacion. Ejemplo con
  update de asignacion (operacion identificada con 1) y
      de suma (operacion identificada con 0): */
    // Updates:
    ll updateOp(ll a, ll b, ll len, int op){ // op ->
        assign, !op -> suma
        if(op == -1) return neutro;
        if(b == neutro) return a;
        if(a == neutro) return b * len;
        return op ? b * len : a + b * len;
    }
    // Propagacion:
    void propagate(int v, int tl, int tr){
        if(tr - tl == 1 || lazy[v].se == -1) return;
        int tm = (tr + tl) / 2;

        lazy[2*v + 1].fi = updateOp(lazy[2*v + 1].fi,
            lazy[v].fi, 1, lazy[v].se);
        if(lazy[2*v + 1].se == -1) lazy[2*v + 1].se =
            lazy[v].se;
        else lazy[2*v + 1].se = min(1, lazy[2*v + 1].se
            + lazy[v].se); // La asignacion prevalece
            ya que se le suma al valor que se debe
            asignar

        tree[2*v + 1] = updateOp(tree[2*v + 1], lazy[v].
            fi, tm - tl, lazy[v].se);


        lazy[2*v + 2].fi = updateOp(lazy[2*v + 2].fi,
            lazy[v].fi, 1, lazy[v].se);
        if(lazy[2*v + 2].se == -1) lazy[2*v + 2].se =
            lazy[v].se;
        else lazy[2*v + 2].se = min(1, lazy[2*v + 2].se
            + lazy[v].se);

        tree[2*v + 2] = updateOp(tree[2*v + 2], lazy[v].
            fi, tm - tl, lazy[v].se);
        lazy[v] = {neutro, -1};
    }
```

2.9 Segment tree lazy arithmetic progression

```
/*
- Updates del tipo l, r, val, inc
  Hago a[l] += val, a[l+1] += val + inc, a[l+2] += val +
       2*inc, hasta llegar a r-1.
- Queries de suma.
*/

ll gauss(ll n){
    return n <= 0 ? 0 : (n * (n + 1)) >> 1;
}

class segTee{
private:
    int size;
    vll lazy;
    vl tree;

    ll neutro = LLONG_MAX - 1;

    ll updateOp(ll a, pll b, ll len){
        if(b.fi == neutro) return a;
        assert(a != neutro);
        return a + b.fi * len + b.se * gauss(len - 1);
    }

    pll updateOpL(pll a, pll b, ll len){
        if(b.fi == neutro) return a;
        if(a.fi == neutro) return {b.fi + b.se * len, b.
            se};
        return {a.fi + b.fi + b.se * len, a.se + b.se};
    }

    ll calcOp(ll a, ll b){
        if(a == neutro) return b;
        if(b == neutro) return a;
        return a + b;
    }

    void applyUpdOp(ll &a, pll b, ll len){
        a = updateOp(a, b, len);
    }

    void applyUpdOpL(pll &a, pll b, ll len){
        a = updateOpL(a, b, len);
    }

    void propagate(int v, int tl, int tr){
        if(tr - tl == 1) return;
        int tm = (tr + tl) / 2;
        applyUpdOpL(lazy[2*v + 1], lazy[v], 0);
        applyUpdOp(tree[2*v + 1], lazy[v], tm - tl);
        applyUpdOpL(lazy[2*v + 2], lazy[v], tm - tl);
        applyUpdOp(tree[2*v + 2], {lazy[v].fi + (ll)(tm
            - tl)*lazy[v].se, lazy[v].se}, tr - tm);
        lazy[v] = {neutro, 0};
    }

    // [l, r)
    void update(int l, int r, ll val, ll inc, int v, int
         tl, int tr){
        propagate(v, tl, tr);
        if(tl >= r || l >= tr) return;
        if(tl >= l && tr <= r){
            applyUpdOpL(lazy[v], {val, inc}, tl - l);
            applyUpdOp(tree[v], {val + inc * (tl - l),
                inc}, tr - tl);
            return;
        }

        int tm = (tl + tr) / 2;
        update(l, r, val, inc, 2*v + 1, tl, tm);
        update(l, r, val, inc, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

    // [l, r)
    ll calc(int l, int r, int v, int tl, int tr){
        propagate(v, tl, tr);
        if(tl >= r || l >= tr) return neutro;
        if(tl >= l && tr <= r) return tree[v];

        int tm = (tl + tr) / 2;
        ll m1 = calc(l, r, 2*v + 1, tl, tm);
        ll m2 = calc(l, r, 2*v + 2, tm, tr);
        return calcOp(m1, m2);
    }

    void build(vl& a, int v, int tl, int tr){
        if(tr == tl + 1){
            if(tl < sz(a)) tree[v] = a[tl];
            return;
        }
        int tm = (tr + tl) / 2;
        build(a, 2*v + 1, tl, tm);
        build(a, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

public:
    void init(int n){
        size = 1;
        while(size < n) size *= 2;
        lazy.assign(2*size, {neutro, 0});
        tree.assign(2*size, 0LL);
    }
    void update(int l, int r, ll val, ll inc){update(l,
        r, val, inc, 0, 0, size);}
    ll calc(int l, int r){return calc(l, r, 0, 0, size)
        ;}
    void build(vl& a){build(a, 0, 0, size);}
};
```

2.10 Segment tree lazy segmento con mayor suma

```
// Update de asignacion, query de segmento con mayor
     suma
// La respuesta se guarda en .val

struct node{
    ll val, pref, suf, sum;
};
```

```
class segTree{
private:
    int size;
    vector<node> lazy;
    vector<node> tree;

    node neutro = {0LL, 0LL, 0LL, LLONG_MAX - 1};

    node updateOp(node a, node b, ll len){
        if(b.sum == neutro.sum) return a;
        node ans = {0LL, 0LL, 0LL, 0LL};
        ans.sum = b.sum * len;
        if(b.sum > 0LL) ans.val = ans.suf = ans.pref =
            ans.sum;

        return ans;
    }

    node calcOp(node a, node b){
        if(a.sum == neutro.sum) return b;
        if(b.sum == neutro.sum) return a;
        node ans;
        ans.val = max(0LL, max(max(a.val, b.val), a.suf
            + b.pref));
        ans.pref = max(0LL, max(a.pref, a.sum + b.pref))
            ;
        ans.suf = max(0LL, max(b.suf, b.sum + a.suf));
        ans.sum = a.sum + b.sum;

        return ans;
    }

    void applyUpdOp(node &a, node b, ll len){
        a = updateOp(a, b, len);
    }

    void propagate(int v, int tl, int tr){
        if(tr - tl == 1) return;
        int tm = (tr + tl) / 2;
        applyUpdOp(lazy[2*v + 1], lazy[v], 1);
        applyUpdOp(tree[2*v + 1], lazy[v], tm - tl);
        applyUpdOp(lazy[2*v + 2], lazy[v], 1);
        applyUpdOp(tree[2*v + 2], lazy[v], tr - tm);
        lazy[v] = neutro;
    }

    // [l, r)
    void update(int l, int r, ll val, int v, int tl, int
         tr){
        propagate(v, tl, tr);
        if(tl >= r || l >= tr) return;
        if(tl >= l && tr <= r){
            node x = {0LL, 0LL, 0LL, val};
            applyUpdOp(lazy[v], x, 1);
            applyUpdOp(tree[v], x, tr - tl);
            return;
        }

        int tm = (tl + tr) / 2;
        update(l, r, val, 2*v + 1, tl, tm);
        update(l, r, val, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

    // [l, r)
    node calc(int l, int r, int v, int tl, int tr){
        propagate(v, tl, tr);
        if(tl >= r || l >= tr) return neutro;
        if(tl >= l && tr <= r) return tree[v];

        int tm = (tl + tr) / 2;
        node m1 = calc(l, r, 2*v + 1, tl, tm);
        node m2 = calc(l, r, 2*v + 2, tm, tr);
        return calcOp(m1, m2);
    }

    void build(vl& a, int v, int tl, int tr){
        if(tr == tl + 1){
            if(tl < sz(a)) tree[v] = tree[v] = {max(0LL,
                a[tl]), max(0LL, a[tl]), max(0LL, a[tl
                ]), a[tl]};
            return;
        }
        int tm = (tr + tl) / 2;
        build(a, 2*v + 1, tl, tm);
        build(a, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

public:
    void init(int n){
        size = 1;
        while(size < n) size *= 2;
        lazy.assign(2*size, neutro);
        tree.assign(2*size, neutro);
    }
    void update(int l, int r, ll val){update(l, r, val,
        0, 0, size);}
    node calc(int l, int r){return calc(l, r, 0, 0, size
        );}
    void build(vl& a){build(a, 0, 0, size);}
};
```

2.11 Segment tree persistent

```
/*
- 0-index. Implementacion de sumas.
- Los nodos se relacionan entre versiones usando los
    indices, es por
  eso que build y update retornan indices.
- Tener cuidado con los casos donde se retorna neutro,
    agregar un if.
- Cada update crea una nueva version.
- Puede ser util guardar un mapa que diga la version a
    la que
  corresponde cada update si para el ejercicio no
      necesariamente
  cada update representa una version nueva. En este caso
      tener
  cuidado porque cada update se hace sobre una nueva
      version y
  puede ser necesario actualizar tambien el mapa.
- Se usan las funciones between para hacer calculos
    entre versiones,
  en esta implementacion se hacen calculos de suma en v1
      - suma en v2.
  Esto se puede usar para cualquier tipo de calculo que
      tenga inversa.
- Pude ser util construir el array mediante updates, ya
    que en
  ocasiones se requiere hacer las queries usando l y r
      como v1 y v2.

Aplicaciones tipicas:
- Cantidad de distintos en un rango (no updates):
  Para hallarlo se recorre el array original con i entre
      1 y n, sobre
  la ultima version se hace una update poniendo en la
      posicion i un
  valor de 1, y se revisa la ultima vez j que el valor a
      [i] aparecio
  en el array, y se hace una update en la ultima version
      podiendo
  en la posicion j un valor de 0.
  Para las queries l, r se hace el calculo en la version
      r del array
  (usar un mapa para manejar las versiones adecuadamente
      ), en el rango
  [l, r], el valor de esa query es el numero de
      elementos distintos
  en ese segmento.
- kth elemento menor en un rango (no updates):
  De manera similar, se recorre el array original con i
      entre 0 y n-1,
  sobre la ultima version se hace un update agregando 1
      a la posicion
  a[i]. Para las queries se toman l, r como las
      versiones del segtree
  y se hacen las queries entre 0 y n. Dentro de
      calcBetween se usa una
  tecnica similar a la del kth one, donde se mira si en
      el hijo left
  la resta entre versiones es > k y se va hacia ese hijo
       o de lo
  contrario se va hacia el hijo derecho, hasta que
      finalmente el tamano
  del rango sea == 1 y se retorna tl.
- Cantidad de elementos >= x en un rango (no updates) (
    tambien se puede
  hacer con merge sort tree). Si hay updates se debe
      usar Treap.
*/

struct node{
    ll val; int l, r; // Change
};

const int LEN = 1e7; // tope de nodos totales (usar
    aprox 4*n + (q+5)*(log(n) + 5))

class segTree {
private:
    node neutro = {LLONG_MAX - 1, -1, -1}; // Change

    node calcOp(node a, node b){ // Change
        if(a.val == neutro.val) return b;
        if(b.val == neutro.val) return a;
        node ans = {a.val + b.val, -1, -1}; // Change
        return ans;
    }

    node calcDifBetween(node v1, node v2){
        if(v1.val == neutro.val) return neutro;
        return {v2.val - v1.val, -1, -1}; // Change
    }

    node calcSolapeBetween(node a, node b){
        if(a.val == neutro.val) return b;
        if(b.val == neutro.val) return a;
        return {a.val + b.val, -1, -1};  // Change
    }

    vector<node> tree;
    int n, idx = 0;

    int update(int prv, int pos, ll val, int tl, int tr)
        {
        int v = idx++;
        tree[v] = tree[prv];
        if(tr - tl == 1){
            tree[v].val = val; // Change
            return v;
        }
        int tm = (tl + tr) / 2;
        if(pos < tm) tree[v].l = update(tree[v].l, pos,
            val, tl, tm);
        else tree[v].r = update(tree[v].r, pos, val, tm,
             tr);
        node combined = calcOp(tree[tree[v].l], tree[
            tree[v].r]);
        tree[v].val = combined.val;
        return v;
    }

    // [l, r)
```

```
    node calc(int v, int l, int r, int tl, int tr){ //
        Change si walking on segment tree
        if(tl >= r || l >= tr) return neutro;
        if(tl >= l && tr <= r) return tree[v]; // si
            walking -> tr - tl == 1

        int tm = (tl + tr) / 2;
        node m1 = calc(tree[v].l, l, r, tl, tm);
        node m2 = calc(tree[v].r, l, r, tm, tr);
        return calcOp(m1, m2);
    }

    // [l, r) y [v1, v2]
    node calcBetween(int v1, int v2, int l, int r, int
        tl, int tr){ // Change si walking on segment
        tree
        if(tl >= r || l >= tr) return neutro;
        if(tl >= l && tr <= r) return calcDifBetween(
            tree[v1], tree[v2]); // si walking -> tr -
            tl == 1
        int tm = (tl + tr) / 2;
        node lc = calcBetween(tree[v1].l, tree[v2].l, l,
             r, tl, tm);
        node rc = calcBetween(tree[v1].r, tree[v2].r, l,
             r, tm, tr);
        return calcSolapeBetween(lc, rc);
    }

    int build(int tl, int tr, vl& a){
        int v = idx++;
        if(tr - tl == 1){
            if(tl < sz(a)) tree[v] = {a[tl], -1, -1}; //
                Change
            return v;
        }
        int tm = (tl + tr) / 2;
        int l = build(tl, tm, a), r = build(tm, tr, a);
        node combined = calcOp(tree[l], tree[r]);
        tree[v] = {combined.val, l, r};
        return v;
    }

public:
    vi roots; // roots[i] = indice de la raiz de la
        version i

    void init(int n_){
        n = n_;
        idx = 0;
        tree.assign(LEN, neutro); // Cuidado si se debe
            inicializar en 0 o similar
        roots.clear();
    }

    // Crea una nueva version a partir de "ver" (indice
        en roots)
    void update(int ver, int pos, ll val){
        roots.pb(update(roots[ver], pos, val, 0, n));
    }

    // Calculo sobre la version "ver"
    node calc(int ver, int l, int r){
        return calc(roots[ver], l, r, 0, n);
    }

    // Para hacer calculos entre versiones
    node calcBetween(int v1, int v2, int l, int r){
        return calcBetween(roots[v1], roots[v2], l, r,
            0, n);
    }

    // Crea la version 0 a partir del arreglo inicial
    void build(vl& a){
        roots.pb(build(0, sz(a), a));
    }
};

void solver(){
    int n; cin>>n;
    vl a(n);
    for0(i,n) cin>>a[i];

    // Inicializar
    segTree st;
    st.init(n);
    st.build(a);

    // Updates
    int ver, idx; ll val; cin>>ver>>idx>>val;
    st.update(ver, idx, val);

    // Calc
    int ver, l, r; cin>>ver>>l>>r;
    cout<<st.calc(ver, l, r).val<<endl;

    // Calc between
    int v1, v2, l, r; cin>>v1>>v2>>l>>r;
    cout<<st.calcBetween(v1, v2, l, r).val<<endl;

    // Copy any version
    int ver; cin>>ver;
    st.roots.pb(st.roots[ver]);

    // Valor de la ultima version
    int lastV = sz(st.roots) - 1;
}
```

2.12 Segment tree sparse

```
// 0-index
// Implementacion ejemplo con suma
// Recordar que si los indices son muy grandes, cambiar
    SZ, L, R por ll
// Tener cuidado con el valor neutro ya que es comun que
    este sea el retorno en algunas calc
// Complejidad temporal: O(lg(SZ)) para updates y calcs.
// Complejidad espacial: O(n*lg(SZ))

const int SZ = 1 << 17; // Techo del log2 del maximo
    indice

template<class T>
T neutro(){
    return 0; // Change
}

template<class T>
T oper(T a, T b){
    if(a == neutro<T>()) return b;
    if(b == neutro<T>()) return a;
    return a + b; // Change
}

template<class T> struct node {
  T val = neutro<T>();
    node<T>* c[2];
  node() { c[0] = c[1] = NULL; }

  void upd(int ind, T v, int L = 0, int R = SZ-1){ //
      asignar v
    if(L == ind && R == ind){
            val = v;
            return;
        }
    int M = (L+R) / 2;
    if(ind <= M){
      if(!c[0]) c[0] = new node();
      c[0]->upd(ind,v,L,M);
    }
      else{
      if(!c[1]) c[1] = new node();
      c[1]->upd(ind,v,M+1,R);
    }
        T lv = c[0] ? c[0]->val : neutro<T>();
        T rv = c[1] ? c[1]->val : neutro<T>();
    val = oper(lv, rv);
  }

    // [lo, hi]
  T calc(int lo, int hi, int L = 0, int R = SZ-1){
    if(hi < L || R < lo) return neutro<T>();
    if(lo <= L && R <= hi) return val;

    int M = (L+R) / 2;
        T lv = c[0] ? c[0]->calc(lo, hi, L, M) : neutro<
            T>();
        T rv = c[1] ? c[1]->calc(lo, hi, M+1, R) :
            neutro<T>();
    return oper(lv, rv);
  }

    // for 2D segtree
  void UPD(int ind, node* c0, node* c1, int L = 0, int R
      = SZ-1){
    if(L != R){
      int M = (L+R) / 2;
      if(ind <= M){
        if(!c[0]) c[0] = new node();
        c[0]->UPD(ind, c0 ? c0->c[0] : NULL, c1 ? c1->c
            [0] : NULL, L, M);
      }
          else{
        if(!c[1]) c[1] = new node();
        c[1]->UPD(ind, c0 ? c0->c[1] : NULL, c1 ? c1->c
            [1] : NULL, M+1, R);
      }
    }
    val = oper((c0 ? c0->val : neutro<T>()), (c1 ? c1->
        val : neutro<T>()));
  }
};

void solver(){
    // Inicializar
    node<ll> st;

    // Update
    int idx; ll val;
    st.upd(idx, val);

    // Calc
    int lo, hi; cin>>lo>>hi;
    ll ans = st.calc(lo, hi);
    if(ans == neutro<ll>()) cout<<0<<endl; // Rango sin
        valores asignados
    else cout<<ans<<endl;
}
```

2.13 Segment tree sparse lazy

```
// 0-index
// Implementacion ejemplo con suma
// Recordar que si los indices son muy grandes, cambiar
    SZ, L, R, etc, por ll
// Tener cuidado con el valor neutro ya que es comun que
    este sea el retorno en algunas calc
// Complejidad: O(lg(SZ)) para updates y calcs.

const int SZ = 1 << 30; // Techo del log2 del maximo
    indice
const ll neutro = LLONG_MAX - 1; // Change
const int LEN = 1e7; // tope de nodos totales (usar
    aprox lg(SZ) * q * 2)

struct node {
    ll val = neutro, lazy = neutro;
    int c[2] = {-1, -1};
```

```
};
class segTree {
private:
    vector<node> tree;
    int idx = 0;
    int root;

    // Para query suma y update assign
    ll updateOp(ll a, ll b, ll len){
        if(b == neutro) return a;
        return b * len;
    }
    ll calcOp(ll a, ll b){
        if(a == neutro) return b;
        if(b == neutro) return a;
        return a + b; // Change
    }
    void applyUpdOp(ll &a, ll b, ll len){
        a = updateOp(a, b, len);
    }

    int newNode(){
        tree[idx] = node();
        return idx++;
    }

    void propagate(int v, int tl, int tr){
        if(tr - tl == 1) return;
        int tm = (tl + tr) / 2;
        if(tree[v].c[0] == -1) tree[v].c[0] = newNode();
        if(tree[v].c[1] == -1) tree[v].c[1] = newNode();

        int L = tree[v].c[0], R = tree[v].c[1];
        applyUpdOp(tree[L].lazy, tree[v].lazy, 1);
        applyUpdOp(tree[L].val, tree[v].lazy, tm - tl);
        applyUpdOp(tree[R].lazy, tree[v].lazy, 1);
        applyUpdOp(tree[R].val, tree[v].lazy, tr - tm);
        tree[v].lazy = neutro;
    }

    // [l, r)
    void update(int v, int l, int r, ll val, int tl, int
         tr){
        if(r <= tl || tr <= l) return;
        if(l <= tl && tr <= r){
            applyUpdOp(tree[v].lazy, val, 1);
            applyUpdOp(tree[v].val, val, tr - tl);
            return;
        }
        propagate(v, tl, tr);
        int tm = (tl + tr) / 2;
        update(tree[v].c[0], l, r, val, tl, tm);
        update(tree[v].c[1], l, r, val, tm, tr);
        tree[v].val = calcOp(tree[tree[v].c[0]].val,
            tree[tree[v].c[1]].val);
    }

    // [l, r)
    ll calc(int v, int l, int r, int tl, int tr){
        if(r <= tl || tr <= l) return neutro;
        if(l <= tl && tr <= r) return tree[v].val;
        propagate(v, tl, tr);
        int tm = (tl + tr) / 2;
        ll lv = tree[v].c[0] != -1 ? calc(tree[v].c[0],
            l, r, tl, tm) : neutro;
        ll rv = tree[v].c[1] != -1 ? calc(tree[v].c[1],
            l, r, tm, tr) : neutro;
        return calcOp(lv, rv);
    }

public:
    void init(){
        tree.assign(LEN, node());
        idx = 0;
        root = newNode();
    }
    void update(int l, int r, ll val){ update(root, l, r
        , val, 0, SZ); }
    ll calc(int l, int r){ return calc(root, l, r, 0, SZ
        ); }
};

void solver(){
    // Inicializar
    segTree st; st.init();

    // Update
    int l, r; ll val;
    st.update(l, r, val);

    // Calc
    int l, r; cin>>l>>r;
    ll ans = st.calc(l, r);
    cout<<(ans == neutro ? 0 : ans)<<endl;
}
```

2.14 Segment tree 2D

```
/*
- Procurar que las dimensiones n, m sean <= 1000, si no,
     posible MLE
  ya que la memoria es logm * logn * m * n. En ese caso,
       usar Sparse2D.
- Update de asignacion.
*/

struct node{ // Change
    ll val;
};

class segTree2D {
private:
    int size_n, size_m; // tree[n][m], n es 'y', m es 'x
    '

    vector<vector<node>> tree;

    node neutro = {LLONG_MAX - 1}; // Change

    node calcOp(node a, node b){ // Change
        if(a.val == neutro.val) return b;
        if(b.val == neutro.val) return a;
        node ans = {a.val + b.val};
        return ans;
    }


    // O(lg(n)*lg(m))
    void update_x(int y, int x, ll val, int yv, int ytl,
         int ytr, int xv, int xtl, int xtr){
        if(xtr - xtl == 1){
            if(ytr - ytl == 1) tree[yv][xv] = {val};
            else tree[yv][xv] = calcOp(tree[2*yv + 1][xv
                ], tree[2*yv + 2][xv]);
        }
        else{
            int xtm = (xtl + xtr) / 2;
            if(x < xtm) update_x(y, x, val, yv, ytl, ytr
                , 2*xv + 1, xtl, xtm);
            else update_x(y, x, val, yv, ytl, ytr, 2*xv
                + 2, xtm, xtr);

            tree[yv][xv] = calcOp(tree[yv][2*xv + 1],
                tree[yv][2*xv + 2]);
        }
    }

    void update_y(int y, int x, ll val, int yv, int ytl,
         int ytr){
        if(ytr - ytl != 1){
            int ytm = (ytl + ytr) / 2;
            if(y < ytm) update_y(y, x, val, 2*yv + 1,
                ytl, ytm);
            else update_y(y, x, val, 2*yv + 2, ytm, ytr)
                ;
        }

        update_x(y, x, val, yv, ytl, ytr, 0, 0, size_m);
    }

    // O(lg(n)*lg(m))
    // [yl, yr), [xl, xr)
    node calc_x(int xl, int xr, int yv, int xv, int xtl,
         int xtr){
        if(xtl >= xr || xl >= xtr) return neutro;
        if(xtl >= xl && xtr <= xr) return tree[yv][xv];

        int xtm = (xtl + xtr) / 2;
        node m1 = calc_x(xl, xr, yv, 2*xv + 1, xtl, xtm)
            ;
        node m2 = calc_x(xl, xr, yv, 2*xv + 2, xtm, xtr)
            ;
        return calcOp(m1, m2);
    }

    node calc_y(int yl, int yr, int xl, int xr, int yv,
        int ytl, int ytr){
        if(ytl >= yr || yl >= ytr) return neutro;
        if(ytl >= yl && ytr <= yr) return calc_x(xl, xr,
            yv, 0, 0, size_m);

        int ytm = (ytl + ytr) / 2;
        node m1 = calc_y(yl, yr, xl, xr, 2*yv + 1, ytl,
            ytm);
        node m2 = calc_y(yl, yr, xl, xr, 2*yv + 2, ytm,
            ytr);
        return calcOp(m1, m2);
    }

    void build_x(vector<vector<node>>& a, int yv, int
        ytl, int ytr, int xv, int xtl, int xtr){
        if(xtr - xtl == 1){
            if(ytr - ytl == 1){
                if(ytl < sz(a) && xtl < sz(a[ytl])) tree
                    [yv][xv] = {a[ytl][xtl]};
            }
            else tree[yv][xv] = calcOp(tree[2*yv + 1][xv
                ], tree[2*yv + 2][xv]);
        }
        else{
            int xtm = (xtl + xtr) / 2;
            build_x(a, yv, ytl, ytr, 2*xv + 1, xtl, xtm)
                ;
            build_x(a, yv, ytl, ytr, 2*xv + 2, xtm, xtr)
                ;

            tree[yv][xv] = calcOp(tree[yv][2*xv + 1],
                tree[yv][2*xv + 2]);
        }
    }

    void build_y(vector<vector<node>>& a, int yv, int
        ytl, int ytr){ // O(n)
        if(ytr - ytl != 1){
            int ytm = (ytr + ytl) / 2;
            build_y(a, 2*yv + 1, ytl, ytm);
            build_y(a, 2*yv + 2, ytm, ytr);
        }

        build_x(a, yv, ytl, ytr, 0, 0, size_m);
    }

public:
    void init(int n, int m){
        size_n = 1, size_m = 1;
        while(size_n < n) size_n *= 2;
```

```
        while(size_m < m) size_m *= 2;
        tree.assign(2*size_n, vector<node>(2*size_m,
            {0}));
        // build(a, 0, 0, size_n);
    }

    void update(int y, int x, ll val){
        update_y(y, x, val, 0, 0, size_n);
    }

    node calc(int y1, int yr, int x1, int xr){
        return calc_y(y1, yr, x1, xr, 0, 0, size_n);
    }

    void build(vector<vector<node>>& a){
        build_y(a, 0, 0, size_n);
    }
};

void solver(){
    int n, m; cin>>n>>m;
    vector<vector<node>> a(n, vector<node>(m));
    for0(i,n){
        for0(j,m){
            cin>>a[i][j].val;
        }
    }

    // Inicializar
    segTree2D st;
    st.init(n, m);
    st.build(a);

    // Updates
    int y, x; ll val; cin>>y>>x>>val;
    st.update(y, x, val);

    // Calcular
    int y1, x1, y2, x2; cin>>y1>>x1>>y2>>x2;
    cout<<st.calc(y1, y2, x1, x2).val<<endl;
}
```

2.15 Segment tree 2D sparse

```
#include "./SegTree_sparse.h"

/*
- 0-index
- Copiar el segtree Sparse
- Para cambiar la operacion modificar el sparse
- Update de asignacion
- Complejidad temporal: O(log^2(SZ)) para updates y
    calcs
- Complejidad espacial: O(n*log^2(SZ)) (Puede ser
    necesario comprimir ambas coordenadas)
*/

template<class T> struct Node {
  node<T> seg;
    Node* c[2];
  Node() { c[0] = c[1] = NULL; }

  void upd(int x, int y, T v, int L = 0, int R = SZ-1){
      // Asigna v en (x,y)
    if(L == x && R == x){
            seg.upd(y,v);
            return;
        }
    int M = (L+R) / 2;
    if(x <= M){
      if(!c[0]) c[0] = new Node();
      c[0]->upd(x,y,v,L,M);
    }
        else{
      if (!c[1]) c[1] = new Node();
      c[1]->upd(x,y,v,M+1,R);
    }
    seg.UPD(y, c[0] ? &c[0]->seg : NULL, c[1] ? &c[1]->
        seg : NULL);
  }

    // Query rectangle [(x1, y1), (x2, y2)]
  T calc(int x1, int x2, int y1, int y2, int L = 0, int
      R = SZ-1){
    if(x1 <= L && R <= x2) return seg.calc(y1,y2);
    if(x2 < L || R < x1) return neutro<T>();

        int M = (L+R) / 2;
        T lv = c[0] ? c[0]->calc(x1, x2, y1, y2, L, M) :
            neutro<T>();
        T rv = c[1] ? c[1]->calc(x1, x2, y1, y2, M+1, R)
            : neutro<T>();
    return oper(lv, rv);
  }
};

void solver(){
    // Inicializar
    Node<ll> st;

    // Update
    int x, y; ll val;
    st.upd(x, y, val);

    // Calc
    int x1, y1, x2, y2; cin>>x1>>y1>>x2>>y2;
    ll ans = st.calc(x1, x2, y1, y2);
    if(ans == neutro<ll>()) cout<<0<<endl; // Rango sin
        valores asignados
    else cout<<ans<<endl;
}
```

2.16 Fenwick Tree

```
/*
Fenwick tree (Binary Indexed Tree), variante "1-indexada
    ":

Que guarda cada nodo t[p]: la suma de un rango de
    posiciones
determinado por los bits de p (no es simplemente "el
    valor en p").
Por eso nunca se lee t[p] directamente, siempre a traves
     de get().

Complejidad: O(log N) por update y por get.
Soporta operaciones con cualquier operacion asociativa
    CON INVERSA
(poder "deshacer" un update). Ejemplos: suma (inversa =
    resta), xor
(inversa = si mismo), conteo (caso particular de suma).
*/

template<int N> struct FenW {
    ll t[N + 1]; // indices validos: [1, N] (la posicion
        0 no se usa)

    FenW(){
        fill(t + 1, t + N + 1, 0ll);
    }

    // Suma x en la posicion p (point update).
    void update(int p, ll x){
        for(; p <= N; p += p & (-p)){
            t[p] += x;
        }
    }

    // Suma acumulada de las posiciones [1, p] (prefix
        query).
    // get(0) = 0;
    ll get(int p){
        ll ret = 0;
        for(; p > 0; p -= p & (-p)){
            ret += t[p];
        }
        return ret;
    }

    // Suma acumulada de las posiciones [l, r] (range
        query).
    ll get(int l, int r){ return get(r) - get(l - 1); }
};

// How to use
void solver(){
    // Puedes crearlo con un valor fijo, o variable
        constante
    FenW<10> A;

    // Update
    int p; ll val; cin>>p>>val;
    A.update(p, val);

    // Calcular rango
    int l, r; cin>>l>>r;
    cout<<A.get(l, r);

    // Calcular prefix
    int p; cin>>p;
    cout<<A.get(p);
}
```

2.17 Sparse Table

```
/*
- 0-index.
- Para cualquier operacion asociativa e idempotente (
    oper(a,a) == a),
  como min, max, and, or, gcd.
- Cuidado con el neutro.
- Complejidad: calc: 0(1), build(nlog(n))
*/

struct rmq {
    ll neutro = LLONG_MAX - 1; // Change
    int n, lgn;
    vector<vl> st;
    inline ll oper(ll a, ll b){ return a < b ? a : b; }
        // Change

    void build(const vl& a){
        n = sz(a);
        lgn = 32 - __builtin_clz(n); // floor(log2(n))
        st.assign(lgn, vl(n));
        st[0] = a;
        for1(j,lgn)
            for(int i = 0; i + (1 << j) - 1 < n; i++)
                st[j][i] = oper(st[j-1][i], st[j-1][i +
                    (1 << (j-1))]);
    }

    ll calc(int l, int r){ // [l, r]
        if(l > r) return neutro;
        int j = 31 - __builtin_clz(r-l+1); // floor(log2
            (r-l+1))
        return oper(st[j][l], st[j][r - (1 << j) + 1]);
    }
};

void solver(){
    int n; cin>>n;
    vl a(n);
    for0(i,n) cin>>a[i];

    // Inicializar
    rmq st;
```

```
    st.build(a);

    // Calc
    int l, r; cin>>l>>r;
    cout<<st.calc(l, r)<<endl;
}
```

2.18 SQRT Decomposition

```
/*
- 0-index
- Complejidad: build O(n), update O(1), calc O(sqrt(n)).
- Ajustar el tamano del bloque a conveniencia.
- La complejidad de update depende de la complejidad de
    reversar el calculo.
- Ejemplo con suma en rango.

Trucos para hacer soluciones en sqrt(n) (no
    necesariamente usando la estructura):
- Queries de menor distancia a un nodo marcado, y
    queries de marcar nodos:
  Para un problema donde se tienen queries de hallar la
      menor distancia desde
  un nodo u hacia cualquier nodo marcado (ya sea en un
      arbol o matriz) y
  queries de marcar algun nodo, se puede resolver
      combinando 2 algoritmos:
  * Se crea un array con la distancia actual de cada
      nodo hacia todos los
    marcados inicialmente (si no hay ninguno se pone
        como inf).
  * Cuando se marca un nodo, agregarlo a una lista de
      ultimos nodos marcados.
  * Cuando dicha lista llega a sqrt(n) nodos, se hace un
       BFS empezando
    simultaneamente en cada uno. Se halla la distancia
        de todos los nodos a
    los de esta lista y se revisa si esta distancia es
        menor a la que se tenia
    guardada antes (y se actualiza). Al final se limpia
        la lista.
  * Cuando se pregunta la menor distancia de u hacia
      cualquier nodo marcado,
    primero se revisa la distancia hacia cualquiera de
        los nodos ya procesados
    y despues se revisa la distancia de u hacia
        cualquiera de los nodos que
    estan actualmente en la lista (procurar que esa
        distancia se pueda revisar
    en O(1) o de alguna forma muy eficiente).
- A cada nodo se le da una letra, hallar minima
    distancia entre dos nodos con
  la misma letra:
  * Si una letra k esta en <= sqrt(n) nodos, se va a
      traves de cada par nodos
    con esta letra y se calcula su distancia (hacerlo en
         O(1)).
  * Si una letra k esta en mas de sqrt(n) nodos, se hace
       un BFS empezando
    simultaneamente en cada uno para hallar la distancia
        .
- Si me dan numeros positivos (o longitudes) diferentes,
     y me dicen que la suma
  de estos es maximo n, entonces como maximo tengo sqrt(
      n) numeros diferentes.
  A veces eso se puede usar para resolver ejercicios en
      O(sqrt(n)) o O(n*sqrt(n)).
*/

struct sqrtDecomp{
    int n, blen;
    vl a, b;
    sqrtDecomp(){}
    sqrtDecomp(vl& arr): n(sz(arr)), blen(sqrt(n)+1), a(
        arr), b(blen){
        for0(i,n) b[i/blen] += a[i];
    }

    void update(int pos, ll val){
        b[pos/blen] += val - a[pos];
        a[pos] = val;
    }

    ll calc(int l, int r){ // [l, r]
        ll sum = 0;
        int bl = l/blen, br = r/blen;
        if(bl == br) forlr(i,l,r) sum += a[i]; // Si l y
            r estan en el mismo bloque
        else{
            forlr(i, l, blen*(bl+1)-1) sum += a[i]; //
                Indices en el bloque izquierdo
            forlr(i, blen*br, r) sum += a[i]; // Indices
                en el bloque derecho
            forlr(i, bl+1, br-1) sum += b[i]; // Bloques
                centrales
        }
        return sum;
    }
};

void solver(){
    int n; cin>>n;
    vl a(n);
    for0(i,n) cin>>a[i];

    // Inicializar
    sqrtDecomp sq(a);

    // Update
    int pos; ll val; cin>>pos>>val;
    sq.update(pos, val);

    // Calc
    int l, r; cin>>l>>r;
    cout<<sq.calc(l, r)<<endl;
}
```

2.19 Mo's Algorithm

```
/*
- Queries [l, r]
- Complejidad: O((n+q) * sqrt(n) * (complejidad add +
    del))
- Se ordenan las querys por bloque al que pertenece l y
    por
  r creciente si el bloque es par y r decreciente si el
  bloque es impar.
- Se usa principalmente para ejercicios en los cuales la
  funcion de consulta es facil de actualizar agregando o
  eliminando un elemento a la vez pero dificil de
      combinar
  entre dos rangos disjuntos, como contar la cantidad de
  valores diferentes en un rango.
- Si para hacer las actualizaciones necesito hacer algun
  tipo de consultas en rango, sqrt decomposition es la
      mejor
  opcion ya que no aumenta la complejidad. Esto debido a
      que
  las updates son O(1) y las consultas O(sqrt(n)), y al
      ser
  q consultas, se mantiene el factor de q*sqrt(n).
- Hallar el MEX en rango se puede hacer con Mo + SQRT (
    si
  bien tambien se ppuede resolver con segTree).
- Se puede variar con que los bloques se definan por r
    en
  lugar de l, que en el lado que se extiende solo se
      agregue
  y nunca se quite (siempre ++r o --l), que en el lado
      del
  bloque siempre se eliminen los valores y se vuelvan a
  agregar en cada query (por ejemplo para queries de
      solo
  tomar aristas en cierto rango).
- Consultas en subarbol:
  Se halla el rango de cada subarbol con Euler Tour
      Tecnique
  y se utiliza este para las queries, al pasar por la
  posicion i, no se agrega ni elimina directamente la
  posicion i sino la posicion orden[i]. (Euler Tour es
  1-index asi que Inicializar con l = 1 y r = 0).
- Consultas en un camino de u hacia v en un arbol:
  * De manera similar, se usa Euler Tour pero esta vez
      cada
    nodo se copia en el array tanto al iniciar como al
        terminar
    la visita de su subarbol (estos puntos de inicio y
        de fin
    son marcados como su rango).
  * Para cada query [u, v], primero se asegura que
    ran[u].fi sea <= ran[v].fi.
  * Si lca(u,v) es u, se hace la query [ran[u].fi, ran[v
      ].fi].
    Si no, se hace la query [ran[u].se, ran[v].fi] y
    [ran[lca].fi, ran[lca].fi].
  * Igual que en la aplicacion anterior, al pasar los
      indices
    por la posicion i, no se usa el valor de i sino de
        orden[i].
  * Como cada valor se encuentra 2 veces en el array
      orden, si
    en el rango que se analiza el nodo i solo esta una
        vez,
    entonces se tiene en cuenta, pero si en el rango el
        nodo i
    esta 0 o 2 veces, entonces no se tiene en cuenta.
*/

struct query {
    int l, r, idx;
};
int blen; // blen = sqrt(n)
bool cmp(query u, query v){
    int x = u.l / blen;
    if(x != v.l / blen) return x < v.l / blen;
    return x&1 ? u.r > v.r : u.r < v.r; // Bloque impar
        r decreciente, par r creciente
}

// Estas funciones se usan para hacer el proceso
    requerido al
// agregar o eliminar valores y contestar queries
void add(int idx){

}
void del(int idx){

}
int getAns(){

}

void solver(){
    // Leer y ordenar las queries
    int q; cin>>q;
    blen = sqrt(n); // Raiz del tamano del arreglo
    vector<query> qs(q);
    for0(i,q){
        cin>>qs[i].l>>qs[i].r;
        qs[i].idx = i;
    }
    sort(all(qs), cmp);

    // Procesar
    vi ans(q);
```

```
    int l = 0, r = -1;
    for0(i,q){
        while(r < qs[i].r) add(++r);
        while(l > qs[i].l) add(--l);
        while(r > qs[i].r) del(r--);
        while(l < qs[i].l) del(l++);
        ans[qs[i].idx] = getAns();
    }
}
```

2.20 Rectangular partial sums

```
// 1-index.
// Complejidad: o(n^2).
const int maxn = 1001;
int a[maxn][maxn];
ll s[maxn][maxn];

void solver(){
    int n, m; cin>>n>>m;

    // La formula para calcular cada termino es:
    // S[i][j] = A[i][j] + S[i-1][j] + S[i][j-1] - S[i
        -1][j-1]
    for1(i,n){
        for1(j,m){
            cin>>a[i][j];
            s[i][j] = (ll)a[i][j] + s[i-1][j] + s[i][j
                -1] - s[i-1][j-1];
        }
    }

    // Para consultar cada rectangulo:
    // y2 >= y1, x2 >= x1.
    // rectangulo = S[y2][x2] - S[y1-1][x2] - S[y2][x1
        -1] + S[y1-1][x1-1]
    int q; cin>>q;
    while(q--){
        int x1, y1, x2, y2; cin>>x1>>y1>>x2>>y2;
        cout<<s[y2][x2] - s[y1-1][x2] - s[y2][x1-1] + s[
            y1-1][x1-1]<<endl;
    }
}
```

2.21 Kadane

```
// Calcula el valor del subsegmento con mayor suma para
    un array a con numeros positivos y negativos
// Complejidad: O(n)
int kadane(vl& a){
    int n = sz(a);
    ll cur = 0, best = -INF;
    for0(i,n){
        cur = max(a[i], cur + a[i]); // cur guarda el
            valor mayor de un subsegmento terminado en
            i
        best = max(best, cur); // Actualiza el resultado
    }

    return best;
}

// Se calcula el valor del subsegmento con mayor suma
    para cada subsegmento [l, r] existente
// Complejidad: O(n^2), debido a que hay n*(n+1) / 2
    subsegmentos
void kadaneAll(vl& a, vector<vl>& ans){
    int n = sz(a);
    for0(l,n){ // Calculo para los subsegmentos
        iniciados en l
        ll cur = 0, best = -INF;
        for(int r = l; r < n; r++){
            cur = max(a[r], cur + a[r]); // cur guarda
                el valor del mayor subsegmento
                terminado en r
            best = max(best, cur); // Actualiza el
                resultado
            ans[l][r] = best;
        }
    }
}
```

3 Graphs

3.1 Dijkstra

```
/*
- Complejidad O(m*log(n)).
Aplicaciones:
- Sirve para grafos con pesos negativos siempre y cuando
    no haya ciclos
  negativos, en ese caso usar Bellman-Ford. Pero si el
      grafo es grande,
  es propenso a dar TLE.
- Hallar las 'k' menores distancias (usando 'k' caminos
    diferentes, que
  pueden compartir aristas entre si, y donde cada camino
      puede visitar
  el mismo nodo varias veces). Complejidad O(m*k*log(n*k
      )):
  Se puede modificar el vector de distancias, haciendo
      que sea un vector
  de colas de mayor prioridad. Asi en cada nodo se
      almacenan las 'k'
  distancias menores. Al sacar un par nodo 'u' peso 'w1'
      de la cola de
  prioridad del dijkstra, se verifica si su peso mayor
      en su cola de
  prioridad es menor que 'w1', y en caso de que si, se
      continua con la
  siguiente iteracion. En caso de que no, para cada
      arista que va hacia
  'v' con peso 'w2', se verifica si este aun no tiene 'k
      ' elementos en
  su cola de prioridad y agregamos 'w1' + 'w2' tanto en
      su cola de
  prioridad como en la del dijkstra. En caso de que si
      tenga 'k'
  elementos, se revisa si 'w1' + 'w2' es menor que el
      valor mayor de la
  cola de prioridad de 'v', en caso de que si, se saca
      el valor mayor
  de la cola de prioridad de 'v' y se pone 'w1' + 'w2'
      tanto en la cola
  de prioridad de 'v' como en la del dijkstra.
- Potencial de un grafo (existe si y solo si no hay
    ciclos negativos):
  Si el grafo tiene pesos negativos, se halla una
      funcion p tal que para
  cada arista u->v, w(u->v) + p[u] - p[v] >= 0. (Si el
      grafo no tiene
  pesos negativos, p[i] = 0 para todo nodo 'i'):
  Se halla creando un nuevo vertice 's' y aristas que
      vayan desde 's' a
  todos los vertices del grafo con peso 0. Entonces se
      corre dijkstra
  desde 's' y su distancia hacia cada vertice sera el
      valor de la
  funcion p (todos estos seran <= 0 ya que hay una
      arista con pesos 0
  desde 's' hacia todos los nodos). Ahora se reemplaza
      el peso 'w' de
  cada arista u->v por 'w' + p[u] - p[v].
  Para saber la distancia desde 'u' hasta 'v' (la
      distancia real, que
  habria usando los pesos del grafo original) se corre
      el dijkstra desde
  'u' y la respuesta es d[v] - p[u] + p[v]
  Se puede usar para mejorar la complejidad del dijkstra
      cuando el grafo
  tiene pesos negativos si nos dan una funcion potencial
      valida de
  antemano o alguna forma de calcularla sin correr el
      dijkstra.
*/

typedef pair<ll, int> pli;
const int maxn = 1e5 + 1;
const ll inf = 1e15;
vector<vector<pli>> graph(maxn);

void dijkstra(int s, vl& dis, vi& pa){
    dis[s] = 0;
    pqueue<pli, vector<pli>, greater<pli>> pq;
    pq.push({0, s});
    while(!pq.empty()){
        int u = pq.top().se; ll w1 = pq.top().fi; pq.pop
            ();
        if(dis[u] < w1) continue;
        for(pli &e : graph[u]){
            int v = e.se; ll w2 = e.fi;
            if(dis[v] > w1 + w2){
                dis[v] = w1 + w2;
                pa[v] = u;
                pq.push({dis[v], v});
            }
        }
    }
}

void solver(){
    int n, m, s; cin>>n>>m>>s;
    for(int i = 0; i < m; i++){
        int u, v; ll w; cin>>u>>v>>w;
        graph[u].pb({w, v});
        graph[v].pb({w, u});
    }
    vl dis(n+1, INFL);
    vi pa(n+1, -1);
    dijkstra(s, dis, pa);
    for1(i,n) cout<<"vertex: "<<i<<" distancia: "<<dis[i
        ]<<" padre: "<<pa[i]<<endl;
}
```

3.2 Bellman Ford

```
/*
- Para grafos dirigidos que pueden tener aristas de peso
    negativo.
- Complejidad O(n * m)
- Si hay un ciclo de peso negativo alcanzable desde s
    retorna 0. Si
  no, retorna 1 y la minima distancia desde s hacia
      cualquier nodo.
- Para saber si hay algun ciclo negativo no
    necesariamente alcanzable
  desde un vertice s, se inicializan todas las
      distancias en 0 y el
  proceso es el mismo: se hacen n-1 rondas y despues una
      extra donde
  si hay una mejora quiere decir que si hay ciclos
      negativos.
- Si queremos saber el camino de algun ciclo negativo (
    sabiendo que
  hay alguno), se debe tambien calcular el padre de cada
      nodo (si
  hay mejoria desde a hacia b, se dice que a es el padre
      de b). En el
  Bellman Ford retornamos algun vertice que en la ultima
```

```
      ronda haya
  reducido su distancia, luego viajamos hacia su padre n
      veces (ya que
  puede ser que el vertice que sufrio una reduccion no
      este dentro del
  ciclo sino despues de uno) y decimos que ese nodo al
      que se llega es
  'v'. Despues se va viajando desde 'v' yendo a sus
      antecesores hasta
  que volvamos a llegar a 'v' y finalmente se invierte
      ese recorrido.
- Para saber si hay un ciclo negativo en algun camino
    desde s hasta t,
  se corre el Bellman-Ford, se marcan los vertices para
      los cuales hay
  una mejora en la n-esima ronda y se corre un BFS con t
      como source,
  invirtiendo las aristas del grafo. Si algun vertice
      marcado es
  alcanzable desde t, quiere decir que si hay ciclos
      negativos en el
  camino desde s hasta t.
*/

struct Edge{
    int u, v; ll w;
};

bool bellmanFord(int s, int n, vector<Edge>& edges, vl&
    dis){
    dis[s] = 0;
    for1(i,n){ // Se hace un maximo de n-1 rondas
        bool cond = 0;
        for(Edge &e : edges){
            int u = e.u, v = e.v; ll w = e.w;
            if(dis[u] < INFL && dis[v] > dis[u] + w){
                if(i == n) return 0; // Hay ciclos de
                    peso negativo
                dis[v] = dis[u] + w;
                cond = 1; // Marcamos que en esta ronda
                    hubo al menos una mejora
            }
        }

        if(!cond) return 1;
    }

    return 1;
}

void solver(){
    int n, m; cin>>n>>m;
    vector<Edge> edges;
    for0(i,m){
        int u, v; ll w; cin>>u>>v>>w;
        edges.pb({u, v, w});
    }

    vl dis(n+1, INFL);
    bool cond = bellmanFord(source, n, edges, dis);
}
```

3.3 SPFA

```
/*
- Complejidad O(n * m)
- Retorna 0 si hay ciclos negativos. Si no, retorna 1 y
    halla la
  minima distancia desde s hacia cualquiuer nodo.
- Misma complejidad que Bellman-Ford, pero mejor en la
    practica.
- mcfx optimization:
  Tambien corre en O(n * m), pero suele ser mas rapido
      en la
  practica teniendo en cuenta:
  * Suele funcionar muy bien en grid graphs (grafos tipo
      cuadro).
  * No funciona bien en grafos estrella.
  Consiste en usar una deque en lugar de una queue.
      Antes de agregar
  cada vertice se revisa si este se ha procesado entre 2
      y sqrt(n)
  veces, en cuyo caso de pone al frente de la deque. De
      lo contrario
  se pone al final de la deque.
*/

typedef pair<ll, int> pli;
const int maxn = 1e4 + 1;
vector<vector<pli>> graph(maxn);

// Implementacion normal:
bool spfa(int s, int n, vl& dis){
    dis[s] = 0;
    vi count(n+1);
    vb in(n+1); in[s] = 1;
    queue<int> q; q.push(s);

    while(!q.empty()){
        int u = q.front(); q.pop();
        in[u] = 0; // Marca que el vertice ya no esta en
            la cola

        for(pli edge : graph[u]){
            int v = edge.se; ll w = edge.fi;

            if(dis[v] > dis[u] + w){
                dis[v] = dis[u] + w;
                if(!in[v]){ // Antes de meter un vertice
                    a la cola, revisa que no se este
                    ya
                    in[v] = 1; // Marca que el vertice
                        ahora esta en la cola
                    q.push(v);
                    count[v]++;
                    if(count[v] > n) return 0; // Si se
                        ha metido a la cola mas de n
                        veces -> ciclo negativo
                }
            }
        }
    }

    return 1; // No hay ciclos negativos
}

void solver(){
    int n, m; cin>>n>>m;
    for0(i,m){
        int u, v; ll w; cin>>u>>v>>w;
        graph[u].pb({w, v});
    }

    vl dis(n+1, INFL);
    bool cond = spfa(source, n, dis);
}
```

3.4 Floyd Warshall

```
/*
- Complejidad O(n^3)
- Sirve tanto para grafos dirigidos como no dirigidos
- Esta implementacion es 1-index
- Aplicacion afest Path:
  Se quiere maximizar la probabilidad de sobrevivir por
      algun camino
  y los valores son porcentajes.
  d[i][j] = max(d[i][j], d[i][k] * d[k][j]);
*/

const int maxn = 501;
int d[maxn][maxn]; // d[i][j] = Longitud de la ruta mas
    corta desde i hasta j

void floyd(int n){
    // Cada ciclo hasta k = x halla los costos minimos
        entre cada par de nodos
    // usando como nodos intermedios solo los primeros '
        x' nodos.
    for1(k,n){
        for1(i,n){
            for1(j,n){
                // Este if solo es necesario en caso de
                    hayan pesos negativos
                if(d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k
                        ][j]);
            }
        }
    }
}

void solver(){
    int n, m; cin>>n>>m;

    // Inicializar de la siguiente manera:
    // - Si i == j, la distancia es 0
    // - Si hay una arista entre i y j, la distancia
        toma ese valor
    // - De lo contrario inicializamos la distancia como
        infinito
    for1(i,n-1) forlr(j,i+1,n) d[i][j] = d[j][i] = INF;
    for0(i,m){
        int u, v, w; cin>>u>>v>>w;
        if(d[u][v] > w) d[u][v] = d[v][u] = w; //
            Cambiar en caso de grafo dirigido
    }

    floyd(n); // Si algun d[i][i] < 0, significa que hay
        un ciclo negativo
}
```

3.5 BFS 0-1

```
/*
- BFS para grafo donde todas las aristas tienen peso 0 o
    w (w > 0).
- Complejidad: O(n + m).
- Se asume que el peso de las aristas no se da como
    parte de la
  entrada sino que se puede hallar facilmente con una
      condicion
  segun sean u y v. En caso de que el peso se de en la
      entrada es
  mejor guardar el grafo como listas de pares.
*/

const int maxn = 1e5 + 1;
vi dis(maxn, INF);
vector<vi> graph(maxn);

void BFS(int s){
    dis[s] = 0;
    deque<int> q; q.push_front(s);
    while(!q.empty()){
        int u = q.front(); q.pop_front();
        for(int v : graph[u]){
            int add = 0;
            if(condicion) add++;
            if(dis[v] > dis[u] + add){
                dis[v] = dis[u] + add;
                if(!add) q.push_front(v);
                else q.pb(v);
            }
        }
```

```
    }
}
```

3.6 Find edges and vertexes in shortest paths

```
/*
- Aristas que pueden ser parte de un camino mas corto desde 's' a 't'
  (si hay pesos negativos, no deben haber ciclos negativos entre 's' y
  't'):
  Se corre BFS, Dijkstra o SPFA desde s y desde t (si el grafo es
  dirigido toca crear un grafo invertido para el dijkstra desde t),
  despues se recorre cada arista u->v y si la distancia de 's' a 'u',
  mas la distancia de 't' a 'v' mas el peso de esa arista es igual a
  la distancia de 's' a 't', en caso de que si, esa arista puede hacer
  parte de un camino mas corto de 's' a 't'.
  Si el grafo solo tiene pesos positivos, se puede armar un grafo con
  estas aristas se obtiene un DAG.
- Hallar las aristas y/o vertices por los cuales es obligatorio pasar
  para llegar de 's' a 't' de la manera mas eficiente posible (grafo no
  ponderado):
  Para los nodos: solo se toman en cuenta aquellos nodos 'u' donde
  dis1[u] + disn[u] == dis1[n] y se forman capas. Cada nodo 'v' hace parte
  de la capa dis1[v]. Entonces se revisa capa por capa y en aquellas que
  solo haya un nodo se dice que dichos nodos son obligatorios.
  Para las aristas es similar: se toman en cuenta solo aquellas aristas
  u->v donde dis1[u] + 1 + disn[v] == dis1[n]. Cada arista u->v hace parte
  de la capa dis1[u]. Se revisa capa por capa y en aquellas que solo haya
  una arista se dice que dichas aristas son obligatorias.
- Hallar las aristas y/o vertices por las cuales es obligatorio pasar para
  llegar de 's' a 't' de la manera mas corta posible (pesos estrictamente
  positivos) (tener cuidado en caso de que no se pueda llegar de 's' a 't'):
  Para las aristas:
  * Complejidad: O(n+m) del BFS mas la complejidad de armar el dag.
  * En el grafo original tambien se debe guardar el indice de las aristas.
  * Se arma el dag con las aristas que pueden estar en algun camino mas
    corto de 's' a 't'.
  * Se corre la funcion "findObligatory"
  Para los nodos: todos aquellos que hagan parte de alguna arista
  obligatoria se consideran obligatorios.
*/

typedef pair<ll, pii> plii;
const int maxn = 1e5 + 1;
vector<vector<plii>> graph(maxn);
vector<vii> dag(maxn);

void findObligatory(int s, int t, int n, vb& ans, vi& inDegree){
    vb vis(n+1); vis[s] = 1;
    queue<int> q; q.push(s);
    int roads = 1;
    while(!q.empty()){
        int u = q.front(); q.pop();
        if(u != t) roads += sz(dag[u]) - 1; // Aumentamos roads con el numero de salidas de u-1
        for(pii &e : dag[u]){
            int v = e.fi, idx = e.se;
            if(roads == 1) ans[idx] = 1; // Esa arista esta en todos los caminos

            if(vis[v]) roads--; // Si ya habia visitado v antes, disminuyo roads
            vis[v] = 1;
            inDegree[v]--; // Resto su numero de aristas de entrada
            if(!inDegree[v]) q.push(v); // Al visitar la ultima arista que va a 'v', entra en la cola
        }
    }
}

void solver(){
    int n, m, s, t; cin>>n>>m>>s>>t;
    for0(i,m){
        int u, v; ll w; cin>>u>>v>>w; // Ejemplo grafo con pesos
        graph[u].pb({w, {v, i}}); // Si es grafo dirigido, ponerlas tambien en el grafo inverso
        graph[v].pb({w, {u, i}});
    }

    // Se debe correr BFS, Dijsktra o SPFA para hallar ds (distancias desde s) y dt (distancias desde t)
    vl ds(n+1, INFL), dt(n+1, INFL);

    // Hallar las aristas que estan en algun camino mas corto de 's' a 't'
    vi inDegree(n+1); // Cantidad de aristas que llegan a cada nodo en el dag
    for1(i,n){
        for(plii &e : graph[i]){
            int v = e.se.fi, idx = e.se.se; ll w = e.fi;
            if(ds[i] + dt[v] + w == ds[t]){ // Esta arista puede estar (y en esa direccion)
                dag[i].pb({v, idx}); // Si todos los pesos son positivos, armar el dag
                inDegree[v]++;
            }
        }
    }

    // Hallar aristas obligatorias, solo si todos los pesos son positivos
    vb ans(m);
    findObligatory(s, t, n, ans, inDegree);
    for0(i,m) if(ans[i]) cout<<"La arista con indice "<<i<<" es obligatoria"<<endl;
}
```

3.7 LCA BFS

```
/*
- Construye el arbol de modo que cada nodo esta lo mas cerca posible
  de la raiz, mediante BFS. Desde este arbol se hacen querys de LCA.
- Sirve para 1-index y 0-index.
- Sirve para grafos no dirigidos y dirigidos.
*/

const int maxn = 2e5 + 1;
const int LOG = 19; // El logaritmo base 2 del numero maximo de nodos
vector<vi> graph(maxn), up(maxn, vi(LOG));
vi dep(maxn, -1);

void BFS(int s){ // O(m + nlog(n))
    dep[s] = 0;
    queue<int> q; q.push(s);
    while(!q.empty()){
        int u = q.front(); q.pop();
        for(int v : graph[u]){
            if(dep[v] != -1) continue;
            dep[v] = dep[u] + 1;
            up[v][0] = u;
            for1(j,LOG-1) up[v][j] = up[up[v][j-1]][j-1];
            q.push(v);
        }
    }
}

int LCA(int a, int b){ // log(n)
    if(dep[a] < dep[b]) swap(a, b);
    int k = dep[a] - dep[b];
    forn0(j,LOG) if(k & (1 << j)) a = up[a][j]; // Pone a y b al mismo nivel
    if(a == b) return a;
    forn0(j,LOG){
        if(up[a][j] != up[b][j]){
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}

void solver(){
    // Construir
    int n, m, root; cin>>n>>m>>root;
    for0(i,m){
        int u, v; cin>>u>>v;
        graph[u].pb(v);
        graph[v].pb(u);
    }
    BFS(root);

    cout<<LCA(u, v)<<endl; // Query
}
```

3.8 DSU

```
/*
- 1-index, para 0-index cambiar en la inicializacion "comp(n-1)"
  por "comp(n)" y en lugar de llamarlo como "DSU uf(n+1)",
  llamarlo como "DSU uf(n)".
- Compresion de caminos y union-by-size.
- Al hacer join(a, b), retorna 0 si 'a' y 'b' ya estaban en el mismo
  componente, retorna 1 y actualiza en caso contrario.
- Complejidad: Inicializacion O(n), find y join O(alpha(n)).
*/

struct DSU {
    vi p, len; int comp; // p es el lider de cada nodo
    DSU(int n) : p(n), len(n, 1), comp(n-1) { iota(all(p), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
```

```
    bool join(int a, int b){
        if((a = find(a)) == (b = find(b))) return 0;
        if(len[a] < len[b]) swap(a, b);
        p[b] = a; len[a] += len[b]; --comp;
        return 1;
    }
};

void solver(){
    int n, m; cin>>n>>m;
    DSU uf(n+1);
    for0(i,m){
        int u, v; cin>>u>>v;
        uf.join(u, v);
    }
    // Preguntar si 2 vertices estan en el
        mismocomponente
    cout<<(uf.find(u) == uf.find(v) ? "Mismo componente"
         : "Diferente componente")<<endl;

    cout<<uf.comp<<endl; // Numero de componentes tras
        agregar todas las aristas
}
```

3.9 DSU bipartition test

```
/*
- Se le van agregando aristas a un grafo y avisa cual es
    la
  primer arista que al agregarse hace que el grafo deje
      de ser
  bipartito.
- Para realizarlo se duplican los nodos, de modo que
    cada nodo
  nodo 'u' se duplica siendo tambien el nodo 'u+n' (
      siendo 'n'
  el total de nodos del grafo original). Entonces en
      este
  grafo el nodo 'u' representa al 'u' original en el
      lado '0'
  y 'u+n' representa al 'u' original en el lado '1' del
  bipartito.
  Cada que se agrega una arista u->v en el grafo
      original, en
  el grafo duplicado se crean aristas u->v+n y u+n->v.
  Si en algun punto, cuando se va a agregar una arista u
      ->v en
  el grafo original, 'u' y 'v' estan en el mismo
      componente
  del grafo duplicado, quiere decir que hay una conexion
       entre
  nodos del mismo color, asi que el grafo deja de ser
  bipartito desde esa arista.
- En cualquier punto (mientras el grafo aun pueda ser
  considerado bipartito), si dos nodos 'u' y 'v' (del
      grafo
  duplicado) estan en el mismo componente, quiere decir
      que
  en el grafo original son del mismo color. Si 'u' y 'v+
      n'
  estan en el mismo componente entonces son de diferente
      .
  Si no se cumple ninguna, quiere decir que no estan en
      el
  mismo componente en el grafo original y por ende
      podrian
  ser de cualquier color.
*/

struct DSU {
    vi p, len; int comp;
    DSU(int n) : p(n), len(n, 1), comp(n-1) { iota(all(p
        ), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find
        (p[x]); }
    bool join(int a, int b){
        if((a = find(a)) == (b = find(b))) return 0;
        if(len[a] < len[b]) swap(a, b);
        p[b] = a; len[a] += len[b]; --comp;
        return 1;
    }
};

void solver(){
    int n, m; cin>>n>>m;

    DSU uf(2*n + 1);
    int badIdx = 0;
    for1(i,m){
        int u, v; cin>>u>>v;
        if(badIdx) continue;

        if(uf.find(u) == uf.find(v)) badIdx = i; //
            Primer arista que rompe el bipartito
        else{
            uf.join(u, v+n);
            uf.join(u+n, v);
        }
    }

    cout<<"La primera arista que rompe el bipartito es:
        "<<badIdx<<endl;
}
```

3.10 DSU rollback

```
/*
- 1-index.
- Permite hacer borrado de aristas, con el orden de una
    pila.
- La complejidad es la igual a la del DSU normal, en
    este
  caso usando union-by-size sin compresion.
Funcionamiento:
- Cuando se agrega una arista u->v, si 'u' y 'v' ya
    estaban
  en el mismo componente esta se ignora. De lo contrario
      se
  agrega al DSU y a la pila de aristas.
- Se hacen checkpoints para guardar estados del DSU, asi
  cuando se quiere hacer rollback se restaura el DSU
      hasta el
  ultimo checkpoint y se borra este.
- checkpoint guarda la cantidad de operaciones reales
    que se
  han hecho en el DSU hasta ahora. Quiere decir que al
      hacer
  join entre dos nodos que ya estan en el mismo
      componente,
  esta operacion no se cuenta.
- Si en los rollback se quiere borrar la arista
  inmediatamente anterior, es posible que esta haya sido
  ignorada ya que conectaba dos nodos que ya estaban en
      el
  mismo componente.
  Para manejar este caso se hace checkpoint cada que se
  agrega una arista al grafo.
Aplicaciones:
- Tambien se puede usar para cualquier otro tipo de DSU
    como
  revisar si un grafo es bipartito.
*/

struct DSURollback {
    vi p, len;
    stack<pii> st; stack<int> save;
    int comps;

    DSURollback(int n) : p(n), len(n, 1), comps(n-1) {
        iota(all(p), 0); }
    int find(int x){
        while(p[x] != x) x = p[x];
        return x;
    }
    bool join(int a, int b){
        a = find(a); b = find(b);
        if(a == b) return 0;
        if(len[a] < len[b]) swap(a, b);
        p[b] = a; len[a] += len[b]; comps--;
        st.push({a, b});
        return 1;
    }
    void checkpoint() { save.push(sz(st)); }
    void rollback(){ // Deshace hasta el ultimo
        checkpoint
        if(save.empty()) save.push(0); // Si no habia
            checkpoint, borrar todo
        while(sz(st) > save.top()){
            auto [a, b] = st.top(); st.pop();
            len[a] -= len[b]; p[b] = b; comps++;
        }
        save.pop();
    }
};

void solver(){
    int n, q; cin>>n>>q;
    DSURollback uf(n+1);
    for0(i,q){
        int op; cin>>op;
        if(op == 1){ // Agregar arista
            int u, v; cin>>u>>v;
            uf.join(u, v);
        }
        else if(op == 2) uf.checkpoint(); // Agregar
            guardado
        else if(op == 3) uf.rollback(); // Hacer
            rollback al ultimo guardado
        else cout<<uf.comps<<endl; // Query -> Numero
            actual de componentes del grafo
    }
}
```

3.11 Dynamic connectivity offline

```
/*
IMPORTAR DSU ROLLBACK
- 1-index.
- Complejidad: O(Q*log(Q)*log(n)) tiempo, O(n + Q)
    memoria
  (Q = operaciones).
- Dynamic connectivity offline: se guardan todas las
    operaciones
  de agregar arista, borrar arista y las queries.
  process() responde las consultas en orden.
  * add(u, v): agrega la arista u->v (no debe estar ya
      presente).
  * remove(u, v): borra la arista u->v (debe estar
      presente).
  * query(): imprime el numero de componentes en ese
      momento.
    + Si se quiere otro tipo de consulta, se debe
        cambiar la
      informacion que se esta imprimiendo en el primer
          if dentro
      de la funcion "go".
    + Asi mismo, se podrian hacer varios tipos de
        queries,
      agregando las necesarias en "enum" y en el primer
          if de la
      funcion "go".
    + Si se quieren queries que pregunten informacion de
        algun
```

```
      nodo o par de nodos, se cambia "query()" por "
          query(u)" o
      "query(u, v)".
- mt[i] es el emparejamiento de la operacion i:
  * ADD: indice de su DEL (operacion en que se elimina)
       (sz(q) si
    nunca se elimina)
  * DEL: indice de su ADD (operacion en que se agrega)
  * QUERY: -1.
- go(s, e) al entrar, el DSU ya tiene todas las aristas
    vivas
  durante todo el rango [s, e). Para cada mitad [s, m) y
       [m, e)
  (con m = (s+e)/2) se agregan las aristas que cubren
       esa mitad
  completa (y no todo [s, e)), se hacen los llamados
       recursivos
  a cada mitad y se hace rollback.
EXTRAS
- No maneja aristas multiples. Para hacerlo habria que
    agregar el
  parametro "idx" a las funciones "add" y "remove", y
       que la
  clave del mapa "prv" ahora no solo incluya los 2
       vertices sino
  tambien "idx". Tambien habria que ajustar el solver
       para que se
  guarden las aristas como un mapa de stack, asi cuando
       se quiera
  eliminar una arista se busca la ultima vez que se
       agrego y este
  sea el parametro "idx" para la funcion "delete" (
       ademas de
  eliminar esta ultima aparicion en la pila).
*/

enum {ADD, DEL, QUERY}; // Agregar mas operaciones de
    consulta de ser necesario
struct Query { int type, u, v; };

struct DynCon {
    vector<Query> q; DSURollback uf;
    vi mt; map<pii,int> prv;
    DynCon(int n) : uf(n+1) {}
    void add(int u, int v){
        if(u > v) swap(u, v);
        q.pb({ADD, u, v}); mt.pb(-1);
        prv[{u, v}] = sz(q)-1;
    }
    void remove(int u, int v){
        if(u > v) swap(u, v);
        q.pb({DEL, u, v});
        int pr = prv[{u, v}];
        mt[pr] = sz(q)-1; mt.pb(pr);
    }
    void query(){ q.pb({QUERY, -1, -1}); mt.pb(-1); } //
         Agregar parametros nodos de ser necesario
    void process(){ // responde todas las consultas en
         orden
        if(!sz(q)) return;
        for0(i,sz(q)) if(q[i].type == ADD && mt[i] < 0)
            mt[i] = sz(q);
        go(0, sz(q));
    }
    void go(int s, int e){
        if(s+1 == e){
            if(q[s].type == QUERY) cout<<uf.comps<<endl;
                // Cambiar la operacion de ser
                necesario
            return;
        }
        int m = (s+e)/2;
        uf.checkpoint();
        forlr(i,m,e-1) if(mt[i] >= 0 && mt[i] < s) uf.
            join(q[i].u, q[i].v);
        go(s, m);
        uf.rollback();

        uf.checkpoint();
        forlr(i,s,m-1) if(mt[i] >= e) uf.join(q[i].u, q[
            i].v);
        go(m, e);
        uf.rollback();
    }
};

void solver(){
    // Crear
    int n, q; cin>>n>>q;
    DynCon dc(n);
    for0(i,q){
        string op; cin>>op;
        if(op == "ADD"){ int u, v; cin>>u>>v; dc.add(u,
            v); } // Agregar arista
        else if(op == "DEL"){ int u, v; cin>>u>>v; dc.
            remove(u, v); } // Eliminar arista
        else dc.query(); // Consultar el numero de
            componentes
    }
    dc.process(); // Procesar
}
```

3.12 Kruskal

```
/*
IMPORTAR EL DSU.
- Halla el peso del MST, asumiendo que el grafo es
    conexo (sirve
  perfectamente para grafos con pesos negativos).
- Complejidad O(m * log(m)) por el ordenamiento de las
    aristas.
- Como saber si una arista esta en ningun, alguno o
    todos los MST:
  * Para ambos, la complejidad sigue siendo O(m * log(m)
      ).
  * Para decidir que aristas pueden hacer parte del MST:
       se procesan
    las aristas por grupos del mismo peso. Para cada
        grupo nuevo
    inicialmente se revisa cada arista, si los vertices
         que
    conecta ya hacen parte del mismo componente entonces
         esa arista
    no hace parte de ningun MST, en caso contrario si
        hace
    parte de al menos un posible MST.
    Despues de hacer esa comprobacion para todas las
        aristas del
    grupo, se agregan al DSU.
  * Para saber que aristas estan en todos los posibles
      MST: para
    cada grupo se constuye un grafo (que puede ser
        disconexo) solo
    con las aristas de este grupo que pueden ser parte
        de un MST y
    con los representantes de cada nodo (los nodos padre
        ). Se hallan
    los puentes de dicho grafo. Estos puentes son
        aristas que estan
    en todos los MST.
*/

struct edge {
    int u, v; ll w;
    bool operator<(const edge& that) const { // Ordenar
        las aristas por el menor peso
        return w < that.w; // Cambiar '<' por '>' para
            hallar el Maximal Spanning Tree
    }
};

ll kruskal(vector<edge>& edges, int n){
    ll total = 0;
    sort(all(edges));
    DSU uf(n+1);

    for0(i,sz(edges)){
        int u = edges[i].u, v = edges[i].v; ll w = edges
            [i].w;
        if(uf.join(u ,v)) total += w;
    }

    return total; // Retornar el peso total del MST
}

void solver(){
    int n, m; cin>>n>>m;
    vector<edge> edges;
    for0(i,m){
        int u, v; ll w; cin>>u>>v>>w;
        edge newEdge = {u, v, w};
        edges.pb(newEdge);
    }

    cout<<kruskal(edges, n)<<endl;
}
```

3.13 Prim

```
// Hallar el Minimum Spanning Tree de un grafo no
    dirigido y conexo
// Complejidad O(m * lg(n))

const int maxn = 1e5 + 1;
typedef pair<ll, int> pli;
vector<vector<pli>> graph(maxn);

void prim(int s, int n, vl& dis, vi& pa){
    vb inQueue(n+1, 1);
    dis[s] = 0;
    pqueue<pli, vector<pli>, greater<pli>> pq;
    pq.push({0, s});
    while(!pq.empty()){
        int u = pq.top().se;
        pq.pop();
        if(!inQueue[u]) continue;
        inQueue[u] = 0;
        for(pli &edge : graph[u]){
            int v = edge.se; ll w = edge.fi;
            if(inQueue[v] && dis[v] > w){
                dis[v] = w;
                pa[v] = u;
                pq.push({dis[v], v});
            }
        }
    }
}

void solver(){
    // Construir
    int n, m, source; cin>>n>>m>>source;
    for0(i,m){
        int u, v; ll w; cin>>u>>v>>w;
        graph[u].pb({w, v});
        graph[v].pb({w, u});
    }
    vl dis(n+1, INFL); // Distancia de cada nodo a su
        padre en el MST
    vi pa(n+1, -1); // Padre de cada nodo en el MST

    // Calcular
```

```
    prim(source, n, dis, pa);
    // Peso total del MST
    ll totalWeight = 0;
    for(int i = 1; i <= n; i++) totalWeight += dis[i];
    cout<<totalWeight<<endl;
}
```

3.14 Tarjan SCC

```
/*
- Tarjan's algorithm for finding strongly connected
    components.
- Complejidad: O(E + V).
- Los ids del SCC se guardan en 0-index.
- d[i] = Discovery time of node i. (Initialize to -1)
- low[i] = Lowest discovery time reachable from node
    i. (Doesn't need to be initialized)
- scc[i] = Strongly connected component of node i. (
    Doesn't
    need to be initialized) (0-index)
- s = Stack used by the algorithm (Initialize to an
    empty
    stack)
- stacked[i] = True if i was pushed into s. (Initialize
    to
    false)
- ticks = Clock used for discovery times (Initialize to
    0)
- current_scc = ID of the current_scc being discovered
    (Initialize to 0)

Propiedades
- Al comprimir cada SCC como un nodo individual, se
    obtiene un DAG.
- Si de un SCC A hay un camino a otro SCC B, no hay
    camino de B hacia A.
- Si se invierten todas las aristas, los SCC no cambian.
Aplicacion 2-SAT:
- Dada una expresion booleana en CNF (Conjuctive Normal
    Form), lo cual
  es una conjucion de multiples clausulas, donde cada
      clausula tiene
  exactamente 2 literales, ej: (a v b) ^ (c v d) ^ ... .
      Se debe hallar
  un valor para cada variable (True o False) tal que la
      formula sea True.
  Para resolverlo se crean 2 nodos para cada literal (a
      y ~a), y para
  cada clausula (a v b) se crean 2 aristas: '~a'->'b' y
      '~b'->'a'. Se
  corre Tarjan para hallar los SCC, si hay algun literal
      'a' tal que los
  nodos 'a' y '~a' hacen parte del mismo SCC, no hay
      solucion. De lo
  contrario si la hay y para hallar una asignacion
      valida se comprime el
  grafo original y se realiza un toposort, si en el
      orden del toposort el
  SCC de 'a' se encuentra antes que el SCC de '~a',
      entonces 'a' es False,
  de lo contrario es True.
*/

void tarjan(int u, vector<vi>& graph, vi&d, vi& low, vi&
     scc, vi& reps, vb& stacked, stack<int>& s, int&
    ticks, int& current_scc){
    d[u] = low[u] = ticks++;
    s.push(u);
    stacked[u] = 1;
    const vi &out = graph[u];
    for(int k=0, m=sz(out); k<m; ++k){
        const int &v = out[k];
        if(d[v] == -1){
            tarjan(v, graph, d, low, scc, reps, stacked,
                 s, ticks, current_scc);
            low[u] = min(low[u], low[v]);
        }
        else if(stacked[v]) low[u] = min(low[u], low[v])
            ;
    }
    if(d[u] == low[u]){
        int v;
        do{
            v = s.top();
            s.pop();
            stacked[v] = 0;
            scc[v] = current_scc;
        }while (u != v);
        current_scc++;
    }
}

void solver(){
    int n, m; cin>>n>>m;

    vector<vi> graph(n+1);
    vi d(n+1, -1), low(n+1), scc(n+1), reps(n+1, 0);
    vb stacked(n+1);
    stack<int> s;
    int ticks = 0, current_scc = 0;

    for1(i,m){
        int u, v;cin>>u>>v;
        graph[u].pb(v);
    }

    // Hallar los SCC
    for(int i = 1; i <= n; i++){ // 1-index, se puede
        cambiar a 0-index
        if(d[i] == -1)
            tarjan(i, graph, d, low, scc, reps, stacked,
                 s, ticks, current_scc);
    }

    cout<<current_scc<<endl; // Cantidad de SCC del
        grafo

    // Armar grafo comprimido (O(V + E))
    vector<vi> graph2(current_scc+1);
    vi inDegree(current_scc+1);
    for1(i,n){
        for(int v : graph[i]){
            if(scc[v] != scc[i]){ // Verificar que la
                arista vaya a otro SCC
                graph2[scc[i]+1].pb(scc[v]+1);
                inDegree[scc[v]+1]++;
            }
        }
    }
}
```

3.15 Toposort

```
// Complejidad: O(n + m)

void topoSort(int n, vi& orden, vi& inDegree, vector<vi
    >& graph){
    queue<int> q;
    for1(i,n){
        if(!inDegree[i]){ // Agregar si no tiene aristas
            de entrada
            q.push(i);
            orden.pb(i);
        }
    }

    while(!q.empty()){
        int u = q.front(); q.pop();
        for(int v : graph[u]){
            inDegree[v]--;
            if(!inDegree[v]){ // Agregar si ya se
                visitaron todos sus predecesores
                q.push(v);
                orden.pb(v);
            }
        }
    }
}

void solver(){
    // Construir
    int n, m; cin>>n>>m;
    vi inDegree(n+1);
    vector<vi> graph(n+1);
    for0(i,m){
        int u, v; cin>>u>>v;
        graph[u].pb(v);
        inDegree[v]++;
    }

    // Hallar
    vi ans;
    topoSort(n, ans, inDegree, graph);
    if(sz(ans) < n) cout<<"IMPOSSIBLE"<<endl; // Si ans
        no tiene todos los nodos, no es un DAG
    else{
        for0(i,sz(ans)){
            if(i) cout<<' ';
            cout<<ans[i];
        }
        cout<<endl;
    }
}
```

3.16 Bridges

```
/*
// O(n + m)
- Halla las aristas bridge en un grafo no dirigido
    conexo.
- Tener en cuenta que si el grafo no es conexo y se
    quieren hallar puentes
  en algun componente, tocaria lanzar un DFS para cada
      componente y en el
  for del final saltarse todos los vertices desde los
      cuales se lanzo algun
  DFS.
- No toma en cuenta aristas multiples.
- Para hallar los two edge connected components,
    simplemente
  eliminar los bridges del grafo y hallar los
      componentes con
  normalidad.
*/

const int maxn = 2e5 + 5;
vector<vi> graph(maxn);
vb vis(maxn);
vi dp(maxn), sons[maxn], father(maxn), dep(maxn);

void DFS(int u, int prev){
    vis[u] = 1;
    for(int v : graph[u]){
        if(v == prev) continue;
        if(vis[v]){ // Back-edge
            // Revisar que la arista vaya de abajo hacia
                arriba
            // para no considerarla 2 veces
            if(dep[v] > dep[u]) continue;
            dp[u]++;
            dp[v]--;
            continue;
```

```
        }
        sons[u].pb(v); // Span-edge
        father[v] = u;
        dep[v] = dep[u] + 1;
        DFS(v, u);
    }
}

int calc(int u){
    for(int v : sons[u]) dp[u] += calc(v);
    return dp[u];
}

void solver(){
    // Construir
    int n, m; cin>>n>>m;
    for0(i,m){
        int u, v; cin>>u>>v;
        graph[u].pb(v); // Las aristas TIENEN que estar
            en ambas listas de adyacencia
        graph[v].pb(u);
    }

    // Da igual cual sea la raiz asi que se usa root = 1
    DFS(1 /*root*/, -1); // Crear DFS tree
    calc(1 /*root*/); // Calcular el dp

    forlr(i,2,n){
        // Si dp[i] == 0, la arista que conecta i y su
            padre son un puente
        if(!dp[i]) cout<<"Bridge: "<<father[i]<<" - "<<i
            <<endl;
    }
}
```

3.17 Articulation points

```
/* NO TESTED
- Halla los puntos de articulacion de un grafo no
  dirigido.
- Los puntos de articulacion son aquellos nodos que si
  se eliminaran del grafo haria que este se divida en
  2 o mas componentes.
- Tener en cuenta que si solo hay un nodo o dos, estos
  no se marcan como puntos de articulacion (ya que al
  eliminarlo el grafo tendria '0' componentes o
  seguiria siendo de uno solo).
- Si hubieran varios componentes y se quisieran hallar
  los puntos de articulacion de cada componente, se
  recorren todos los nodos y se tira un DFS desde los
  que no han sido visitados. Se cumple la regla de que
  si un componente solo tiene uno o dos nodos estos no
  se cuentan como puntos de articulacion.
- Complejidad: O(n+m).
- Ejemplo 1-index pero sirve para 0-index.
*/

const int maxn = 2e5 + 5;
vector<vi> graph(maxn);
vb vis(maxn), isAP(maxn);
vi low(maxn), dep(maxn);

void DFS(int u, int prev){
    vis[u] = 1;
    low[u] = dep[u];
    int children = 0;
    for(int v : graph[u]){
        if(v == prev) continue;
        if(vis[v]){ // Back-edge
            low[u] = min(low[u], dep[v]);
            continue;
        }
        dep[v] = dep[u] + 1;
        DFS(v, u);
        low[u] = min(low[u], low[v]);
        children++;
        if(prev != -1 && low[v] >= dep[u]) isAP[u] = 1;
    }
    if(prev == -1 && children > 1) isAP[u] = 1; // Raiz
}

void solver(){
    int n, m; cin>>n>>m;
    for0(i,m){
        int u, v; cin>>u>>v;
        graph[u].pb(v);
        graph[v].pb(u);
    }
    DFS(1, -1);
    for1(i,n) if(isAP[i]) cout<<"Articulation point: "<<
        i<<endl;
}
```

3.18 TSP

```
/*
- Complejidad O(2^n * n^2).
- Se asume que se da la entrada en 1-index y se pasa a
    0-index.
- Halla el minimo peso para recorrer todos los vertices
    del grafo
  iniciando en el vertice 'start' (teniendo en cuenta
      que para moverse
  entre 2 vertices solo se pueden usar aristas entre
      ellos).

- Otras aplicaciones:
- Para hallar el camino hamiltoniano de menos costo (
    empezando en
  cualquier vertice) se inicializa el DP con dp[i][1<<i]
      = 0, para
  'i' entre 0 y n-1.
- Para pedir que termine en algun vertice en especifico
    'u' se hace que
  la unica mascara valida que contenga 'u' sea lim-1 (
      con 'lim' = 1<<n),
  y que solo sea valida cuando el vertice final sea 'u',
      es decir,
  dp[u][lim-1].
- Para hallar el ciclo hamiltoniano mas corto, podemos
    elegir el vertice
  0 como inicial, marcar dp[0][1] = 0 e inicializar los
      demas estados en
  INFL, despues corremos el algoritmo con normalidad y
      finalmente para
  cada 1 <= i < n calculamos dp[i][lim-1] + dis(i, 0) (
      tener en cuenta
  que si el grafo es dirigido, el peso de ir desde 'i'
      hacia 0 se guardo
  al reves) y elegimos el menor.
*/

typedef pair<ll, int> pli;
const int maxn = 20;
vector<vector<pli>> graph(maxn);

void solver(){
    int n, m, start; cin>>n>>m>>start; start--;
    for0(i,m){ // Si el grafo es dirigido, guardar solo
        la arista inversa
        int u, v, w; cin>>u>>v>>w; u--; v--; // Pasar a
            0-index
        graph[u].pb({w, v});
        graph[v].pb({w, u});
    }

    // Inicializar el dp
    int lim = 1 << n;
    vector<vl> dp(n, vl(lim, INFL));
    dp[start][1 << start] = 0; // Poner distancia 0 en
        el camino que solo tiene 'start'

    for1(mask,lim-1){
        if(!(mask & (1 << start))) continue; // Si en
            ese camino no se ha visitado 'start'
        for0(u,n){ // Camino finalizando en u
            if(!(mask & (1 << u))) continue;

            for(pli e : graph[u]){
                int v = e.se; ll w = e.fi; // Yendo de '
                    v' hacia 'u'
                if(!(mask & (1 << v))) continue;
                dp[u][mask] = min(dp[u][mask], w + dp[v
                    ][mask ^ (1 << u)]);
            }
        }
    }

    ll ans = INFL;
    for0(i,n) ans = min(ans, dp[i][lim - 1]); // Compara
        el terminar en cada posible nodo
    cout<<(ans == INFL ? -1 : ans)<<endl; // Imprime -1
        si no existe el recorrido
}
```

3.19 Counting cycles

```
/*
- Complejidad O(2^n * n^2).
- Cuenta la cantidad de ciclos simples (donde el unico
  vertice que se repite es aquel en el que inicia y
      termina
  el ciclo) de un grafo no dirigido.
- Puede ser necesario agregar modularidad.
- dp[i][mask] cuenta la cantidad de "simple paths" (que
    no
  repiten vertices) que empiezan por el primer bit
  encendido de 'mask' y terminan en 'i'.
Variantes:
- Para ciclos hamiltonianos revisar el "Counting_paths".
- Si el grafo es dirigido, se arma el grafo normal e
    inverso.
  En el ciclo para caminos de al menos 2 vertices se usa
      el
  inverso. En el ciclo del final se usa el grafo normal,
      se
  debe iniciar desde mask=3 y contar los caminos con 2 o
      mas
  vertices. No aplica la correccion de aristas multiples
      con
  gauss que se menciona abajo. Se elimina la division
      entre 2
  del final.
- Si pueden haber aristas multiples y no se deben tener
    en
  cuenta, guardar con set.
- Si se debe tener en cuenta aristas multiples, sumar a
    la
  respuesta 2*gauss(frecuencia de cada arista - 1) (
      antes de
  la division del final), para contar los ciclos de
      tamano 2
  (que van de 'u' hacia 'v' en una arista y de 'v' hacia
      'u'
  en otra).
- No cuenta loops. Si hay que contarlos toca sumarlos
    aparte.
*/

const int maxn = 20;
vector<vi> graph(maxn);

void solver(){
    int n, m; cin>>n>>m;
    for0(i,m){
```

```
        int u, v; cin>>u>>v; u--; v--; // Se cambia a 0-index
        graph[u].pb(v);
        graph[v].pb(u);
    }

    int lim = (1 << n);
    vector<vl> dp(n, vl(lim));

    // Cada camino de un solo vertice se puede hacer de una forma
    for0(i,n) dp[i][1 << i] = 1;

    // Cuenta los caminos de al menos 2 vertices
    forlr(mask,3,lim-1){
        for0(i,n){
            // Revisa que 'i' haga parte del camino pero no sea el nodo inicial
            if(!(mask & (1 << i)) || !(mask % (1 << i))) continue;

            // Para cada arista j->i, sumar los que tienen la misma mask, pero sin 'i', y que terminan en 'j'
            for(int j : graph[i]){
                if(!(mask & (1 << j))) continue;
                dp[i][mask] += dp[j][mask ^ (1 << i)];
            }
        }
    }

    // Se suman los caminos de tamano >= 3 cuyo primer vertice es 'fst' y ultimo 'i' por la cantidad
    // de aristas de 'i' hacia 'fst'
    ll ans = 0;
    forlr(mask,7,lim-1){
        int cnt =__builtin_popcount(mask), fst = __builtin_ctz(mask);
        if(cnt == 3) continue;
        for0(i,n){
            if(fst == i) continue;

            // Si desde i hacia fst hay aristas, entonces se forman ciclos
            ll mul = 0; // Cantidad de aristas de i hacia fst
            for(int v : graph[i]) if(v == fst) mul++;
            ans += mul * dp[i][mask];
        }
    }
    ans /= 2;
    cout<<ans<<endl; // Se divide por 2 ya que los ciclos se contaron en ambas direcciones
}
```

3.20 Counting paths

```
const ll mod = 1e9 + 7;
const int maxn = 20;
ll dp[maxn][1 << maxn];
vector<vi> graph(maxn);

/*
- Complejidad: O(2^n * n^2).
- Cuenta la cantidad de caminos hamiltonianos que contiene un grafo
  empezando en el vertice 's', pasando por todos los vertices
  exactamente una vez y terminando en el vertice 't'.
- dp[i][mask] cuenta la cantidad de caminos que inician en 's',
  pasan por los vertices de mask y terminan en 'i'.
- Para contar la cantidad de caminos hamiltonianos que contiene un
  grafo iniciando y terminando en cualquier vertice, se inicializa
  dp[i][1 << i] en 1, para i desde '0' hasta 'n-1' y finalmente en la
  respuesta se suma dp[i][2^n - 1] para i desde '0' hasta 'n-1'. Si el
  grafo es no dirigido se debe dividir la respuesta entre 2 (division
  modular).
  Si n=1, solo imprimir '1'.
- Para contar la cantidad de caminos simples que contiene un grafo
  (que pasa por al menos 2 vertices exactamente una vez) se hace el
  mismo dp que en el caso anterior pero en la respuesta se suma
  cada estado cuya mascara contenga 2 o mas vertices. Si el grafo es
  no dirigido se divide la respuesta entre 2 (con division modular) ya
  que de esa forma se esta contando 2 veces cada camino (en ambas
  direcciones).
- Para contar la cantidad de ciclos hamiltonianos que tiene un grafo
  se toma el '0' como nodo inicial, y se cuentan todos los caminos
  hamiltonianos que empiezan en este. Finalmente, para cada nodo
  1 <= i < n, si hay una arista que conecte i->0, se suma dp[i][2^n - 1]
  a la respuesta. Si el grafo es no dirigido se debe dividir la
  respuesta entre 2 (con division modular).
  (El caso con n <= 2 es mejor tratarlo aparte).
*/

void solver(){
    int n, m, s, t; cin>>n>>m>>s>>t;
    s--; t--; // Pasar de 1-index a 0-index
    for(int i = 0; i < m; i++){
        int u, v; cin>>u>>v; u--; v--; // Pasar de 1-index a 0-index
        graph[u].pb(v);
        graph[v].pb(u); // Comentar si es dirigido
    }

    int lim = 1 << n;
    dp[s][1 << s] = 1;
    for1(mask,lim-1){
        if(!(mask & (1 << s))) continue; // Comentar si no hay fuente especifica
        if(mask & (1 << t) && mask < lim - 1) continue; // Comentar si no hay destino especifico

        for0(i,n){
            if(!(mask & (1 << i))) continue;
            for(int v : graph[i]){
                if(mask & (1 << v)) continue;
                dp[v][mask | (1 << v)] += dp[i][mask];
                dp[v][mask | (1 << v)] %= mod;
            }
        }
    }

    cout<<dp[t][lim - 1]<<endl;
}
```

3.21 Eulerian path

```
/*
- Implementacion del algoritmo de Hierholzer para hallar un
  camino (y posiblemente circuito) euleriano.
- Esta implementacion es obligatoriamente 1-index.
- Si no hay aristas se imprime 1 (podria ser cualquier nodo)
- Complejidad: O(V + E).
- Un camino euleriano pasa exactamente una vez por cada arista del grafo.
- Un circuito euleriano es un camino euleriano que inicia y
  termina en el mismo nodo.

Un camino euleriano existe si todas las aristas pertenecen
al mismo componente y:
* Para grafos no dirigidos:
    - El grado de todos los nodos es par (en este caso tambien
      es un circuito).
    - El grado de exactamente 2 nodos es impar y el del resto
      es par (en este caso el camino inicia en uno de estos y
      termina en el otro).
* Para grafos dirigidos:
    - El inDegree y outDegree de todos los nodos es igual
      (tambien es circuito).
    - Exactamente un nodo tiene inDegree + 1 == outDegree
      (nodo inicial) y exactamente un nodo tiene
      inDegree == outDegree + 1 (nodo final).


APLICACION: DE BRUJIN SEQUENCES:
- Es una secuencia en la cual de un conjunto de k caracteres
  aparece cada cadena posible de longitud n exactamente una
  vez como subcadena.
- Si se puede conectar los ultimos caracteres de la cadena
  con los primeros (como si fuera ciclicla), entonces la
  secuencia tiene tamano k^n.
  Ej: K={0,1}, n=2 -> 0011. (00, 01, 11, 10), donde la subcadena
  '10' se da conectando el ultimo caracter con el primero.
- Si no es ciclica, entonces tiene tamano k^n + n - 1. Con el
  mismo ejemplo la secuencia puede ser: 00110.
  La version ciclica es simplemente los primeros k^n caracteres
  de esta.
- COMO HALLARLA:
  * Se construye un grafo dirigido donde los nodos son todas las
    cadenas de longitud n-1 (k^(n-1) nodos) y donde cada cadena
    de longitud n es una arista que conecta el nodo que
    corresponde a su primeros n-1 caracteres con el que
    corresponde a sus ultimos n-1 caracteres.
    De esta forma se obtiene un grafo conexo donde cada nodo va
    a tener k aristas de salida y k aristas de entrada.
  * Se halla el circuito euleriano desde cualquier nodo inicial
    y para la secuencia se toma el ultimo caracter de cada arista
    que se visita (que es el mismo ultimo caracter del nodo al
    que lleva dicha arista) (TENER EN CUENTA que se debe modificar
    el vector ans para que guarde este caracter, ya que actualmente
    lo que guarda son los nodos). Debido a como funciona el codigo,
    al vector ans que se recibe al final toca hacerle
```

```
        reverse y
    ya se obtiene la respuesta en orden.
    Para la version no ciclica, antes de hacer el
        reverse se
    agregan al final los caracteres del nodo inicial en
        orden
    inverso y entonces ya se hace el reverse.
*/

void DFS(int u, vector<vii>& graph, vb& used, vi& ans){
    while(!graph[u].empty()){
        pii edj = graph[u].back();
        int v = edj.fi, idx = edj.se;
        graph[u].pop_back(); // Eliminar arista ya
            visitada
        if(used[idx]) continue; // En caso de ya haber
            visitado esta arista partiendo desde v
        used[idx] = 1;
        DFS(v, graph, used, ans);
    }
    ans.pb(u);
}

// Para grafos no dirigidos
void solverNoDirected(){
    int n, m; cin>>n>>m;
    if(!m){cout<<1<<endl;return;} // SI no hay aristas
        imprime cualquier nodo
    vector<vii> graph(n+1);
    vi degree(n+1);
    for0(i,m){
        int u, v; cin>>u>>v;
        graph[u].pb({v, i});
        graph[v].pb({u, i});
        degree[u]++;
        degree[v]++;
    }

    int a = 0, b = 0; // Nodos desbalanceados
    for1(i,n){
        if(degree[i] & 1){ // Si se quiere un ciclo,
            simplemente retornar que no se puede
            if(!a) a = i;
            else if(!b) b = i;
            else{
                cout<<"IMPOSSIBLE"<<endl;
                return;
            }
        }
    }

    vb used(m+1);
    vi ans;
    if(a) DFS(a, graph, used, ans); // Llamar a partir
        del nodo inicial (si lo hay)
    else{
        for1(i,n){
            if(degree[i]){ // Llamar desde un nodo que
                tenga aristas de salida
                DFS(i, graph, used, ans);
                break;
            }
        }
    }

    if(sz(ans) != m+1){ // Si no todas las aristas estan
        en el mismo componente
        cout<<"IMPOSSIBLE"<<endl;
        return;
    }

    for(int x : ans) cout<<x<<' '; // Camino euleriano
        por orden de visita de los nodos
    cout<<endl;
}
```

```
// Para grafos dirigidos
void solverDirected(){
    int n, m; cin>>n>>m;
    if(!m){cout<<1<<endl;return;} // SI no hay aristas
        imprime cualquier nodo
    vector<vii> graph(n+1);
    vi inDegree(n+1), outDegree(n+1);
    for0(i,m){
        int u, v; cin>>u>>v;
        graph[u].pb({v, i});
        outDegree[u]++;
        inDegree[v]++;
    }

    int a = 0, b = 0; // a seria el nodo inicial y b el
        nodo final
    for1(i,n){
        if(inDegree[i] > outDegree[i]){ // Si se quiere
            un ciclo, simplemente retornar que no se
            puede
            if(inDegree[i] == outDegree[i] + 1 && !b) b
                = i; // Este tendria que ser el nodo
                final
            else{
                cout<<"IMPOSSIBLE"<<endl;
                return;
            }
        }
        else if(inDegree[i] < outDegree[i]){ // Si se
            quiere un ciclo, simplemente retornar que
            no se puede
            if(inDegree[i] + 1 == outDegree[i] && !a) a
                = i; // Este tendria que ser el nodo
                inicial
            else{
                cout<<"IMPOSSIBLE"<<endl;
                return;
            }
        }
    }

    if((a && !b) || (!a && b)){ // Si solo hay nodo
        inicial o solo hay nodo final
        cout<<"IMPOSSIBLE"<<endl;
        return;
    }

    vb used(m+1);
    vi ans;
    if(a) DFS(a, graph, used, ans); // Llamar a partir
        del nodo inicial (si lo hay)
    else{
        for1(i,n){
            if(outDegree[i]){ // Llamar desde un nodo
                que tenga aristas de salida
                DFS(i, graph, used, ans);
                break;
            }
        }
    }

    if(sz(ans) != m+1){ // Si no todas las aristas estan
        en el mismo componente
        cout<<"IMPOSSIBLE"<<endl;
        return;
    }

    reverse(all(ans));
    for(int x : ans) cout<<x<<' '; // Camino euleriano
        por orden de visita de los nodos
    cout<<endl;
}
```

3.22 Grafo funcional Floyd

```
/*
- Un grafo funcional es aquel en el que cada nodo tiene
    exactamente
  una arista de salida. Cada componente tiene
      exactamente un ciclo.
- Dado un nodo 'x', halla el tamano del ciclo que hay en
      el
  componente al que peretenece 'x'.
- func[u] dice hacia donde va la arista que sale de 'u'.
- Complejidad: O(n).
*/

// x es el nodo inicial desde el cual se va a buscar un
    ciclo
int floyd(int x, vi& func){
    int a = func[x];
    int b = func[func[x]];

    // En el momento en que 'a' y 'b' se encuentran, 'a'
        ha caminado 'k' pasos y 'b' '2k' pasos,
    // entonces (k % lenCycle = 0)
    while(a != b){
        a = func[a];
        b = func[func[b]];
    }

    // El primer nodo que pertenece al ciclo (partiendo
        desde 'x') se encuentra moviendo
    // el puntero 'a' al nodo 'x' y avanazando 'a' y 'b'
        paso por paso hasta que se encuentran
    // de nuevo
    a = x;
    while(a != b){
        a = func[a];
        b = func[b];
    }
    int firstCycleNode = a;

    // En este punto tanto 'a' como 'b' estan parados en
        el nodo inicial del ciclo, entonces
    // para hallar la longitud del ciclo simplemente se
        puede avanzar 'b' paso por paso
    // hasta que se vuelva a encontrar con 'a', la
        cantidad de pasos dados es la longitud
    // del ciclo
    b = func[b];
    int len = 1;
    while(a != b){
        b = func[b];
        len++;
    }

    return len;
}
```

3.23 Grafo funcional count steps

```
/*
- Sirve para contestar queries sobre la cantidad de
    pasos que hay
  que dar desde un nodo 'a' para llegar a un nodo 'b'.
- Complejidad: O(n * log(n)) del precalculo, O(log(n))
    por query.
- isCicle marca si un nodo hace parte de un ciclo o no.
- liderCicle guarda el lider de cada ciclo.
- stCicle guarda la distancia desde un nodo de un ciclo
    hasta su
  lider.
- lenCicle[i] si el nodo 'i' es lider de un ciclo dice
    la longitud
  de este, si no dice 0.
- distCicle guarda la distancia de un nodo cualquiera a
```

```
    un ciclo.
- whereCicle guarda para cada nodo fuera de un ciclo cual es el
  nodo mas cercano que esta en un ciclo.
*/

int LOG_N = 18; // techo del log2 del maximo valor de k que me puedan dar en las queries
const int maxn = 2e5 + 1;
int n;
vi func(maxn), liderCicle(maxn), stCicle(maxn), lenCicle(maxn), distCicle(maxn), whereCicle(maxn);
vb vis(maxn), isCicle(maxn);
vector<vi> succ(LOG_N, vi(maxn)); // TAMBIEN COPIAR "GRAFO FUNCIONAL KTH STEP"

void calcSucc(){
    for1(i,n) succ[0][i] = func[i];
    for1(bit,LOG_N-1) for1(i,n) succ[bit][i] = succ[bit-1][succ[bit-1][i]];
}

int kSucc(int s, int k){
    for0(bit,LOG_N) if(k & (1 << bit)) s = succ[bit][s];
    return s;
}

// Precalcula isCicle, liderCicle, stCicle y lenCicle
void floyd(int x){
    vis[x] = 1;
    int a = func[x];
    int b = func[func[x]];

    while(a != b){
        if(vis[a]) return; // Saltar en caso de que ya haya calculado esta rama desde un nodo inferior
        vis[a] = 1; // Marcar los nodos externos a un ciclo que ya han sido visitados
        a = func[a];
        b = func[func[b]];
    }
    a = x;
    while(a != b){
        a = func[a];
        b = func[b];
    }
    if(isCicle[b]) return; // Ya fue procesado este ciclo

    int lider = b;
    liderCicle[lider] = b; // Se guarda el lider
    isCicle[b] = 1;
    lenCicle[b] = 1;
    int last = 0;
    vis[lider] = 1;
    b = func[b];
    while(a != b){
        liderCicle[b] = lider;
        lenCicle[lider]++; // Al llegar a otro nodo se aumenta la longitud del ciclo
        last++;
        stCicle[b] = last;
        isCicle[b] = 1;
        vis[b] = 1; // Se marcan como visitados todos los nodos del ciclo
        b = func[b];
    }
}

// Precalcula whereCicle y distCicle para los nodos fuera de un ciclo
void DFS(int u){
    int v = func[u];
    if(isCicle[v]){
        whereCicle[u] = v;
        distCicle[u] = 1;
    }
    else if(distCicle[v]){
        whereCicle[u] = whereCicle[v];
        distCicle[u] = distCicle[v] + 1;
    }
    else{
        DFS(v);
        whereCicle[u] = whereCicle[v];
        distCicle[u] = distCicle[v] + 1;
    }
}

void solver(){
    int q; cin>>n>>q; // n se declaro global
    vi inDegree(n+1);
    for1(i,n){
        cin>>func[i];
        inDegree[func[i]]++;
    }
    calcSucc();

    for1(i,n) if(!vis[i]) floyd(i);

    for1(i,n){
        if(inDegree[i]) continue;
        DFS(i);
    }

    while(q--){
        int a, b; cin>>a>>b;
        if(a == b){
            cout<<0<<endl;
            continue;
        }

        // Si ambos son parte de un ciclo primero se verifica si es el mismo y luego se calcula la distancia usando stCicle
        if(isCicle[a] && isCicle[b]){
            if(liderCicle[a] != liderCicle[b]) cout<<-1<<endl;
            else cout<<(stCicle[b] - stCicle[a] + lenCicle[liderCicle[a]]) % lenCicle[liderCicle[a]]<<endl;
        }

        // Si ninguno hace parte de ciclo se verifica si hacen parte de la misma rama (usando whereCicle), despues se usa
        // distCicle para calcular dif (a que distancia deberia estar 'a' de 'b') y si es positiva se usa kSucc para verificar que
        // al dar dif pasos desde 'a' se llega hasta 'b' (ya que estas ramas son arboles y se requiere que 'b' sea ancestro de 'a').
        else if(!isCicle[a] && !isCicle[b]){
            if(whereCicle[a] != whereCicle[b]) cout<<-1<<endl;
            else{
                int dif = distCicle[a] - distCicle[b];
                if(dif <= 0) cout<<-1<<endl;
                else{
                    if(kSucc(a, dif) == b) cout<<dif<<endl;
                    else cout<<-1<<endl;
                }
            }
        }

        // Si 'a' no esta en ciclo pero 'b' si, se revisa si 'a' llega al ciclo de 'b' (usando whereCicle) y en caso de que si se
        // suma la distancia de a hasta un ciclo y se usa whereCicle[a] y stCicle para calcular cuanto se tarda desde whereCicle[a] hasta b
        else if(!isCicle[a] && isCicle[b]){
            if(liderCicle[whereCicle[a]] != liderCicle[b]) cout<<-1<<endl;
            else cout<<distCicle[a] + ((stCicle[b] - stCicle[whereCicle[a]] + lenCicle[liderCicle[b]]) % lenCicle[liderCicle[b]])<<endl;
        }

        // Si a esta en un ciclo y b no, no se puede
        else cout<<-1<<endl;
    }
}
```

3.24 Grafo funcional kth step

```
/*
- Se usa en grafos funcionales para contestar a que nodo se llega
  tras realizar k pasos iniciando en el nodo s.
- Complejidad: Precalculo: O(n * log(k)), queries en O(log(k)).
*/

const int maxn = 1e5 + 1;
const int LOG_N = 30; // techo del log2 del maximo valor de k que me puedan dar en las queries
vector<vi> succ(LOG_N, vi(maxn));

void calcSucc(int n, vi& func){
    for1(i,n) succ[0][i] = func[i];
    for1(bit,LOG_N-1) for1(i,n) succ[bit][i] = succ[bit-1][succ[bit-1][i]];
}

int kSucc(int s, int k){
    for0(bit,LOG_N) if(k & (1 << bit)) s = succ[bit][s];
    return s;
}

void solver(){
    int n, q; cin>>n>>q;
    vi func(n+1);
    for1(i,n) cin>>func[i];
    calcSucc(n, func); // Precalcular primero
    while(q--){
        int s, k; cin>>s>>k;
        cout<<kSucc(s, k)<<endl;
    }
}
```

3.25 Find faces grafo plano

```
/*
- Complejidad: O(E * log(E))
- Encuentra las caras de un grafo plano.
- Un grafo plano es uno en el cual cada vertice tiene unas
  coordenadas 'x', 'y' y las aristas que hay entre estos vertices
  jamas se intersectan (excepto en sus extremos).
- Debe ser un grafo conexo.
- Las caras de un grafo plano son las regiones en las que sus
  aristas dividen el plano. Cada region es minima, es decir,
  ninguna arista la atraviesa. La region exterior a todas las
  aristas tambien es una cara.
- faces[0] es la cara exterior.
- En cada cara los vertices estan odenados en sentido antihorario,
```

```
  excepto la exterior que se guarda en sentido horario.
- La cantidad de caras es: F = E - V + 2.
- Un grafo conexo sin ciclos (un arbol) tiene una sola
    cara, la
  exterior.

Ejemplo:
A=(0,0)  B=(1,0)  C=(1,1)  D=(0,1)
D ------- C
|       / |
|     /   |
|   /     |
| /       |
A ------- B
Tiene 3 caras:
    1. Triangulo A-B-C
    2. Triangulo A-C-D
    3. Cara exterior (region infinita), borde A-B-C-D
*/

struct Point {
    ll x, y;

    Point(ll x_, ll y_): x(x_), y(y_) {}

    bool operator < (const Point& p) const {
        return tie(x, y) < tie(p.x, p.y);
    }

    Point operator - (const Point& p) const {
        return Point(x - p.x, y - p.y);
    }

    ll cross (const Point & p) const { // Ciudado con el
         overflow
        return x * p.y - y * p.x;
    }

    ll cross (const Point & p, const Point & q) const {
        return (p - *this).cross(q - *this);
    }

    ll half () const {
        return ll(y < 0 || (y == 0 && x < 0));
    }
};

// Hallar las caras de un grafo plano
vector<vl> find_faces(vector<Point> vertices, vector<vl>
     adj){
    ll n = vertices.size();
    vector<vector<char>> used(n);
    for0(i,n){
        used[i].assign(sz(adj[i]), 0);
        auto compare = [&](ll l, ll r){
            Point pl = vertices[l] - vertices[i];
            Point pr = vertices[r] - vertices[i];
            if(pl.half() != pr.half())
                return pl.half() < pr.half();
            return pl.cross(pr) > 0;
        };
        sort(all(adj[i]), compare);
    }
    vector<vl> faces;
    for0(i,n){
        for(ll edge_id = 0; edge_id < sz(adj[i]);
            edge_id++){
            if(used[i][edge_id]) continue;
            vl face;
            ll v = i, e = edge_id;
            while(!used[v][e]){
                used[v][e] = 1;
                face.pb(v);
                ll u = adj[v][e];
                ll e1 = lower_bound(all(adj[u]), v, [&](
                    ll l, ll r) {
                    Point pl = vertices[l] - vertices[u
                        ];
                    Point pr = vertices[r] - vertices[u
                        ];
                    if(pl.half() != pr.half())
                        return pl.half() < pr.half();
                    return pl.cross(pr) > 0;
                }) - adj[u].begin() + 1;
                if(e1 == sz(adj[u])){
                    e1 = 0;
                }
                v = u;
                e = e1;
            }
            reverse(all(face));
            ll sign = 0;
            for(ll j = 0; j < sz(face); j++){
                ll j1 = (j + 1) % sz(face);
                ll j2 = (j + 2) % sz(face);
                ll val = vertices[face[j]].cross(
                    vertices[face[j1]], vertices[face[
                    j2]]);
                if(val > 0){
                    sign = 1;
                    break;
                }else if(val < 0){
                    sign = -1;
                    break;
                }
            }
            if(sign <= 0){
                faces.insert(faces.begin(), face);
            } else{
                faces.emplace_back(face);
            }
        }
    }
    return faces;
}

void solver(){
  int n; cin>>n; // Nos van a dar n aristas

    // Almacenamiento de vertices y aristas para llamar
        el algoritmo de findFaces
    vector<Point> vertices;
    vector<vector<ll>> adj;
    map<Point, ll> appear;
    // Si pueden haber aristas duplicadas o loops
        tocaria eliminarlas
  for(ll i = 0; i < n; i++){
        ll x1, y1, x2, y2;
        cin>>x1>>y1>>x2>>y2;
        Point save1(x1, y1), save2(x2, y2);
        ll index1, index2;
        map<Point, ll>::iterator it1, it2;
        it1 = appear.find(save1);
        it2 = appear.find(save2);
        if(it1 == appear.end()) {
            index1 = appear[save1] = sz(vertices);
            vertices.pb(save1);
            adj.emplace_back();
        }
        else
            index1 = it1->second;

        if(it2 == appear.end()) {
            index2 = appear[save2] = sz(vertices);
            vertices.pb(save2);
            adj.emplace_back();
        }
        else
            index2 = it2->second;
        adj[index1].pb(index2);
        adj[index2].pb(index1);
    }

    vector<vl> faces = find_faces(vertices, adj);
}
```

4 Trees

4.1 Diameter

```
/*
- El diametro de un arbol es la mayor distancia entre
    cualquier
  par de nodos en el arbol.
- Tambien funciona para arboles con aristas con pesos
    positivos.
- Si se tiene que el diametro de un arbol es la
    distancia entre dos
  vertices u, v, entonces podemos asumir que la mayor
      distancia para
  un nodo x hacia cualquier nodo del arbol es  d(x, u) o
       d(x, v).
- Radio:
  Si el diametro d es par, el nodo que se encuentra en
      mitad del
  camino es el nodo central de cada posible diametro.
  Si el diametro es impar, la arista que se encuentra en
       mitad del
  camino es la arista central de cada posible diametro.
- Aristas en al menos un camino que forma un diametro:
  Para una arista u, v (con depth[u] < depth[v]), si la
      suma entre
  la mayor distancia de v a cualquier nodo en su
      subarbol, la mayor
  distancia de u a cualquier nodo que no este en el
      subarbol de v
  es igual a diametro - 1.
- Aristas en todos los caminos que forman un diametro (
    no tested):
  * Se halla el diametro d y sus extremos u, v.
  * Se halla el conjunto Eu con los nodos que estan a
      distancia d de v.
    Y el conjunto Ev con los que estan a distancia d de
        u.
  * Para cada arista x, y (con depth[x] < depth[y]) si
      todos los nodos
    de Eu se encuentran en el subarbol de y, y todos los
         nodos de Ev
    se encuentran fuera del subarbol de x o en x (o
        viceversa los
    conjuntos) entonces dicha arista esta en todos los
        caminos que
    forman un diametro.
  Todas estas aristas forman un camino.
*/
```

4.2 Euler Tour

```
/*
- Compeljidad: O(n+m)
- Implementacion 1-index
- La posicion de cada nodo i en el arbol aplanado es ran
    [i].fi
- Llamar con DFS(ran, root, -1, cnt(inicializado en 1))
- Los rangos se guardaran ran[u] = [l, r]

- Si nos dan un arbol rooteado y tenemos que hacer
    queries de algun tipo sobre
  algunos subarboles (como sumar los valores de cada
      elemento de algun subarbol)
  y tenemos que hacer updates sobre los valores de los
      nodos, se puede construir
```

```
  un array de modo que todos los elementos de cualquier
       subarbol seran continuos
  y se puede guardar cual es el rango de cada subarbol
       mediante un DFS. Una vez
  se ha procesado todo esto y generado el nuevo array,
       se pueden realizar queries
  y updates de cualquier tipo usando un segment tree.
- Si no necesitamos hallar valores entre el subarbol de
    algun nodo sino en su
  camino hacia la raiz entonces hacemos updates en rango
       y queries individuales,
  es decir, el valor de un nodo se le actualiza a todo
       su correspondiente subarbol,
  y para hacer una query entre el camino de un nodo
       hacia la raiz, se hace la query
  especificamente sobre el nodo que se pregunta (query
       individual).
- Si se pide la suma de valores en un camino se puede
    usar la misma tecnica de
  arriba asi: calc(u) + calc(v) - 2*calc(LCA(u, v)) +
       val[LCA(u, v)].
*/

const int maxn = 2e5 + 1;
vector<vi> tree(maxn);

void DFS(vii& ran, int u, int pa, int &cnt){
    // El rango de u inicia en cnt. Ademas ese sera su
         indice en el array que se construira.
    ran[u].fi = cnt;
    cnt++; // Se suma 1 para guardar que se ha visitado
         un nuevo nodo
    for(int v : tree[u]){
        if(v == pa) continue;
        DFS(ran, v, u, cnt);
    }

    // Despues de visitar los nodos del subarbol de u,
         decimos en que indice termina el rango de u
    ran[u].se = cnt - 1;
}

void solver(){
    // Llamar el DFS para hallar los rangos de cada
         subarbol
    vii ran(n+1);
    int cnt = 1;
    DFS(ran, root, -1, cnt);

    // Arbol aplanado
    vi orden(n+1);
    for1(i,n) orden[ran[i].fi] = i; // El vertice i se
         encuentra en la posicion ran[i].fi

    // Rango del subarbol de cada nodo [l, r]
    for1(i,n) cout<<"i -> "<<i<<" l -> "<<ran[i].fi<<" r
         -> "<<ran[i].se<<endl;
}
```

4.3 LCA Euler Tour

```
/*
LCA usando Euler Tour Technique.
- La sparse table se rellenara con los pares
  {depth[id[i]], id[i]}, para 0 <= i < sz(id).
- Para hacer las queries de LCA(u, v) se necesita hallar
     el nodo
  con menor profundidad en el intervalo [start[u], start
       [v]],
  con start[u] <= start[v], lo cual se hara con una
       sparse table
  de minimos. Cada query halla el nodo con menor
       profundidad en
  ese intervalo en O(1).
- Para hallar la distancia entre 2 nodos u,v del arbol:
    distancia(u, v) = depth[u] + depth[v] - 2*depth[LCA(u,
         v)]
*/

// AGREGAR SPARSE TABLE DE PARES Y MINIMOS

const int maxn = 2e5 + 1;
vector<vi> tree(maxn);
vi id, depth(maxn), start(maxn);

// Llenar los arrays depth, id y start
void DFS(int u, int pa){
    start[u] = sz(id); // Marcar en que momento se
         visito por primera vez
    id.pb(u); // Insertar cuando se visita el nodo por
         primera vez
    for(int v : tree[u]){
        if(v == pa) continue;
        depth[v] = depth[u] + 1;
        DFS(v, u);
        id.pb(u); // Insertar cada vez que se regresa al
             nodo
    }
}

// O(1)
int LCA(int a, int b, rmq& st){
    if(start[a] > start[b]) swap(a, b); // Asegura que
         start[a] sea l y start[b] sea r
    return st.calc(start[a], start[b]).se; // Query a la
         sparse table
}

void solver(){
    int n; cin>>n;
    for1(i,n-1){
        int u, v; cin>>u>>v; u--; v--;
        tree[u].pb(v);
        tree[v].pb(u);
    }

    // Precalcular depth e id
    DFS(0, -1);

    // Llamar sparse table
    vii a;
    for0(i,sz(id)) a.pb({depth[id[i]], id[i]});
    rmq st; st.build(a);

    // Hallar LCA
    cout<<LCA(u, v, st)<<endl;
}
```

4.4 LCA and Binary Lifting

```
// Todo es 1-index

int LOG_N = 18; // techo del log2 de n

// Para cada nodo se calcula a donde llega tras realizar
     2^x pasos.
// Se necesita para hallar el LCA y para la funcion
     kStep.
// Complejidad: O(n * log(k))
void calc(int n, vi& pa, vector<vi>& up){
    for1(i,n) up[0][i] = pa[i];
    for1(bit,LOG_N-1){
        for1(i,n){
            up[bit][i] = up[bit-1][up[bit-1][i]];
        }
    }
}

int LCA(int a, int b, vi& depth, vector<vi>& up){
    if(depth[a] < depth[b]) swap(a, b);
    int k = depth[a] - depth[b];

    // Este ciclo pone a y b en el mismo nivel
    forn0(j,LOG_N) if(k & (1 << j)) a = up[j][a];

    if(a == b) return a;

    forn0(j,LOG_N){
        if(up[j][a] != up[j][b]){
            a = up[j][a];
            b = up[j][b];
        }
    }
    return up[0][a];
}

// Retorna el nodo al que se llega tras hacer k pasos
     partiendo desde s
// Complejidad: O(log(k))
int kStep(int s, int k, vector<vi>& up, vi& depth){
    if(depth[s] < k) return -1; // Si s tiene menos
         profundidad que la cantidad de pasos que me
         piden
    for0(bit,LOG_N){
        if(k & (1 << bit)) s = up[bit][s];
    }

    return s;
}

void solver(){
    int n; cin>>n;
    // Precalcular depth y pa con DFS
    vi depth(n+1), pa(n+1);

    // Precalcular la matriz up
    vector<vi> up(LOG_N, vi(n+1));
    calc(n, pa, up);

    // Preguntar LCA
    cout<<LCA(u, v, depth, up)<<endl;

    // Preguntar kth step
    cout<<kStep(node, steps, up, depth)<<endl;;
}
```

4.5 Heavy light decomposition

```
/*
- Puede ser tanto 1-index como 0-index (ojo con la raiz
    por defecto).
- Para hacer queries conmutativas y updates en caminos
    de un arbol y en subarboles.
- Si los valores van en las aristas ve = 1, si van en
    los nodos ve = 0.
- Ajustar el segtree segun el tipo de update y el tipo
    de queries, teniendo en cuenta que para suma de
    valores en un camino
  y updates normales (no lazy) normalmente se puede
       hacer simplemente con Euler Tour.
- Esta escrito para segtree lazy, ajustar si se quiere
    normal.
- Si las updates se dan con el indice de las aristas
    toca almacenar el indice de la arista que conecta u
     con su padre.
- Construccion: O(n), hallar los rangos del path: O(log
    n), update y query path: O(log^2n), update y query
    subtree: O(log n).
- Memoria: O(n).
- Para queries no conmutativas tocaria ordenar los
    rangos que devuelve path para que vayan de x hacia
    y (no se como xd).
*/
struct HLD {
    int n, t = 0; bool ve; vector<vi> g; vi par, dep,
        sub, head, pos, ord; segTree st;
```

```
    // g: lista de adyacencia, a: valor de cada nodo, r:
         raiz (1 por defecto), e: 1 si val en aristas,
        0 si val en nodos
    HLD(vector<vi>& adj, vl& a, int r = 1, bool e = 0) :
         n(sz(adj)), ve(e), g(adj), par(n), dep(n), sub
        (n), head(n), pos(n) {
        dfs1(r); head[r] = r; dfs2(r);
        vl b; for(int x : ord) b.pb(a[x]);
        st.init(sz(b)); st.build(b);
    }
    void dfs1(int x){
        sub[x] = 1; // sub es el tamano del subarbol
        for(int &y : g[x]){
            par[y] = x; dep[y] = dep[x] + 1;
            auto it = find(all(g[y]), x); if(it != g[y].
                end()) g[y].erase(it); // Si tiene
                arista al padre, la elimina
            dfs1(y); sub[x] += sub[y];
            if(sub[y] > sub[g[x][0]]) swap(y, g[x][0]);
        }
    }
    void dfs2(int x){
        pos[x] = t++; ord.pb(x); // pos[x] es la
            POSICION de x en el arreglo del segtree
        for(int y : g[x]){ head[y] = (y == g[x][0] ?
            head[x] : y); dfs2(y); } // g[x][0] es el
            hijo pesado de x
    }
    // rangos [l, r) a procesar (hacen parte del camino
        entre 'x' y 'y'), sin rangos vacios
    vii path(int x, int y){
        vii r;
        for(; head[x] != head[y]; y = par[head[y]]){
            if(dep[head[x]] > dep[head[y]]) swap(x, y);
            r.pb({pos[head[y]], pos[y] + 1});
        }
        if(dep[x] > dep[y]) swap(x, y);
        if(pos[x] + ve < pos[y] + 1) r.pb({pos[x] + ve,
            pos[y] + 1});
        return r;
    }
    void updatePath(int x, int y, ll v){ for(auto p :
        path(x, y)) st.update(p.fi, p.se, v); }
    ll queryPath(int x, int y){
        ll res = 0; // CHANGEEE: Valor neutro de cual
            sea la operacion
        for(auto p : path(x, y)) res += st.calc(p.fi, p.
            se); // CHANGEEE segun la operacion
        return res;
    }
    void updateSub(int x, ll v){ if(ve < sub[x]) st.
        update(pos[x] + ve, pos[x] + sub[x], v); }
    ll querySub(int x){ return ve < sub[x] ? st.calc(pos
        [x] + ve, pos[x] + sub[x]) : 0 /*VALOR NEUTROOO
        */; }
};

void solver(){
    // Inicializar
    int n, q, root; cin>>n>>q>>root;
    vector<vi> tree(n+1);
    // vector<vil> aux(n+1); // Si los pesos estan en
        las aristas
    for0(i, n-1){
        int u, v; cin>>u>>v;
        // ll w; cin>>w; // Si los pesos estan en las
            aristas
        tree[u].pb(v); tree[v].pb(u);
        // aux[u].pb({v, w}); aux[v].pb({u, w}); // Si
            los pesos estan en las aristas
    }
    vl val(n+1);
    for1(i, n) cin>>val[i]; // Si los pesos estan en los
         nodos, si no comentar

    // Si los pesos estan en las aristas:
    // function<void(int,int)> go = [&](int x, int p){
        // for(auto [y, c] : aux[x]) if(y != p){ val[y]
            = c; go(y, x); }
    // };
    // go(root, -1);

    HLD h(tree, val, root, valsInEdges); // 1 para
        valores en aristas, 0 para valores en vertices

    h.updatePath(x, y, valor); // Update path
    cout<<h.queryPath(x, y)<<endl; // Query path
    h.updateSub(x, valor); // Update subtree
    cout<<h.querySub(x)<<endl; // Query subtree
}
```

4.6 Max clique

```
/*
- Usar grafo 0-index.
- Halla el maximum clique en un grafo no dirigido.
- Con 50 nodos corre en menos de 1 segundo.
*/

const int N = 50;

typedef vector<bitset<N>> graph;
struct Maxclique {
    double limit = 0.025, pk = 0;
    struct Vertex {
        int i, d = 0;
    };
    typedef vector<Vertex> vv;
    graph e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old;
    void init(vv& r){
        for(auto& v : r) v.d = 0;
        for(auto& v : r) for(auto j : r) v.d += e[v.i][j
            .i];
        sort(all(r), [](auto a, auto b) {
            return a.d > b.d;
            });
        int mxD = r[0].d;
        for0(i,sz(r)) r[i].d = min(i, mxD) + 1;
    }
    void expand(vv& R, int lev = 1){
        S[lev] += S[lev - 1] - old[lev];
        old[lev] = S[lev - 1];
        while(sz(R)){
            if(sz(q) + R.back().d <= sz(qmax)) return;
            q.pb(R.back().i);
            vv T;
            for(auto v : R) if(e[R.back().i][v.i]) T.pb
                ({ v.i });
            if(sz(T)) {
                if(S[lev]++ / ++pk < limit) init(T);
                int j = 0, mxk = 1, mnk = max(sz(qmax) -
                    sz(q) + 1, 1);
                C[1].clear(), C[2].clear();
                for(auto v : T){
                    int k = 1;
                    auto f = [&](int i){
                        return e[v.i][i];
                        };
                    while(any_of(all(C[k]), f)) k++;
                    if(k > mxk) mxk = k, C[mxk + 1].
                        clear();
                    if(k < mnk) T[j++].i = v.i;
                    C[k].pb(v.i);
                }
                if(j > 0) T[j - 1].d = 0;
                forlr(k,mnk,mxk) for(int i : C[k])
                    T[j].i = i, T[j++].d = k;
                expand(T, lev + 1);
            }
            else if(sz(q) > sz(qmax)) qmax = q;
            q.pop_back(), R.pop_back();
        }
    }
    Maxclique(graph g) : e(g), C(sz(e) + 1), S(sz(C)),
        old(S) {
        for0(i,sz(e)) V.pb({i});
    }
    vector<int> solve(){ // returns the clique
        init(V), expand(V);
        return qmax;
    }
};

void solver(){
    // Construir el grafo
    int n; cin>>n;
    graph g(n);
    for0(i,n){
        for0(j,n){
            int num; cin>>num;
            g[i][j] = (num ? 1 : 0); // Lectura como
                matriz de adyacencia
        }
    }

    // Correr el algoritmo
    Maxclique mc(g);
    vi clique = mc.solve();
    cout<<sz(clique)<<endl; // Tamano del max clique

    // Vertices en el max clique
    for(int v : clique) cout<<v<<' ';
    cout<<endl;
}
```

4.7 Minimum vertex cover

```
/*
- Un Vertex Cover es un subconjunto de vertices CsubsetV
    tal
  que cada arista del grafo tiene al menos un extremo en
      C.
  Un minimum vertex cover es el conjunto C de menor
      tamano
  que cumple dicha condicion.
- Complejidad: en un grafo cualquiera es NP, pero para
  arboles, la complejidad es O(n).
*/

const int maxn = 2e5 + 1;
vector<vi> tree(maxn);
int dp[maxn][2];

void DFS(int u, int pa){
    // dp[u][1] guarda el minimo si u en el cover, 0 si
        no esta
    dp[u][0] = 0; dp[u][1] = 1;
    for(int v : tree[u]){
        if(v == pa) continue;
        DFS(v, u);
    }

    // Si u esta en el cover, sus hijos pueden estar o
        no.
    // Si u no esta en el cover, todos sus hijos tienen
        que estar
    for(int v : tree[u]){
        if(v == pa) continue;
```

```
        dp[u][0] += dp[v][1];
        dp[u][1] += min(dp[v][0], dp[v][1]);
    }
}

void solver(){
    int n; cin>>n;
    for(int i = 1; i < n; i++){
        int u, v; cin>>u>>v;
        tree[u].pb(v);
        tree[v].pb(u);
    }

    DFS(1, -1); // Da igual la raiz del arbol
    cout<<min(dp[1][0], dp[1][1])<<endl;
}
```

5 Flow

5.1 Dinic

```
/*
- Aplicado en grafos de flujo, dirigidos, con
    capacidades en las aristas.
- Complejidad O(v^2 * E).
- Solo se puede tirar el calculo una vez.
- Para grafos de red unitaria (cada arista tiene
    capacidad 1 y para cada nodo
  excepto s y t el grado de salida o entrada es de
      maximo uno): O(sqrt(V) * E).
- Para grafos de red de capacidad unitaria (todas las
    aristas tienen
  capacidad 1): O(sqrt(E) * E).

Aplicaciones:
- Maximo flujo
- Minimo corte:
  Es lo mismo que el maximo flujo. El conjunto de nodos
      alcanzables (partiendo
  desde source) sin pasar por aristas saturadas es un
      lado del corte (left), y
  los no alcanzables son el otro lado del corte (right).
       Las aristas que
  pertenecen al mincut son aquellas que van de left a
      right y estan saturadas.
- Edge-disjoint paths:
  Para hallar el maximo de caminos desde source hasta
      sink sin repetir aristas,
  se usa un flujo donde cada arista tiene capacidad 1.
      Para reconstruir estos
  caminos podemos guardar para cada nodo u una cola con
      los nodos v a los que
  este envia flujo, entonces mientras la cola de source
      no este vacia, tomamos
  el primer elemento v, nos movemos a la cola de v y
      sacamos el primer vertice x
  al que este envia flujo y asi sucesivamente hasta que
      llegamos a sink.
- Node-disjoint paths:
  Se resuelve de la misma manera que Edge-disjoint paths
       pero dividiendo los
  nodos en u_in (2*u) y u_out (2*u + 1) y creando una
      arista con capacidad 1
  entre ellos. Si hay una arista entre u y v, conectamos
       u_out con vin.
- Minimum vertex cut:
  Para hallar el minimum vertex cut tendriamos que
      llamar Dinic con 2*n + 2
  y anadir una arista entre cada 2*i y 2*i + 1 con
      capacidad de 1, para
  0 <= i < n. Despues se agregan aristas con capacidad 1
       segun las condiciones
  del ejercicio o del grafo original. Si buscamos anadir
       una arista desde un
  vertice u hasta un vertice v, usamos: graph.addEdge(2*
      u + 1, 2*v, 1).
  Despues agregamos aristas desde el nodo 2*n hacia
      todos los nodos iniciales
  y desde todos los nodos finales hacia el nodo 2*n + 1.
  Finalmente, usamos graph.calc(2*n, 2*n + 1), para
      hallar el maximo flujo
  desde 2*n (que cumple la funcion de source-fuente),
      hasta 2*n + 1 (que cumple
  la funcion de sink-sumidero). El valor del maximo
      flujo corresponde al valor
  del minimum vertex cut.
- Node-disjoint path cover en un DAG:
  Un path cover es un conjunto de caminos en un grafo
      tal que cada nodo pertenece
  a al menos un camino. Un Node-disjoint path cover
      quiere decir que cada nodo
  debe pertenecer a exactamente un camino.
  Para hallar el minimum Node-disjoint path cover
      podemos crear un grafo bipartito
  donde separamos cada nodo u en u_in (2*u)(right) y
      u_out (2*u + 1)(left), entonces
  para cada arista u, v conectamos u_out con vin.
      Despues hallamos el matching y
  esas son las aristas que vamos a elegir para armar el
      Node-disjoint path cover,
  entonces el total de caminos sera n - c, siendo n la
      cantidad de nodos en el grafo
  original y c la cantidad de aristas en el matching.
  Las aristas que se usan en los caminos de la particion
       son las aristas del matching,
  para saber donde empiezan los caminos, tomo los u tal
      que u_in no fue matcheado.
- General path cover en un DAG:
  El minimum path cover se puede hallar de manera
      similar al minimum node-disjoint
  path cover, pero esta vez vamos a agregar una arista
      de u_out hacia vin si en el
  grafo original se puede llegar desde u hasta v (
      posiblemente a traves de multiples
  aristas).
- Maximum Antichain:
  Una antichain es un conjunto de nodos de un grafo en
      el cual ningun nodo tiene un
  camino hacia cualquier otro usando las aristas del
      grafo. En un DAG, la longitud
  de la maximum antichain es igual a la longitud del
      minimum path cover. Para
  reconstruirla hallamos el minimum vertex cover M, los
      nodos tales que u_in no
  pertenece a M y u_out tampoco, pertenecen a la maximum
       antichain.
- Maquinas y tareas (problema de asignacion):
  Tengo un conjunto B de tareas que me dan un beneficio
      B(t), puedo comprar un
  conjunto de maquinas M por un costo M(t). Para
      realizar la tarea B tengo que comprar
  ciertas maquinas. Cada maquina la puedo utilizar para
      realizar cualquier cantidad de
  tareas. Para resolverlo asumo que ya me pagaron las
      tareas y tengo un beneficio total
  X, entonces no realizar una tarea tiene un costo B(t).
       Construyo un grafo bipartito,
  conecto S con cada maquina con una capacidad M(t),
      conecto cada tarea con T con una
  capacidad B(t), y finalmente para cada tarea B y cada
      maquina M requerida para
  realizar la tarea B agrego una arista con capacidad
      infinita. Finalmente, el beneficio
  maximo es X menos el corte minimo. Si la arista S, M
      forma parte del corte, entonces
  compro la maquina M, si la arista B, T hace parte del
      corte, entonces no realizo la
  tarea B.
  En general, si tengo un problema que consiste en
      elegir un conjunto de nodos V' (donde
  cada nodo tiene un valor w(v)) tal que para toda
      arista u, v, v pertenece a V' si y solo
  si u tambien pertenece a V', y se maximice el valor de
       los nodos elegidos, lo puedo
  resolver asi: agrego nodos S y T, agrego una arista S
      con capacidad abs(w(v)), v si w(v) < 0,
  una arista v, T con capacidad w(v) si w(v) > 0, y
      todas las aristas u, v del grafo original
  con capacidad inf. Los nodos elegidos (V') son los
      alcanzables desde S en la red residual, el
  valor de V' es la suma de los w(v) postivos menos el
      valor del corte minimo.
- Eleccion con restricciones y pesos:
  Si tengo un problema en el cual elegir determinado
      nodo hace que no pueda elegir otros, y
  quiero elegir un conjunto de nodos que maximice la
      suma de w(v) para cada v elegido, lo puedo
  resolver separando los nodos en 2 lados U y V (para
      formar un bipartito) asegurandome que
  podria elegir todos los nodos de alguno de los lados
      sin ningun problema (que los nodos
  que pertenezcan al mismo lado no se restringen entre
      si), agrego aristas S, u con capacidad w(u)
  para cada u que pertenezca a U, agrego aristas v, T
      con capacidad w(v) para cada v que pertenezca
  a V, y aristas u, v con capacidad infinita para cada
      par de nodos tal que si elijo u no puedo
  elegir v y viceversa. La respuesta es la suma w(v)
      para cada v que pertenece al grafo, menos el
  valor del minimo corte.
- Max clique:
  Si el grafo complemento (es aquel grafo donde solo
      estan las aristas que no hacen parte del grafo
  original) es bipartito, se construye y se halla un MIS
       .

Para bipartitos:
- Maximum Matching:
  Es la maxima cantidad de aristas que se pueden tomar
      sin que compartan algun vertice.
  Se conectan todos los nodos de un lado left a un nuevo
       nodo source y todos los
  nodos de otro lado right a un nuevo nodo sink, le hace
       que las aristas del grafo
  original tengan direccion desde su correspondiente
      nodo en left hacia su
  correspondiente nodo en right y finalmente se calcula
      el flujo. Las aristas de
  left hacia right con flujo = 1 son las que haran parte
       del matching.
- Minimum vertex cover (MVC):
  Es el minimo conjunto de nodos tal que cada arista del
       grafo tiene al menos
  uno de sus nodos en dicho conjunto. El tamano del
      minimum vertex cover es el mismo
  del maximum matching. Para hallar cuales vertices
      pertenecen podemos ver entre la lista
  de adyacencia de source cuales nodos hacen parte de
      right en el corte, ademas revisamos
  para cada vertice de left si este tiene a sink en su
      lista de adyacencia, estos tambien
  hacen parte del minimum vertex cover.
- Maximum independent set (MIS):
  Es el mayor conjunto de nodos tal que no hay 2 nodos
      del conjunto conectados por
  una arista. Todos los nodos que no hacen parte del
      minimum vertex cover hacen parte
  del maximum independent set, por lo tanto el tamano de
       este es el total de nodos
  del grafo menos el tamano del maximum matching.
```

```
- Minimun edge cover:
  Es el minimo conjunto de aristas tal que cada nodo del
       grafo tiene al menos una de sus
  aristas en dicho conjunto. Esta compuesto por las
       aristas del matching + para todo
  vertice no cubierto aun, tomar cualquier arista de
       este.
*/

struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you
         need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0){
    adj[a].pb({b, sz(adj[b]), c, c});
    adj[b].pb({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f){
    if(v == t || !f) return f;
    for(int& i = ptr[v]; i < sz(adj[v]); i++){
      Edge& e = adj[v][i];
      if(lvl[e.to] == lvl[v] + 1)
        if(ll p = dfs(e.to, t, min(f, e.c))){
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t){
    ll flow = 0; q[0] = s;
    for(ll L = 0; L < 31; L++) do { // 'int L=30' maybe
         faster for random data
      lvl = ptr = vector<int> (sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while(qi < qe && !lvl[t]){
        int v = q[qi++];
        for(Edge e : adj[v])
          if(!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while(ll p = dfs(s, t, LLONG_MAX)) flow += p;
        } while(lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; } // 1
       si es parte del lado de source
};

void solver(){
    int n; cin>>n;
    Dinic graph(n + 2); // Crear un grafo para usar
         Dinic

  // Agregar una arista desde s hasta d con capacidad c
  // Tener en cuenta que es un grafo dirigido, si se
       quiere un grafo no dirigido,
  // agregar la misma arista en direccion contraria
    graph.addEdge(s, d, c);

    graph.calc(source, sink); // Maximo flujo desde
         source hast sink

    // Para reconstruir el flujo enviado por cuales
         aristas
  for0(u,n){
    for(auto &e : graph.adj[u]){
      int v = e.to;
      ll f = e.flow(); // f = flujo enviado desde u a v

      // Mostrar solo aristas "reales" (no las de
           retroceso)
      if(f > 0 && e.oc > 0){
        cout << "Flujo de " << u << " a " << v << ": "
             << f << endl;
      }
    }
  }

  // Hallar los vertices que pertenecen al minimum
       vertex cover:
  // Sea source = 0, el lado izquierdo del bipartito los
       nodos de 1 hasta n,
  // el lado derecho los nodos de n+1 hasta 2*n y sink =
       2*n + 1
  for(auto &e : graph.adj[0]){ // Recorremos la lista de
       adyacencia de source (lado izquierdo del
       bipartito)
      int v = e.to;
      if(!graph.leftOfMinCut(v)) cout<<1<<' '<<v<<endl
          ; // Si v no esta en left en el corte, hace
           parte del MVC
    }
    forlr(u,n+1,2*n){
    // Recorremos los nodos que hacen parte de left en
         el corte y en la derecha del bipartito
      if(!graph.leftOfMinCut(u)) continue;
      for(auto &e : graph.adj[u]){
      int v = e.to;
        if(v != 2*n + 1) continue; // Si tiene
             arista hacia sink, hace parte del MVC
          cout<<2<<' '<<u - n<<endl;
          break;
    }
    }
}
```

5.2 Push Relabel

```
/*
- Complejidad: O(v^2*sqrt(e)) o (v^3)
- Si las constantes son bajas, usar Dinic (mas corto).
- Para bipartitos con capacidad 1, usar Dinic (mas
    eficiente).
- Si se requiere hallar el flujo que se envia por cada
    arista o cuales
  nodos estan en cada lado del corte, usar Dinic.
*/

struct HLPP {
    typedef ll F; // flow type
    struct Edge { int to, rev; F f; };
    const F INF = numeric_limits<F>::max();

    int N, s, t;
    vector<vector<Edge>> adj;
    vector<vi> lst, gap;
    vector<F> excess;
    vi height, cnt;
    int highest, work;

    HLPP(int n) : N(n), s(0), t(0), adj(n), lst(n), gap(
        n),
                  excess(n), height(n), cnt(n), highest
                       (0), work(0) {}

    void ae(int u, int v, F cap){
        assert(cap >= 0);
        Edge a{v, sz(adj[v]), cap}, b{u, sz(adj[u]), 0};
        adj[u].pb(a), adj[v].pb(b);
    }
    void updHeight(int v, int nh){
        work++;
        if(height[v] != N) cnt[height[v]]--;
        height[v] = nh;
        if(nh == N) return;
        cnt[nh]++, highest = nh;
        gap[nh].pb(v);
        if(excess[v] > 0) lst[nh].pb(v);
    }
    void globalRelabel(){
        work = 0;
        for0(i,N) height[i] = N, cnt[i] = 0;
        for0(i,highest) lst[i].clear(), gap[i].clear();
        height[t] = 0;
        queue<int> q({t});
        while(sz(q)){
            int v = q.front(); q.pop();
            for(auto &e : adj[v]){
                if(e.to != s && height[e.to] == N && adj
                    [e.to][e.rev].f > 0){
                    q.push(e.to), updHeight(e.to, height
                        [v] + 1);
                }
            }
            highest = height[v];
        }
    }
    void push(int v, Edge& e){
        if(excess[e.to] == 0 && height[e.to] < N) lst[
            height[e.to]].pb(e.to);
        F df = min(excess[v], e.f);
        e.f -= df, adj[e.to][e.rev].f += df;
        excess[v] -= df, excess[e.to] += df;
    }
    void discharge(int v){
        if(height[v] >= N) return;
        int nh = N;
        for(auto &e : adj[v]){
            if(e.f > 0){
                if(height[v] == height[e.to] + 1){
                    push(v, e);
                    if(excess[v] <= 0) return;
                } else nh = min(nh, height[e.to]+1);
            }
        }
        if(cnt[height[v]] > 1) updHeight(v, nh);
        else{
            forlr(i,height[v],highest) {
                for(auto &j : gap[i]) updHeight(j, N);
                gap[i].clear();
            }
        }
    }
    F maxFlow(int _s, int _t){
        s = _s, t = _t; if(s == t) return -1;
        for0(i,N) excess[i] = 0;
        excess[s] = INF, excess[t] = -INF;
        globalRelabel();
        for(auto &e : adj[s]) push(s,e);
        for(; highest >= 0; highest--){
            while(sz(lst[highest])){
                int v = lst[highest].back();
                lst[highest].pop_back();
                discharge(v);
                if(work > 4*N) globalRelabel();
            }
        }
        return excess[t] + INF;
    }
};

void solver(){
    int n, m; cin>>n>>m;
  HLPP graph(n+1); // Crear el grafo
    for0(i,m){
```

```
        int u, v; ll c; cin>>u>>v>>c;
        graph.ae(u, v, c); // Agregar arista dirigida u
            -> v con capacidad c
    }
    ll flow = graph.maxFlow(s, t); // Si s == t, retorna
        -1
    cout<<flow<<endl;
}
```

5.3 Hungarian

```
/*
- 0-index.
- Complejidad: O(J * W^2)
- Suele ser un poco mas eficiente que el algoritmo de
    MaxFlow MinCost.
- Sea n la cantidad en el lado izquierdo y m la cantidad
    en el lado
  derecho, se requiere n >= m, es decir, deben haber
      igual o mas
  workers que jobs.
- Minimo costo para el maximum matching de un bipartito
    ponderado.
- Para maximo costo se puede guardar los costos en
    negativo y despues
  negar la respuesta.
- Si el costo total es el producto de cada asignacion,
    pasar los
  costos a algun logaritmo antes de tirar el matching.
- Para explicar el algoritmo se documenta utilizando
    worker (W) como L
  y job (J) como R. Cada trabajador cobra cierta
      cantidad por realizar
  cierto trabajo sin embargo cada trabajador solo puede
      realizar un
  trabajo. Se halla el menor costo para realizar todos
      los trabajos.
*/

// Si los costos son grandes cambiar por ll
int ckmin(int &a, int b){ return a > b ? ((a = b), true)
    : false;}

/**
 * return the jobs of each worker in the optimal
     assignment,
 * or -1 if the worker is not assigned
 */
template <class T> vi hungarian(const vector<vector<T>>
    &C){
  int J = C.size();
  int W = C[0].size();
  assert(J <= W); // Verifica que el lado izquierdo (W)
      sea >= al lado derecho (J)

  // job[w] = job assigned to w-th worker, or -1 if no
      job assigned
  // note: a W-th worker was added for convenience
  vi job(W + 1, -1);
  vector<T> h(W);  // Johnson potentials

  const T inf = numeric_limits<T>::max();
  // assign j_cur-th job using Dijkstra with potentials
  for0(j_cur,J){
    int w_cur = W;  // unvisited worker with minimum
        distance
    job[w_cur] = j_cur;

    vector<T> dist(W + 1, inf);  // Johnson-reduced
        distances
    dist[W] = 0;
    vb vis(W + 1);      // whether visited yet
    vi prv(W + 1, -1);  // previous worker on shortest
        path
    while(job[w_cur] != -1){   // Dijkstra step: pop min
        worker from heap
      T min_dist = inf;
      vis[w_cur] = 1;
      int w_next = -1;  // next unvisited worker with
          minimum distance

      // consider extending shortest path by w_cur ->
          job[w_cur] -> w
      for0(w,W){
        if(!vis[w]){
          // sum of reduced edge weights w_cur -> job[
              w_cur] -> w
          T edge = C[job[w_cur]][w] - h[w];
          if(w_cur != W){
            edge -= C[job[w_cur]][w_cur] - h[w_cur];
            assert(edge >= 0);
          }
          if(ckmin(dist[w], dist[w_cur] + edge)) { prv[w
              ] = w_cur; }
          if(ckmin(min_dist, dist[w])) { w_next = w; }
        }
      }
      w_cur = w_next;
    }

    for(int w = 0; w < W; w++){  // update potentials
      ckmin(dist[w], dist[w_cur]);
      h[w] += dist[w];
    }

    while(w_cur != W){  // update job assignment
      job[w_cur] = job[prv[w_cur]];
      w_cur = prv[w_cur];
    }
  }

  return job; // Retorna la mejor asignacion encontrada
}

void solver(){
  int n, m; cin>>n>>m; // n < m, si no, swapear o
      rellenar con workers falsos.

  /* - Matriz con el costo de que el iesimo job sea
      realizado por el jesimo worker
     - Se asume que cada worker puede realizar cada job.
         Si no:
       * Inicializar la matriz en infinitos.
     * Al sumar el costo total, la cantidad de aristas
         en el matching (y cuales)
       tener en cuenta que aquellas donde el costo sea
           inf no se deben contar.
     * Cambiar la lectura segun sea el caso */
  vector<vi> c(n, vector<int>(n));
  for0(i,n){ // ith job
    for0(j,m){ // jth worker
          cin>>c[i][j];
        }
  }

  vi mat = hungarian(c); // Halla el matching. mat[i] es
      el job realizado por el worker i

  // Se suma segun el costo de que el job mat[i] sea
      realizado por el worker i
  int cost = 0;
  for0(i,n) cost += c[mat[i]][i];
  cout<<cost<<endl;
  for0(i,n) cout<<mat[i]<<' '<<i<<endl; // El job mat[i]
      es realizado por el worker i
}
```

5.4 Min Cost Max Flow

```
/**
 * Author: Stanford
 * Date: Unknown
 * Source: Stanford Notebook
 * Description: Min-cost max-flow.
 *  If costs can be negative, call setpi before maxflow,
      but note that negative cost cycles are not
     supported.
 *  To obtain the actual flow, look at positive values
     only.
 * Status: Tested on kattis:mincostmaxflow, stress-
     tested against another implementation
 * Time: O(F*E*log(V)) where F is max flow. O(V*E) for
     setpi.
 * 0-index.
*/
#include <ext/pb_ds/priority_queue.hpp>

struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  };
  int N;
  vector<vector<edge>> ed;
  vi seen;
  vl dist, pi;
  vector<edge*> par;

  MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N),
      par(N) {}

  void addEdge(int from, int to, ll cap, ll cost){
    if(from == to) return;
    ed[from].pb(edge{from,to, sz(ed[to]), cap, cost, 0});
    ed[to].pb(edge{to, from, sz(ed[from])-1, 0, -cost,
        0});
  }

  void path(int s){
    fill(all(seen), 0);
    fill(all(dist), INFL);
    dist[s] = 0; ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({0, s});

    while(!q.empty()){
      s = q.top().se; q.pop();
      seen[s] = 1; di = dist[s] + pi[s];
      for(edge& e : ed[s]) if(!seen[e.to]){
        ll val = di - pi[e.to] + e.cost;
        if(e.cap - e.flow > 0 && val < dist[e.to]){
          dist[e.to] = val;
          par[e.to] = &e;
          if(its[e.to] == q.end())
            its[e.to] = q.push({-dist[e.to], e.to});
          else
            q.modify(its[e.to], {-dist[e.to], e.to});
        }
      }
    }
    for0(i,N) pi[i] = min(pi[i] + dist[i], INFL);
  }

  pll maxflow(int s, int t){
    ll totflow = 0, totcost = 0;
    while(path(s), seen[t]){
      ll fl = INFL;
      for(edge* x = par[t]; x; x = par[x->from])
        fl = min(fl, x->cap - x->flow);
```

```
      totflow += fl;
      for(edge* x = par[t]; x; x = par[x->from]){
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
      }
    }
    for0(i,N) for(edge& e : ed[i]) totcost += e.cost * e
        .flow;
    return {totflow, totcost/2};
  }
  // If some costs can be negative, call this before
      maxflow:
  void setpi(int s){ // (otherwise, leave this out)
    fill(all(pi), INFL); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while(ch-- && it--)
      for0(i,N) if(pi[i] != INFL)
          for(edge& e : ed[i]) if(e.cap)
            if((v = pi[i] + e.cost) < pi[e.to])
              pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
  }
};

void solver(){
    int n, m; cin>>n>>m;
    MCMF graph(n + 1); // Inicializa la red

  // Agregar aristas u -> v, con capacidad cap y costo
      cost por cada unidad de flujo
    for0(i,m){
        int u, v; ll cap, cost; cin>>u>>v>>cap>>cost;
        graph.addEdge(u, v, cap, cost);
    }

    cout<<graph.maxflow(s, t).se<<endl;
}
```

5.5 Blossom

```
/*
- Complejidad: O(|E||V|^2)
- Maximum matching en cualquier grafo no dirigido
- Minimum edge cover:
- Es el minimo conjunto de aristas tal que cada nodo del
      grafo tiene al
  menos una de sus aristas en dicho conjunto. Esta
      compuesto por las
  aristas del matching + para todo vertice no cubierto
      aun, tomar
  cualquier arista de este.
*/

const int MAXE = ; // MAXE = 2*n*n
struct network {
    struct struct_edge { int v; struct_edge * n; };
    typedef struct_edge* edge;
    int n;
    struct_edge pool[MAXE];
    edge top;
    vector<edge> adj;
    queue<int> q;
    vi f, base, inq, inb, inp, match;
    vector<vi> ed;
    network(int n) : n(n), match(n, -1), adj(n), top(
        pool), f(n), base(n),
                     inq(n), inb(n), inp(n), ed(n, vi(n)
                         ) {}
    void add_edge(int u, int v){
        if(ed[u][v]) return;
        ed[u][v] = 1;
        top->v = v, top->n = adj[u], adj[u] = top++;
        top->v = u, top->n = adj[v], adj[v] = top++;
    }
    int get_lca(int root, int u, int v){
        fill(inp.begin(), inp.end(), 0);
        while(1){
            inp[u = base[u]] = 1;
            if(u == root) break;
            u = f[match[u]];
        }
        while(1){
            if(inp[v = base[v]]) return v;
            else v = f[match[v]];
        }
    }
    void mark(int lca, int u){
        while(base[u] != lca){
            int v = match[u];
            inb[base[u]] = 1;
            inb[base[v]] = 1;
            u = f[v];
            if(base[u] != lca) f[u] = v;
        }
    }
    void blossom_contraction(int s, int u, int v){
        int lca = get_lca(s, u, v);
        fill(all(inb), 0);
        mark(lca, u); mark(lca, v);
        if(base[u] != lca) f[u] = v;
        if(base[v] != lca) f[v] = u;
        for0(u,n){
            if(inb[base[u]]){
                base[u] = lca;
                if(!inq[u]){
                    inq[u] = 1;
                    q.push(u);
                }
            }
        }
    }
    int bfs(int s){
        fill(all(inq), 0);
        fill(all(f), -1);
        for(int i = 0; i < n; i++) base[i] = i;
        q = queue<int>();
        q.push(s);
        inq[s] = 1;
        while(sz(q)){
            int u = q.front(); q.pop();
            for(edge e = adj[u]; e; e = e->n){
                int v = e->v;
                if(base[u] != base[v] && match[u] != v){
                    if((v == s) || (match[v] != -1 && f[
                        match[v]] != -1))
                        blossom_contraction(s, u, v);
                    else if(f[v] == -1){
                        f[v] = u;
                        if(match[v] == -1) return v;
                        else if(!inq[match[v]]){
                            inq[match[v]] = 1;
                            q.push(match[v]);
                        }
                    }
                }
            }
        }
        return -1;
    }
    int doit(int u){
        if(u == -1) return 0;
        int v = f[u];
        doit(match[v]);
        match[v] = u; match[u] = v;
        return u != -1;
    }
    /// (i < net.match[i]) => means match
    int maximum_matching(){
        int ans = 0;
        for0(u,n)
        ans += (match[u] == -1) && doit(bfs(u));
        return ans;
    }
};

void solver(){
    int n, m; cin>>n>>m;

    // Inicializar
    network graph(n+1);
    for0(i,m){
        int u, v; cin>>u>>v;
        graph.add_edge(u, v);
    }

    int len = graph.maximum_matching(); // Hallar el
        matching
    cout<<len<<endl; // Tamano del matching maximo
    for1(i,n){ // Aristas en el matching
        // La segunda condicion asegura que cada arista
            se escriba solo una vez
        if(graph.match[i] != -1 && i < graph.match[i]){
            cout<<i<<' '<<graph.match[i]<<endl;
        }
    }
}
```

6 DP

6.1 Divide and Conquer

```
/*
DP con divide and conquer optimization.
- Para dp de la forma dp[i][j] = min(dp[i-1][m-1] + C[m
    ][j]), donde:
  * 1 <= i <= k
  * 0 <= j <= n
  * 1 <= m <= j
  * C[m][j] es una funcion de costo evaluable
      rapidamente.
- La interpretacion tipica (pero no unica) es que se
    quieren partir
  los elementos en grupos, entonces dp[i][j] es el
      minimo costo de
  partir los primeros 'j' elementos en exactamente 'i'
      grupos no
  vacios, donde el ultimo grupo es [m...j] y C[m][j] es
      el costo de
  este grupo.
- COMPLEJIDAD se reduce de O(k*n*n) a O(k*n*log(n)).
  (Mas la complejidad de hallar la funcion de costo).
- Para poder usarlo se requiere que:
  sea opt(i,j) el valor de 'm' que minimiza dp[i][j],
  opt(i,j) <= opt(i,j+1) para todo par i,j.
- Esa restriccion se garantiza si C cumple la
    desigualdad del
  cuadrilatero:
  para a <= b <= c <= d, C(a,c) + C(b,d) <= C(a,d) + C(b
      ,c).
  Sin embargo usualmente no se hace la demostracion sino
      que se
  intuye o compara contra una solucion O(k*n*n) en un
      caso pequeno.
  Una forma de reformular esta desigualdad es:
  cuando se incrementa el punto 'r' del rango,
      decrementar el punto
  'l' hace que sea mas costoso. Formalmente:
  para todo l<=r, C(l-1,r) - C(l,r) <= C(l-1,r+1) - C(l,
```

```
      r+1).
MAXIMIZAR:
- Si en lugar de minimizar se quiere maximizar:
  Para toda la tabla C hacer C[i][j] *= -1. Esta tabla C
       negada es
  la que debe cumplir la desigualdad del cuadrilatero.
  El resultado final tambien hacerle *= -1.
APLICACIONES:
- Tipico en problemas de partir un arreglo en 'k'
    segmentos
  minimizando la suma de costos como suma de cuadrados,
       distancia
  a la mediana, distancia a una particion.
*/

const int maxn = 1e3 + 1, maxk = 20 + 1;
ll C[maxn][maxn]; // se quita si se puede hallar facil
    para cada m,j
ll dp[maxk][maxn];

void compute(int k, int l, int r, int optl, int optr){
    if(l > r) return;
    int mid = (l+r) >> 1, opt = -1; ll best = INFL;
    for(int m = optl; m <= min(mid, optr); m++){
        ll aux = dp[k-1][m-1] + C[m][mid];
        if(aux < best) best = aux, opt = m; // Desempata
             el menor m
    }
    dp[k][mid] = best;
    compute(k, l,     mid-1, optl, opt);  // j menores:
        opt a la izquierda
    compute(k, mid+1, r,     opt,  optr); // j mayores:
        opt a la derecha
}

void solver(){
    int n, k; cin>>n>>k;
    for1(m,n) forlr(j,m,n) cin>>C[m][j];

    for0(j,n+1) dp[0][j] = INFL; // No se pueden partir
        en 0 grupos
    dp[0][0] = 0; // 0 elementos si se pueden partir en
        0 grupos
    for1(i,k){ // Hallar el dp para 'i' grupos
        for0(j,n+1) dp[i][j] = INFL; // Inicializar en
            INFL
        compute(i, i, n, i, n);
    }

    cout<<dp[k][n]<<endl;
}
```

6.2 Knuth

```
/*
DP con optimizacion de Knuth, pasa de O(n^3) a O(n^2).
- Es un caso especial de DP en rangos, se aplica en
  transiciones de la forma:
  * dp[l][r] = C[l][r] + min(dp[l][i] + dp[i+1][r]),
  * con l <= i < r.
  * 'C' es una funcion de costos.
- Tener en cuenta que no son puntos desde 'l' hasta
  'r' sino los rangos desde 'l' hasta 'r', donde por
  ejemplo el rango 'l' abarca una serie de puntos
  [left, right]. Entonces dp[l][r] busca el valor
  minimo para el conjunto de rangos
  l, l+1, l+2, ..., r-2, r-1, r.
CONDICION:
- Sea opt[l][r] el menor valor de 'i' que minimiza
  dp[l][r], se debe cumplir que:
  opt[l][r-1] <= opt[l][r] <= opt[l+1][r].
- Dicha condicion se cumple cuando la funcion de costo
  'C' satisface que para todo a <= b <= c <= d:
  * C[b][c] <= C[a][d].
  * C[a][c] + C[b][d] <= C[a][d] + C[b][c].
- Sin embargo usualmente no se hace la demostracion
  sino que se intuye o compara contra una solucion
  O(n^3) en un caso pequeno.

- 0-index.
MAXIMIZAR
- Para toda la tabla C hacer C[i][j] *= -1. Esta tabla
  'C' negada es la que debe cumplir las condiciones.
- Al resultado final tambien hacerle *= -1.
*/

const int maxn = 1e3 + 5;
ll dp[maxn][maxn], C[maxn][maxn];
int opt[maxn][maxn];

void solver(){
    int n; cin>>n;

    // LEER O PRECALCULAR C

    // Rangos individuales, tener en cuenta que no
        siempre dp[i][i] tiene porque ser '0'
    for0(i,n) dp[i][i] = 0, opt[i][i] = i;

    forlr(len,2,n){
        forlr(l,0,n-len){
            int r = l+len-1;
            dp[l][r] = INFL;
            int lo = opt[l][r-1], hi = min(opt[l+1][r],
                r-1);
            forlr(i,lo,hi){
                ll cur = dp[l][i] + dp[i+1][r];
                if(cur < dp[l][r]){
                    dp[l][r] = cur;
                    opt[l][r] = i;
                }
            }
            dp[l][r] += C[l][r];
        }
    }
    cout<<dp[0][n-1]<<endl;
}
```

6.3 Convex Hull Trick

```
/*
- Complejidad: add O(1), get_max O(log n).
- Para dp donde las transiciones son en forma de recta:
  dp_i = v_i + max_o_min(dp_j + a_i * m_j), donde:
  * 'j' es un indice menor que 'i'.
  * los valores 'm' representan las pendientes y estan
    ordenados ascendente o descendentemente.
  * los valores 'a' son el punto donde se evalua la
      funcion.
  * 'dp_j' es la interseccion en el eje Y.
  * 'v_i' es un valor que solo depende de 'i'.
  * 'dp_i' es el mejor resultado usando los valores de '
      i'.
- Tabla para saber como hacer las queries segun el caso
    que se busca:
Problema  Pendiente     Recta     Respuesta
max         ascendentes     (m, b)      get_max(x)
min         descendentes  (-m, -b)  -get_max(x)
max         descendentes  (-m, b)     get_max(-x)
min         ascendentes     (m, -b)     -get_max(-x)

- Las queries no necesitan estar en orden.
- Como respuesta se toma el valor mayor (o menor) de
    todas las
  posiciones del dp (a no ser que me pidan usar el
      elemento 'n' en la
  respuesta, en cuyo caso se toma 'dp_(n-1)').
  Si se permite no usar ningun elemento, tener en cuenta
       que la
  respuesta podria ser '0' o algun elemento neutro.
*/

struct line {
    ll m, b;
    ll eval(ll x) const { return m * x + b; }
    ld inter(const line &l) const { return (ld)(b - l.b)
        / (l.m - m); }
};

struct cht {
    vector<line> lines;
    vector<ld> inter;
    int n;
    bool ok(const line &a, const line &b, const line &c)
         const {
        return a.inter(c) > a.inter(b);
    }
    void add(const line &l){
        n = sz(lines);
        if(n && lines.back().m == l.m && lines.back().b
            >= l.b) return;
        if(n == 1 && lines.back().m == l.m && lines.back
            ().b < l.b) lines.pop_back(), n--;
        while(n >= 2 && !ok(lines[n-2], lines[n-1], l)){
            --n;
            lines.pop_back(); inter.pop_back();
        }
        lines.pb(l); n++;
        if(n >= 2) inter.pb(lines[n-2].inter(lines[n-1])
            );
    }
    ll get_max(ld x){
        if(lines.empty()) return LLONG_MIN;
        if(sz(lines) == 1) return lines[0].eval(x);
        int pos = lb(all(inter), x) - inter.begin();
        return lines[pos].eval(x);
    }
};

void solver(){ // Ejemplo con el caso de hallar maximos
    con pendientes ascendentes
    int n; cin>>n;
    // Si me dan 'm' desordenado, agrupar los 3 vectores
        en un vector {m, a, v} y ordenar
    // (solo si ordenar se permite, de lo contrario usar
        CHT dinamico)
    vi v(n), a(n), m(n);
    for0(i,n) cin>>v[i];
    for0(i,n) cin>>a[i];
    for0(i,n) cin>>m[i];

    vi dp(n); dp[0] = v[0];
    cht h; h.add({m[0], dp[0]});

    for1(i,n-1){
        dp[i] = v[i] + max(0LL, h.get_max(a[i])); // max
            con '0' por si conviene no tomar ningun
            elemento anterior
        h.add({m[i], dp[i]});
    }
}
```

6.4 Convex Hull Trick dinamico

```
/* NO TESTED
CHT dinamico (no importa el orden en que se agregan las
    rectas).
- Complejidad: add O(log(n)), get_max O(log(n)).
- Uso tipico para dp en arboles con small-to-large.
- Solo para 'x' entero.
*/

struct line {
    ll m, b;
    mutable ll p;                       // optima para x
```

```
        <= p
    bool operator<(const line &o) const { return m < o.m
        ; }
    bool operator<(ll x) const { return p < x; }
};

struct dyn_cht : multiset<line, less<>> {
    static ll fdiv(ll a, ll b) { return a / b - ((a ^ b)
        < 0 && a % b); } // piso

    // fija x->p (frontera con y); true si x tapa por
        completo a y
    bool isect(iterator x, iterator y){
        if(y == end()) { x->p = INFL; return false; }
        if(x->m == y->m) x->p = (x->b > y->b ? INFL : -
            INFL);
        else x->p = fdiv(y->b - x->b, x->m - y->m);
        return x->p >= y->p;
    }
    void add(ll m, ll b){
        auto z = insert({m, b, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
                                // limpia a la derecha
        if (x != begin() && isect(--x, y)) isect(x, y =
            erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
                // limpia a la izquierda
            isect(x, erase(y));
    }
    ll get_max(ll x){ // Falla si el hull esta vacio
        assert(!empty());
        auto l = *lower_bound(x);
        return l.m * x + l.b;
    }
};

void solver(){
    dyn_cht h;

    // Uso para maximo
    h.add(m, b);          // en cualquier orden de
        pendientes
    ll v = h.get_max(x);  // x en cualquier orden,
        intercalado con add

    // Uso para minimo
    h.add(-m, -b);
    ll v = -h.get_max(x);
}
```

6.5 Digit DP

```
/*
- Se usa en ejercicios en los que se pide contar la
    cantidad de numeros
  en un rango [L, R] muy grande (entre 1 y 10^18) cuyos
      digitos cumplen
  determinadas condiciones.
- Se hace un dp a traves de los digitos de cada numero,
    con una
  dimension para saber en que digito vamos, otra para
      guardar si ya nos
  aseguramos que el numero sea menor al mayor numero en
      rango y mas
  dimensiones dependiendo de las condiciones del
      ejercicio.
- Se calcula la respuesta en rango: ansL = (0, L - 1] y
    ansR = (0, R] y
  la respuesta definitiva es ansR - ansL.
- En este ejemplo se quiere calcular la suma de los
    digitos para cada
  numero entre [L, R], asi que la dimension extra es la
      suma de digitos
  de un numero, como el numero mayor es 10^18, se sabe
      que la suma de
  digitos de algun numero sera 19 * 9 = 171, asi que se
      asigna una
  talla donde entre ese valor.
*/

const int digits = 20; // Cantidad de digitos del numero
    mas grande que nos pueden dar
ll dp[digits][2][180];
vi num; // Vector que representa el numero que se esta
    analizando
int len; // Tamano del numero que se esta analizando

// Se convierte el numero en vector para hacer
    facilmente comparaciones entre digitos
void intToVec(ll x){
    // Se limpia lo que se tuviera antes
    num.clear();
    len = 0;

    for0(i,digits){
        num.pb(x % 10);
        len++;
        x /= 10;
    }

    reverse(num.begin(), num.end());
}

// Se limpia el dp para calcular el resultado para el
    siguiente numero
void reset(){
    for0(i,digits){
        for0(j,2){
            for0(k,180){
                dp[i][j][k] = -1;
            }
        }
    }
}

ll calc(int idx, bool under, int sum){
    if(idx == len) return sum; // Si se llega a la
        longitud maxima, se retorna la suma

    if(dp[idx][under][sum] != -1) return dp[idx][under][
        sum]; // Ya fue calculado

    // Si ya nos aseguramos que estamos viendo un numero
        menor a R, el siguiente digito
    // agregado puede ser cualquiera, es decir, puede ir
        de 0 a 9; de lo contrario, debemos
    // asegurarnos que el siguiente digito agregado sea
        menor o igual al digito de R, pues
    // de lo contrario ya estariamos probando un numero
        mayor.
    int lim = under ? 9 : num[idx];

    ll ans = 0; // Se inicializa la respuesta en 0

    for0(i,lim+1){
        // Si ya sabemos que estamos analizando numeros
            menores, o el digito que vamos a agregar es
        // estrictamente menor al de R, haremos el
            siguiente llamado sabiendo que estamos
            analizando
        // numeros menores
        bool nUnder = under || i < num[idx];

        // Se llama con el siguiente indice de digito,
            aclarando si ya nos aseguramos que los
            numeros
        // a analizar son menores a R, y a la suma se le
            agrega el digito que se va a poner
        ans += calc(idx + 1, nUnder, sum + i);
    }

    dp[idx][under][sum] = ans;

    return ans;
}

ll func(ll x){
    if(x <= 0) return 0;

    intToVec(x);
    reset();
    return calc(0, 0, 0);
}

void solver(){
    ll l, r; cin>>l>>r;
    cout<<func(r) - func(l-1)<<endl;
}
```

6.6 LIS

```
// Complejidad: O(n * log(n)).
// Se recibe un array 1-index.
/*
- LIS:
  es un array a donde para cada i, j con i < j, a[i] < a
      [j].
- Reconstruccion:
  Se puede usar un enfoque con indices en el cual cada
      que se ubica
  un elemento en el array del LIS se guarda el indice
      del elemento
  anterior y su propio indice. De esta manera cuando se
      termina el
  procesamiento, se toma el indice del ultimo valor del
      LIS y se va
  viajando hacia el indice que se puso como anterior a
      este, hasta
  llegar al primer elemento.
- Elementos que no estan en ningun LIS:
  Si mi array original es a, contruyo otro array b donde
  b[n-i-1] = -a[i] (0-index). Entonces hago el LIS para
      ambos arrays
  y calculo la posicion en la que ubico cada elemento en
       ambos.
  Si pos_a[i] + pos_b[i] - 1 != ValLIS, a[i] no puede
      estar en el LIS.
- Elementos que estan en todos los LIS:
  Tomo los elementos que si pueden estar en el LIS y
      reviso en que
  posicion se puso cada uno, si a[i] es el unico
      elemento ubicado en la
  posicion pos, quiere decir que esta en todos los LIS.
- Longest Non-Decreasing Subsequence:
  el tamano del mayor array a donde para cada i < j, a[i
      ] <= a[j],
  solo hay que revisar la inecuacion de la busqueda
      binaria.
- Longest Decreasing Subsequence (LDS):
  es quivalente a el LIS de un array invertido.
- Menor numero de Increasing Subsequences:
  es equivalente al tamano de la Longest Non-Increasing
      Subsequence.
  Es decir, el mayor tamano de array a, donde para cada
  i < j, a[i] >= a[j].
- Mayor subsecuencia de numeros consecutivos:
  se puede usar un enfoque con sets o mapas, en el que
      se guarde
  cada valor en donde termina una subsecuencia
      consecutiva, y el
  tamano de esta. Entonces cada que queramos agregar un
      valor x,
  buscamos si existe una subsecuencia que termine en el
      x - 1,
  con longitud l, en caso de que si la eliminamos y
      agregamos una
  nueva subsecuencia que termina en x con longitud l +
```

```
    1. Finalmente,
  recorremos todas las subsecuencias guardadas en el set
       o map y nos
  quedamos con la que tenga mayor longitud. Para
       reconstruir los
  indices podemos encontrar la ultima ubicacion del
       numero mayor x de
  subsecuencia mas larga y recorrer hacia la izquierda
       buscando x-1,
  despues x-2, hasta llegar a x - l + 1.
*/
/*
Con Segmet Tree (compresion):
- Para hallar el LIS usando Segment Tree, decimos que
  t[a[i]] = max(t[0], t[1], ..., t[a[i] - 1]) + 1
  Esta definicion quiere decir que el LIS que puedo
       obtener finalizando en el
  valor a[i] es igual a 1 mas el mayor de los LIS para j
       < i && a[j] < a[i]
  Para hallar el valor mayor usamos Segment Tree.
- Cantidad total de Increasing Subsequence's:
  tenemos que modificar este enfoque haciendo que el
       Segment Tree no nos de el
  maximo en un rango si no la suma en un rango. Asi cada
       elemento t[i]
  representa la cantidad de LIS que terminan en a[i].
*/

void LIS(){
    int n; cin>>n;
    vi a(n+1);
    for1(i,n) cin>>a[i];

    vi d(n+1, INF); d[0] = -INF;

    for1(i,n){
        auto it = ub(all(d), a[i]);
        if(it == d.end()) continue;

        int index = distance(d.begin(), it);
        // Este if se encarga de no poner numeros
            iguales
        // Si me piden que cada numero sea estrictamente
            mayor
        // Si piden mayor o igual se puede quitar
        if(a[i] > d[index - 1]) d[index] = a[i];
    }

    int ans = 0;
    for1(i,n) if(d[i] < INF) ans = i;

    cout<<ans<<endl;
}
```

6.7 Recorrer submasks

```
/*
- Complejidad O(3^n)
- Si contamos el numero de pares ordenados (T, S), donde
     T esta contenido
  en S, tenemos que cada elemento x tiene 3 opciones:
       esta en T y en S,
  esta solo en S, no esta en ninguno. Pues si x
       estuviera en T y no en S,
  entonces T no estaria contenido en S. Por lo tanto,
       cada elemento puede
  estar en 3 posibles estados.
- Cuando se le resta uno a la submascara, el bit en 1
     mas a la derecha
  pasa a ser 0 y todos los bits a su derecha se
       convierten en 1. Luego se
  aplica AND para eliminar todos los bits que no estan
       en la mascara.
  Finalmente se usa la XOR para recorrer los
       subconjuntos en orden
    incremental
- Source: https://usaco.guide/gold/dp-bitmasks
*/
for0(mask,(1 << n)){
  for(int submask = mask; submask != 0; submask = (
       submask - 1) & mask){
    int subset = mask ^ submask;
    // do whatever you need to do here
    // submask contiene todo lo que esta en mask pero no
         en subset
  }
}
```

7 Math

7.1 Combinatory

Coeficientes binomiales

Notación: $\binom{n}{k}$

- Se asume que si $k > n$, entonces $\binom{n}{k} = 0$.

$$\binom{n}{k} = \frac{n!}{(n-k)!\,k!}$$

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

$$\binom{n+1}{k} = \frac{n+1}{n-k+1}\binom{n}{k}$$

$$\binom{n}{k+1} = \frac{n-k}{k+1}\binom{n}{k}$$

$$\binom{n}{k} = \frac{n}{n-k}\binom{n-1}{k}$$

$$\binom{n}{k} = \frac{n-k+1}{k}\binom{n}{k-1}$$

- **Suma de fila completa:**

$$\sum_{k=0}^{n}\binom{n}{k} = 2^n$$

- **Identidad de la suma superior (hockey stick):**

$$\sum_{m=0}^{n}\binom{m}{k} = \binom{n+1}{k+1}$$

- **Variante de la identidad de la suma superior:**

$$\sum_{k=0}^{m}\binom{n+k}{k} = \binom{n+m+1}{m}$$

- **Identidad de Vandermonde:**

$$\sum_{k=0}^{r}\binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$$

- **Suma de cuadrados:**

$$\sum_{k=0}^{n}\binom{n}{k}^2 = \binom{2n}{n}$$

- **Suma ponderada:**

$$\sum_{k=1}^{n} k\binom{n}{k} = n\cdot 2^{n-1}$$

- **Conexión con Fibonacci:**

$$\sum_{k=0}^{n}\binom{n-k}{k} = F_{n+1}$$

El número de caminos de $(0,0)$ hacia (a,b), donde cada paso se da al norte o al este: $\binom{a+b}{a}$.

El número de formas de pintar n objetos con r colores si cada color debe ser utilizado al menos una vez: $\sum_{k=0}^{r}\binom{r}{k}(-1)^{r-k}k^n$ o $\sum_{k=0}^{r}\binom{r}{r-k}(-1)^k(r-k)^n$

El número de tuplas de k números $x_1 + x_2 + ... + x_k$ cuya suma es igual a n (Tener en cuenta que una tupla $\{1,2\}$ y otra $\{2,1\}$ se consideran diferentes):

- Si los números deben ser positivos: $\binom{n-1}{k-1}$.
- Si los números deben ser no negativos: $\binom{n+k-1}{n}$
- Si dan lower bounds $x_i \geq a_i$, se halla $n' = n - \sum_{i=1}^{k} a_i$ y se calcula: $\binom{n'+k-1}{n'}$
- Si dan un upper bound $x_i \leq a$: $\sum_{i=0}^{k}(-1)^i\binom{k}{i}\binom{n+k-1-(a+1)i}{k-1}$ (Tener en cuenta que un coeficiente binomial es igual a 0 si su parte superior es negativa)
- Si me dan upper_bounds $x_i \leq a_i$, se halla el resultado sin restricciones y con inclusión-exclusión se restan los casos donde $x_i \geq a_i$

El número de tuplas de k números $x_1 + x_2 + ... + x_k$ cuya suma es menor o igual a n: $\binom{n+k}{k}$

El número de formas de poner n pelotas en k cajas (similar a decir que tengo que elegir n elementos de un conjunto de tamaño k):

- Si cada caja puede tener maximo una pelota: $\binom{k}{n}$
- Si cada caja puede contener multiples pelotas: $\binom{k+n-1}{n}$
- Si cada caja puede contener maximo una pelota y dos cajas adyacentes no pueden tener una pelota: $\binom{k-n+1}{k-2n+1}$

Coeficientes multinomiales

El coeficiente multinomial

$$\binom{n}{n_1, n_2, ..., n_k} = M(n_1, n_2, ..., n_k) = \frac{n!}{n_1!n_2!...n_k!}$$

es igual a la cantidad de formas en que se pueden dividir n elementos en subconjuntos de tamaño $n_1, n_2, ..., n_k$, donde $n_1 + n_2 + \cdots + n_k = n$.

$$M(a, ..., b, c, ...) = M(a + ... + b, c, ...)M(a, ..., b)$$

$$(a_1 + ... + a_k)^n = \sum_{n_1+...+n_k=n,\ n_i \geq 0} \binom{n}{n_1, ..., n_k} a_1^{n_1}...a_k^{n_k}$$

La cantidad de permutaciones que se pueden obtener a partir de un multiset de n elementos con valores de 1 a k, esta dada por: $\binom{n}{freq_1, freq_2, ..., freq_k}$

Números Catalanes

El número catalán C_n se define como la cantidad de expresiones de paréntesis válidas que consisten de n paréntesis de apertura y n paréntesis de cierre.

Por ejemplo, $C_3 = 5$: `()()()`, `()(())`, `(())()`, `(()())`, `((()))`.

Partiendo desde C_0, los primeros números catalanes son: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670

$$C_n = \sum_{i=0}^{n-1} C_i \cdot C_{n-i-1}$$

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

- Donde $\binom{2n}{n}$ es la cantidad de formas de construir una expresión no necesariamente válida de n paréntesis de apertura y n paréntesis de cierre, mientras que $\binom{2n}{n-1}$ es la cantidad de expresiones inválidas, así que haciendo la resta y despejando, se obtiene la fórmula escrita.

Interpretaciones de los números catalanes

El número de Catalan C_n es la solución para:

- El número de secuencias de paréntesis correctas que consisten de n paréntesis de apertura y n de cierre.
- El número de árboles binarios completos con raíz que tienen $n+1$ hojas (los vértices no están numerados). Un árbol binario con raíz es *completo* si cada vértice tiene dos hijos o ninguno.
- El número de árboles binarios completos no isomorfos con n nodos internos (es decir, nodos con al menos un hijo).
- El número de árboles binarios de n nodos (tener en cuenta que un árbol root -> left -> left es diferente que un árbol root -> right -> right).
- El número de árboles con raíz de $n+1$ nodos (los vértices no están enumerados), donde si en un árbol la primera rama tiene 1 elemento y la segunda rama tiene 2, mientras que en otro árbol la primera rama tiene 2 y la segunda tiene 1, estos árboles se consideran diferentes; es decir, si se cambia el orden de las ramas y estas tienen estructuras diferentes, se considera un árbol distinto.
- El número de formas de parentesizar completamente $n+1$ factores.
- El número de triangulaciones de un polígono convexo de $n+2$ lados (es decir, el número de particiones del polígono en triángulos disjuntos usando diagonales).
- El número de formas de conectar $2n$ puntos en un círculo para formar n cuerdas que no se crucen entre sí.
- El número de caminos monótonos (que cada paso se da al norte o al este) en una red de $(0, 0)$ a (n, n) en una malla cuadrada de tamaño $n \times n$, que no cruzan la diagonal principal (la que conecta $(0, 0)$ con (n, n)). Tener en cuenta que no se está pasando por los cuadros de la malla sino por los puntos.
- El número de permutaciones de longitud n que pueden ordenarse con una pila (*stack sorted*); es decir, la permutación es stack sorted si y solo si no existe un índice $i < j < k$ tal que $a_k < a_i < a_j$.
- El número de particiones no cruzadas de un conjunto de n elementos. Consiste en imaginar un círculo que en los bordes tiene los elementos de 1 a n en orden, y se quiere separarlos en subconjuntos de modo que al dibujar un arco con los elementos de cada subconjunto, no se haga ningún cruce.
- El número de formas de cubrir la escalera $1, 2, 3, \ldots, n$ usando n rectángulos.

Derangements

La cantidad de derangements de n números, expresada como D_n, es la cantidad de permutaciones tal que ningún elemento aparece en su posición original, es decir, donde para todo i, $1 \leq i \leq n$, $p_i \neq i$.

Partiendo desde D_0, los primeros D_i son: 1, 0, 1, 2, 9, 44, 265, 1854, 14833

- **Calculando con inclusión-exclusión:** $D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}$
- **Calculando con DP:** $D_0 = 1$, $D_1 = 0$, $D_i = (i-1)(D_{i-1} + D_{i-2})$

La cantidad de permutaciones con exactamente k indices donde $p_i = i$ es: $\binom{n}{k} D_{n-k}$

Sumatorias

$$\sum_{k=0}^{n} k = \frac{n(n+1)}{2}$$

$$\sum_{k=a}^{b} k = \frac{(a+b)(b-a+1)}{2}$$

$$\sum_{k=0}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{k=0}^{n} k^3 = \frac{n^2(n+1)^2}{4}$$

$$\sum_{k=0}^{n} k^4 = \frac{6n^5 + 15n^4 + 10n^3 - n}{30}$$

$$\sum_{k=0}^{n} k^5 = \frac{2n^6 + 6n^5 + 5n^4 - n^2}{12}$$

$$\sum_{k=0}^{n} x^k = \frac{x^{n+1} - 1}{x - 1} \quad \text{(válida solo para } x \neq 1\text{)}$$

$$\sum_{k=0}^{n} kx^k = \frac{x - (n+1)x^{n+1} + nx^{n+2}}{(x-1)^2}$$

$$1 + x + x^2 + \cdots = \frac{1}{1-x} \quad \text{(válida solo para } |x| < 1\text{)}$$

Lema de Burnside

Cuenta la cantidad de representaciones (o clases) de un objeto que posee n elementos de los cuales cada uno puede ser pintado con k colores diferentes. Además, dicho objeto puede tener ciertos movimientos, como rotaciones o reflexiones, que hacen que algunas combinaciones de colores diferentes realmente sean iguales (y que por lo tanto, se deben contar solo una vez). Con cada combinación de movimientos (incluyendo el no hacer ningún movimiento) se generan permutaciones g que componen el conjunto G, se dice que g_i es igual a la posición que ocupa el elemento i después de hacer esa combinación de movimientos.

$$|Classes| = \frac{1}{|G|} \sum_{g \in G} k^{c(g)}$$

donde $c(g)$ es la cantidad de ciclos que hay en la permutación g.

Ejemplo: collar de 4 cuentas, 3 colores posibles, al collar se le pueden hacer rotaciones

$G = \{e, r, r^2, r^3\}$ Donde e representa la permutación que se obtiene al no hacer ninguna rotación, r es al rotar una posición y r^x es al rotar x posiciones. Solo nos interesa rotar hasta 3 posiciones ya que rotar 4, 5, 6, ... sería equivalente a rotar 0, 1, 2, ..., posiciones.

En este caso, la cantidad de ciclos que tiene cada permutación se obtiene con el $gcd(x, 4)$

$$c(e) = \gcd(0,4) = 4, \quad c(r) = \gcd(1,4) = 1, \quad c(r^2) = \gcd(2,4) = 2, \quad c(r^3) = \gcd(3,4) = 1$$

$$|Classes| = \frac{1}{4}\left(3^4 + 3^1 + 3^2 + 3^1\right) = \frac{1}{4}(81 + 3 + 9 + 3) = \frac{96}{4} = 24$$

Ejemplo: grid de $n \times n$, colores blanco y negro, con rotaciones

$G = \{e, r, r^2, r^3\}$ donde: e: no rotar (rotación de 0°), r: rotar 90°, r^2: rotar 180° y r^3: rotar 270°

$$|Classes| = \frac{1}{|G|} \sum_{g \in G} k^{c(g)} = \frac{1}{4}\left(2^{c(e)} + 2^{c(r)} + 2^{c(r^2)} + 2^{c(r^3)}\right)$$

Con un poco de observación y matemática se puede hallar la cantidad de ciclos que hay en la permutación que forma cada rotación.

Fórmula de Cayley

La cantidad de árboles etiquetados (que cada vertice tiene su respectivo número) distintos que se pueden formar con n vértices

$$T(n) = n^{n-2}$$

Código de Prüfer

El código de Prüfer es una forma de codificar un árbol etiquetado de n vértices como una secuencia de $n-2$ números. Cada árbol tiene una codificación única.

Cómo hallar el código a partir del árbol

Se repite $n-2$ veces:

1. Buscar la **hoja con la menor etiqueta** entre las hojas actuales del árbol.
2. Anotar en la secuencia el único nodo al que está conectada esa hoja.
3. Eliminar esa hoja del árbol (y su arista).

Al terminar las $n-2$ repeticiones, quedan exactamente 2 vértices, que ya no se anotan.

Cómo reconstruir el árbol a partir del código

1. Calcular el **grado** de cada vértice v: veces que v aparece en la secuencia.
2. Se repite $n-2$ veces:
   - Buscar el vértice de **menor etiqueta** que tenga grado 0 (es decir, una hoja disponible) y que aún no haya sido usado.
   - Conectar con el **primer elemento restante** de la secuencia de Prüfer.
   - Se disminuye en 1 el grado de ambos (la hoja usada y el vértice de la secuencia), y se elimina ese primer elemento de la secuencia.
3. Al final quedan exactamente 2 vértices con grado 0 los cuales se conectan entre sí con la última arista.

Número de grafos etiquetados

Dado un grafo con n vertices, el número de grafos etiquetados, con aristas no dirigidas, sin aristas loop ni aristas multiples

$$G_n = 2^{\frac{n(n-1)}{2}}$$

Número de grafos etiquetados conexos

$$C_n = G_n - \sum_{k=1}^{n-1} \binom{n-1}{k-1} C_k G_{n-k}$$

Número de grafos etiquetados con k componentes

Se calcula $D[i][j]$ - el número de grafos etiquetados con i vertices y j componentes, para $0 \le i \le n$ y $0 \le j \le k$. Con $D[0][0] = 1$.

$$D[n][k] = \sum_{s=1}^{n} \binom{n-1}{s-1} C_s D[n-s][k-1]$$

7.2 Coeficientes binomiales

```
////////////////////////////////////////////////
// Complejidad O(n ^ 2).
// binom[i][j] = (i!) / (j! * (i - j)!)
// Usar si N es pequeno y, el modulo no es primo (no se
    puede dividir)
// o no hay modulo y se debe hacer con sumas para no
    tener overflow.
const int N = 501;
const ll mod = 1e9 + 7;
ll binom[N][N];

void calcBinom(){
    for0(i,N) binom[i][0] = binom[i][i] = 1;
    for1(i,N-1){
        for1(j,i-1){
            binom[i][j] = (binom[i-1][j-1] + binom[i-1][
                j]) % mod;
        }
    }
}

////////////////////////////////////////////////
// Si el modulo es primo, se puede obtener en O(n + logk
    ).
// Primero se precalcula el factorial hasta maxn
ll calculateBinomial(int n, int k){
    if(k < 0 || k > n) return 0;
    ll numerator = fact[n];
    ll denominador = (fact[k]*fact[n-k]) % mod;

    // Apply when mod is prime, if mod is not necesary,
        just divide those numbers normaly
    return (numerator * binpow(denominador, mod-2)) %
        mod; // Binary exponancion for eficiencia
}

////////////////////////////////////////////////
/*
Coeficiente binomial modulo mod = p ^ b, siendo p un
    numero primo:
- Si p > max(k, n-k) se puede usar el metodo de siempre.
- De lo contrario, k! y/o (n-k)! no son coprimos con m
    asi que no se puede usar el inverso modular.
- Para cada x! se computa el mayor exponente c(x) tal
    que p ^ c(x) es divisor de x!, y se halla
  g(x) = x! / (p ^ c(x)). g(x) es x! sin factores de p,
      modulo mod.
- Ahora (n) = (g(n) / (g(k)*g(n-k))) * (p ^ (c(n) - c(k)
    - c(n-k)))
       (k)
  Con esto se asegura que g(x) es coprimo con mod, por
      lo tanto se puede usar el inverso modular.
- Si c(n) - c(k) - c(n-k) >= b, entonces el coeficiente
    binomial es 0, ya que es multiplo de m.
- La complejidad es O(maxn) del precalculo de g y c, mas
    O(log(n)) por cada query.
*/
void precompute(int n, ll p, ll b, ll mod, vl& g, vl& c)
    {
    g[0] = 1 % mod;
    for1(x,n){
        ll cur = x, cnt = 0;
        while(cur % p == 0){
            cur /= p;
            cnt++;
        }
        c[x] = c[x - 1] + cnt;
        g[x] = g[x - 1] * (cur % mod) % mod;
    }
}

ll binomial(int n, int k, ll p, ll b, ll mod, ll phimod,
    vl& g, vl& c){
    if(k < 0 || k > n) return 0;
    ll exp_p = c[n] - c[k] - c[n - k];
    if(exp_p >= b) return 0;
    ll res = g[n] % mod;
    res = (res * binpow(g[k], phimod - 1, mod)) % mod;
    res = (res * binpow(g[n-k], phimod - 1, mod)) % mod;
    res = res * binpow(p, exp_p, mod) % mod;
    return res;
}

void solver(){
    // Obtener p, b, mod y phi(mod)
    ll p, b; cin>>p>>b;
    ll mod = 1;
    for0(i,b) mod *= p;
    ll phimod = mod / p * (p-1);

    // Precalcular g y c
    int maxn = 1000; // limite superior de n
    vl g(maxn+1, 1), c(maxn+1);
    precompute(maxn, p, b, mod, g, c);

    // Hallar el coeficiente binomial
    int n, k; cin>>n>>k;
    cout<<binomial(n, k, p, b, mod, phimod, g, c);
}

////////////////////////////////////////////////
/*
Coeficiente binomial con un modulo mod arbitrario:
- Se halla la factorizacion prima: mod = p_1 ^ b_1 * p_2
    ^ b_2 * ... * p_h ^ b_h.
- Se dice que mod_i = p_i ^ b_i.
- Para cada mod_i, se precalculan las funciones g_i y
    c_i igual que en el metodo anterior.
- Para hallar el coeficiente binomial (n k), por cada
    mod_i se halla
  r_i = binomial(n, k, p_i, b_i, mod_i, phi(mod_i), g_i,
      c_i).
- Se usa el Chinese Remainder Theorem para hallar ans =
    (n k) % mod, en el conjunto de ecuaciones:
  ans % mod_1 = r_1 % mod_1
  ans % mod_2 = r_2 % mod_2
  ...
  ans % mod_h = r_h % mod_h
*/
////////////////////////////////////////////////
/*
Coeficiente binomial cuando n es muy grande y mod no
    tanto (mod tiene que ser primo):
- Complejidad: O(mod) del precalculo del factorial, mas
    O(log_mod(n)) por query.
- Se halla el factorial de cada numero hasta mod-1.
- Se tiene que:
  (n) % mod = (n_0) * (n_1) * ... * (n_e) % mod
  (k)          (k_0)   (k_1)        (k_e)
  Donde n = (n_e * mod ^ e) + (n_(e-1) * mod ^ (e-1)) +
      ... + (n_1 * mod) + n_0.
  y     k = (k_e * mod ^ e) + (k_(e-1) * mod ^ (e-1)) +
      ... + (k_1 * mod) + k_0.
- Si para cualquier i, k_i > n_i, el resultado del
    coeficiente binomial es 0.
*/
ll lucasBinomial(ll n, ll k){ // Coeficientes binomiales
    ll ans = 1;
    while(n > 0 || k > 0){
        ll ni = n % mod;
        ll ki = k % mod;
        if(ki > ni) return 0;
        ans *= (fact[ni] * mod_inverse((fact[ki] * fact[
            ni - ki]) % mod)) % mod;
        ans %= mod;
        n /= mod;
        k /= mod;
```

```
    }
    return ans;
}

////////////////////////////////////////////
/*
Coeficiente binomial cuando n es muy grande y mod no
    tanto (mod tiene que ser square free, es decir,
en su descomposicion prima: mod = p_1 ^ e_1 * p_2 ^ e_2
    * ... * p_h ^ e_h, cada e_i es igual a 1):
- Complejidad: O(m) del precalculo de factoriales, mas O
    (h*log(n)) para cada query.
- Para cada p_i se debe precalcular el factorial hasta
    p_i - 1.
- r_i = lucasBinomial(n, k) % p_i.
- Se usa el Chinese Remainder Theorem para hallar ans en
    las ecuaciones: ans % p_i = r_i % p_i.
- Tener cuidado con los llamados a las funciones, usar
    el mod adecuado.
*/
```

7.3 Power of factorial divisor

```
/*
- Dado n! y una base k, se quiere hallar el mayor x tal
    que k ^ x sea divisor de n!
- Se hallan todos los divisores primos de k y se llama a
    la funcion findExp. x es
  la menor respuesta entre todos los llamados.
- La complejidad de findExp es O(log_k(n)), asi que la
    complejidad total es eso por
  la cantidad de divisores primos de k, mas el tiempo
      que se tarde en hallar dichos
  divisores.
*/
int findExp(int n, int k){
    int ans = 0;
    while(n){
        n /= k;
        ans += n;
    }
    return ans;
}
```

7.4 Factorial MOD

```
////////////////////////////////////////////
// Precalculo de Factorial e Inverso de Factorial modulo
    MOD
//
// Complejidad temporal:
//   - Precalculo: O(N + log(MOD)) gracias a la
    propagacion hacia atras:
//     invFact[i] = (invFact[i + 1] * (i + 1)) % MOD
//   - Consultas (nCr, nPr): O(1)
// Complejidad espacial: O(N)
//
// NOTA: MOD debe ser un numero primo para usar el
    Pequeno Teorema de Fermat:
//       a^(MOD - 2) == a^(-1)  (mod MOD)
////////////////////////////////////////////////

const int MAXN = 1e6 + 5;
const ll MOD = 1e9 + 7;

ll fact[MAXN];
ll invFact[MAXN];

// Exponenciacion binaria: O(log(e))
ll binpow(ll b, ll e, ll m = MOD) {
    b %= m;
    ll ans = 1;
    while (e > 0) {
        if (e & 1) ans = (ans * b) % m;
        b = (b * b) % m;
        e >>= 1;
    }
    return ans;
}

// Inverso modular usando el Pequeno Teorema de Fermat (
    MOD primo)
ll mod_inv(ll a, ll m = MOD) {
    return binpow(a, m - 2, m);
}

// Precalculo de factoriales e inversos de factoriales
    hasta n
void precomputeFactorials(int n, ll mod = MOD) {
    fact[0] = 1;
    for1(i, n) {
        fact[i] = (fact[i - 1] * i) % mod;
    }

    // Se calcula el inverso del factorial de n en O(log
        MOD)
    invFact[n] = mod_inv(fact[n], mod);

    // Se calculan los inversos de factorial hacia atras
        en O(1) cada uno
    // (i!)^(-1) = ((i + 1)!)^(-1) * (i + 1) % MOD
    forn0(i, n) {
        invFact[i] = (invFact[i + 1] * (i + 1)) % mod;
    }
}

// Inverso modular de un numero x (1 <= x <= n) en O(1)
ll get_inv(int x, ll mod = MOD) {
    if (x <= 0) return 0;
    return (invFact[x] * fact[x - 1]) % mod;
}
```

7.5 Matriz

```
/*
- Algunas funciones para operaciones con matrices
    perfectas (n x n).
APLICACIONES:
- Se puede hallar el numero de caminos de una longitud
    especifica
  'k' elevando su matriz de adyacencia a la potencia 'k
      '.
  La posicion i,j en dicha matriz dice la cantidad de
      caminos de
  longitud 'k' que inician en el vertice 'i' y terminan
      en el
  vertice 'j'.
  * Sirve tanto para grafos dirigidos como no dirigidos.
  * Soporta loops.
  * Soporta aristas multiples si en la matriz de
      adyacencia se
    guarda cuantas aristas hay desde 'i' hasta 'j'.
  * Si se quieren hallar la cantidad de caminos de
      longitud maximo
    'k' que inician en 'i', se crea un vertice extra 's'
        que tiene
    una arista hacia si mismo y hacia el cual todos los
        demas
    vertices tienen una arista. Se hace exponenciacion
        con potencia
    'k+1'. La cantidad de caminos que inician en 'i' y
        tienen maximo
    longitud 'k' queda en la posicion i,s despues de la
    exponenciacion. (Quedan incluidos los caminos de
        longitud '0').
*/

struct Matrix {
    int n;
    vector<vl> M;

    Matrix(int n) : n(n), M(n, vl(n, 0)) {}

    // La matrix identidad, donde todo es '0' excepto en
        la diagonal principal, que es '1'
    static Matrix identity(int n){
        Matrix I(n);
        for0(i,n) I.M[i][i] = 1;
        return I;
    }

    // Multiplicacion de matrices O(n^3)
    Matrix operator*(const Matrix& other) const {
        Matrix result(n);
        for0(i,n){
            for0(k,n){
                if(!M[i][k]) continue;
                for0(j,n) result.M[i][j] = (result.M[i][
                    j] + (M[i][k] * other.M[k][j] % MOD
                    )) % MOD;
            }
        }
        return result;
    }

    // Suma de matrices (n^2)
    Matrix operator+(const Matrix& other) const {
        Matrix result(n);
        for0(i,n) for0(j,n) result.M[i][j] = (M[i][j] +
            other.M[i][j]) % MOD;
        return result;
    }

    // Exponenciacion de una matrix, usando
        exponenciacion binaria en O(log(exp) * n^3)
    Matrix binpow(ll exp) const {
        Matrix result = identity(n);
        Matrix base = *this;

        while(exp){
            if(exp & 1) result = result * base;
            base = base * base;
            exp >>= 1;
        }
        return result;
    }

    // Transpuesta de la matriz: T[j][i] = M[i][j], O(n
        ^2)
    Matrix transpose() const {
        Matrix T(n);
        for0(i,n) for0(j,n) T.M[j][i] = M[i][j];
        return T;
    }

    // Debug, muestra la matrix en el cout
    friend ostream &operator<<(ostream &os, const Matrix
        &matrix){
        os << "\nMatrix: \n";
        for0(i,matrix.n){
            for0(j,matrix.n){
                os << matrix.M[i][j] << " ";
            }
            os << endl;
        }
        os << " =============== \n";
        return os;
    }
};

void solver(){
    // Lectura
    int n; cin>>n;
    Matrix m(n);
    for0(i,n){
```

```
        for0(j,n){
            cin>>m.M[i][j];
        }
    }

    ll exp; cin>>exp;
    Matrix ans = m.binpow(exp); // Para llamar una
        funcion
    cout<<ans; // Imprime la matriz
}
```

7.6 Probability

Probabilidad básica

Notación: $P(A)$ es la probabilidad del evento A, Ω es el espacio muestral.

- Se asume que $0 \le P(A) \le 1$, $P(\Omega) = 1$ y $P(\emptyset) = 0$.
- **Casos equiprobables (caso discreto):**

$$P(A) = \frac{\text{casos favorables}}{\text{casos totales}}$$

- **Probabilidad de un intervalo bajo una densidad (caso continuo):**

Si X tiene densidad $f(x)$, la probabilidad de que caiga en el intervalo $[a, b]$ es el área bajo la curva f en ese intervalo:

$$P(a \le X \le b) = \int_a^b f(x)\,dx$$

Esto funciona porque el área total bajo f es 1, así que cada trozo representa una fracción de la probabilidad total.

Ejemplo: $f(x) = 2x$ en $(0, 1)$ y queremos $P(X < \frac{1}{2})$:

$$P\left(X < \frac{1}{2}\right) = \int_0^{1/2} 2x\,dx = 2 \cdot \frac{x^2}{2}\Big|_0^{1/2} = \left(\frac{1}{2}\right)^2 = \frac{1}{4}$$

- **Normalizar la densidad:** si dan la forma de la función $h(x)$ pero no la constante de normalización c, se plantea que la función de densidad es $f(x) = c\,h(x)$. Para hallar c se integra $f(x)$ sobre el intervalo $[a, b]$ donde está definida y se despeja c para que el área sea 1:

$$\int_a^b c\,h(x)\,dx = 1 \quad \Longrightarrow \quad c = \frac{1}{\int_a^b h(x)\,dx}$$

  Ejemplo: $h(x) = 1$, entonces $f(x) = c$ en $[a, b]$ (densidad constante, la uniforme).

$$\int_a^b c\,dx = c\,(b-a) = 1 \quad \Longrightarrow \quad c = \frac{1}{b-a} \quad \Longrightarrow \quad f(x) = \frac{1}{b-a}$$

- **Complemento:**

$$P(A^c) = 1 - P(A)$$

- **Unión de dos eventos:**

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

- **Inclusión-exclusión:**

$$P\left(\bigcup_{i=1}^{n} A_i\right) = \sum_{k=1}^{n} (-1)^{k+1} \sum_{i_1 < \cdots < i_k} P(A_{i_1} \cap \cdots \cap A_{i_k})$$

- **Cota de la unión (Boole):**

$$P\left(\bigcup_{i=1}^{n} A_i\right) \le \sum_{i=1}^{n} P(A_i)$$

La probabilidad de que ocurra al menos un éxito en n intentos independientes, cada uno con probabilidad p: $1 - (1-p)^n$ (se calcula como 1 menos la probabilidad de que no ocurra ninguno).

Probabilidad condicional

Notación: $P(A \mid B)$ es la probabilidad de A sabiendo que ocurrió B.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \quad \text{(válida solo para } P(B) > 0)$$

- **Regla del producto:**

$$P(A \cap B) = P(A \mid B)\,P(B) = P(B \mid A)\,P(A)$$

- **Regla de la cadena:**

$$P(A_1 \cap A_2 \cap \cdots \cap A_n) = P(A_1)\,P(A_2 \mid A_1)\,P(A_3 \mid A_1 \cap A_2) \cdots P(A_n \mid A_1 \cap \cdots \cap A_{n-1})$$

- **Probabilidad total** (si $B_1, \ldots, B_k$ es una partición de Ω):

$$P(A) = \sum_{i=1}^{k} P(A \mid B_i)\,P(B_i)$$

Que $B_1, \ldots, B_k$ sea una partición de Ω significa que:

- Son disjuntos: $B_i \cap B_j = \emptyset$ si $i \neq j$.
- Cubren todo el espacio muestral: $B_1 \cup B_2 \cup \cdots \cup B_k = \Omega$.

Es decir, en cualquier resultado del experimento ocurre exactamente uno de los B_i. Un caso común es B y B^c, que siempre forman una partición. Se exige para que A se pueda separar en pedazos sin contar dos veces el mismo caso y sin dejar casos por fuera: $P(A) = \sum_{i=1}^{k} P(A \cap B_i)$.

- **Teorema de Bayes:**

$$P(B_j \mid A) = \frac{P(A \mid B_j)\,P(B_j)}{\sum_{i=1}^{k} P(A \mid B_i)\,P(B_i)}$$

- **Bayes con momios (odds):**

$$\frac{P(H \mid E)}{P(H^c \mid E)} = \frac{P(H)}{P(H^c)} \cdot \frac{P(E \mid H)}{P(E \mid H^c)}$$

- **Complemento condicional** ($P(\cdot \mid B)$ es en sí misma una probabilidad):

$$P(A^c \mid B) = 1 - P(A \mid B)$$

Independencia

- A y B son independientes si y solo si:

$$P(A \cap B) = P(A)\,P(B) \iff P(A \mid B) = P(A)$$

- Los eventos $A_1, \ldots, A_n$ son mutuamente independientes si para todo subconjunto S de índices:

$$P\left(\bigcap_{i \in S} A_i\right) = \prod_{i \in S} P(A_i)$$

- **Independencia condicional:**

$$P(A \cap B \mid C) = P(A \mid C)\,P(B \mid C)$$

- **Al menos un evento ocurre (independientes):**

$$1 - \prod_{i=1}^{n} (1 - P(A_i))$$

Para tres eventos A, B, C, la independencia mutua exige que se cumplan los productos de cada par y también del trío:

$$P(A \cap B \cap C) = P(A)\,P(B)\,P(C)$$

Tener en cuenta que la independencia por pares no implica independencia mutua, y que dos eventos disjuntos con probabilidad positiva no son independientes porque el cumplimiento de uno implica el no cumplimiento del otro.

Valor esperado

Notación: $E[X]$ o μ es el valor esperado de la variable aleatoria X.

- **Caso discreto:**

$$E[X] = \sum_x x\,P(X = x)$$

- **Caso continuo** (con densidad f):

$$E[X] = \int_{-\infty}^{\infty} x\,f(x)\,dx$$

- **Desde la función de distribución:** si se conoce $F(x) = P(X \le x)$, la densidad es su derivada:

$$f(x) = F'(x)$$

  Para la uniforme, $F(x) = \frac{x-a}{b-a}$ y al derivar sale $\frac{1}{b-a}$.

- **Esperanza de una función de X (LOTUS):**

$$E[g(X)] = \sum_x g(x)\,P(X = x) \quad \text{o} \quad \int g(x)\,f(x)\,dx$$

- **Fórmula de la cola** (para X entera y no negativa):

$$E[X] = \sum_{k=1}^{\infty} P(X \geq k)$$

- **Fórmula de la cola, caso continuo** ($X \geq 0$):

$$E[X] = \int_0^{\infty} P(X > t)\, dt$$

Propiedades

Para constantes a, b, c:

$$E[c] = c \qquad E[X + c] = E[X] + c \qquad E[cX] = c\,E[X] \qquad E[aX + b] = a\,E[X] + b$$

- Si $X \leq Y$ siempre, entonces $E[X] \leq E[Y]$.
- Si X y Y son independientes: $E[XY] = E[X]\,E[Y]$ (Pero el hecho de que $E[XY] = E[X]\,E[Y]$ no asegura que X y Y sean independientes).
- $E[g(X)] \neq g(E[X])$ en general. Si g es convexa: $g(E[X]) \leq E[g(X)]$ (Jensen).

Desigualdades

- **Markov** ($X \geq 0$, $a > 0$):

$$P(X \geq a) \leq \frac{E[X]}{a}$$

- **Chebyshev:**

$$P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$$

Aquí μ es la media, σ la desviación estándar y $k > 0$ (solo aporta información si $k > 1$). Se lee así: la probabilidad de que X esté a k desviaciones estándar o más de la media es a lo sumo $\frac{1}{k^2}$.

Linealidad de la esperanza

Para variables aleatorias cualesquiera, sin importar si son independientes o no:

$$E[X + Y] = E[X] + E[Y]$$

$$E\left[\sum_{i=1}^{n} c_i X_i\right] = \sum_{i=1}^{n} c_i\, E[X_i]$$

Cuándo pensar en linealidad:

- Calcular $E[X]$ como promedio ponderado es engorroso porque las probabilidades de cada valor son difíciles de hallar.
- X se puede escribir como suma de variables más simples.

Receta: escribir $X = X_1 + X_2 + \cdots + X_n$, hallar cada $E[X_i]$ y sumarlas.

- **Identidad de Wald** (si N es aleatorio e independiente de los X_i, que están idénticamente distribuidos):

$$E\left[\sum_{i=1}^{N} X_i\right] = E[N]\,E[X_1]$$

Ejemplo: avance esperado de un caminante

Cada minuto avanza $+1$ con probabilidad $\frac{1}{2}$, se queda quieto con $\frac{1}{3}$ y retrocede -1 con $\frac{1}{6}$. En n minutos el avance neto esperado es $n\left(\frac{1}{2} - \frac{1}{6}\right)$, es decir, 20 para $n = 60$.

Variables indicadoras

$$\mathbb{1}_A = \begin{cases} 1 & \text{si } A \text{ ocurre} \\ 0 & \text{si no} \end{cases} \qquad \Longrightarrow \qquad E[\mathbb{1}_A] = P(A)$$

Esperanza condicional

- **Caso discreto:**

$$E[X \mid Y = y] = \sum_x x\, P(X = x \mid Y = y)$$

- **Condicionando a un evento A:**

$$E[X \mid A] = \frac{E[X\,\mathbb{1}_A]}{P(A)}$$

Aquí $E[X\,\mathbb{1}_A]$ suma solo los valores de X donde ocurre A. Dividir entre $P(A)$ reescala ese promedio.

- **Caso continuo** (condicionando a que X caiga en χ):

$$E[X \mid X \in \chi] = \frac{\int_\chi x\, f(x)\, dx}{P(X \in \chi)}$$

- **Ley de la esperanza total** (si $A_1, \ldots, A_k$ es una partición de Ω):

$$E[X] = \sum_{i=1}^{k} E[X \mid A_i]\, P(A_i)$$

- **Ley de la torre:**

$$E[X] = E\big[E[X \mid Y]\big]$$

Tener en cuenta que la ley de la torre se usa cuando **uno elige condicionar a** Y para simplificar el cálculo, pues en casos donde el cálculo de $E[X]$ directo es complicado, dividir el problema en subcasos puede ser más sencillo. Se debe realizar el cálculo para todo valor posible de Y.

La fórmula completa es:

$$E[X] = \sum_y E[X \mid Y = y]\, P(Y = y)$$

O en caso continuo:

$$E[X] = \int E[X \mid Y = y]\, f_Y(y)\, dy$$

Método de estados

Para variables que miden el tiempo o número de pasos hasta completar un proceso. Se define E_s como la cantidad esperada de pasos restantes estando en el estado s:

$$E_s = 1 + \sum_{s'} P(s \to s')\, E_{s'}, \qquad E_{\text{final}} = 0$$

Tener en cuenta que el 1 cuenta el paso que se da desde el estado actual. Si el grafo de estados tiene ciclos, se plantea un sistema de ecuaciones lineales.

- **Geométrica** (primer éxito con probabilidad p): $E_1 = \frac{1}{p}$
- **Racha de n éxitos consecutivos** (cada intento con probabilidad p), con E_n el número esperado de intentos para lograr n éxitos seguidos:

$$E_0 = 0, \qquad E_n = \frac{E_{n-1} + 1}{p} \qquad \Longrightarrow \qquad E_n = \frac{p^{-n} - 1}{1 - p}$$

- **Ruina del jugador:** se avanza $+1$ con probabilidad p y -1 con probabilidad $q = 1 - p$, empezando en k y parando al llegar a 0 o a N.
  - Si $p = q = \frac{1}{2}$: $E[\text{pasos}] = k(N - k)$
  - Si $p \neq q$: $E[\text{pasos}] = \frac{k}{q-p} - \frac{N}{q-p} \cdot \frac{1-(q/p)^k}{1-(q/p)^N}$
- **Llegar al nivel inferior:** el número esperado de pasos para llegar al nivel inmediatamente inferior a aquel desde el cual se inició, donde en cada paso se baja un nivel con probabilidad a y se sube uno con probabilidad $b < a$:

$$E = \frac{1}{a - b}$$

Para llegar m niveles más abajo el tiempo esperado es $\frac{m}{a-b}$.

- **Coleccionista de cupones:** se quiere coleccionar n tipos de cupones donde en cada compra sale uno al azar con la misma probabilidad.

Estando en el estado con $i - 1$ cupones distintos, sale uno nuevo con probabilidad $\frac{n-(i-1)}{n}$, por lo que ese paso dura en promedio $\frac{n}{n-i+1}$. Por linealidad:

$$E[T] = \sum_{i=1}^{n} \frac{n}{n - i + 1}$$

Varianza y covarianza

La varianza mide qué tan dispersos están los valores de X alrededor de su media μ. Es el promedio de la distancia al cuadrado a la media.

La covarianza mide cómo se mueven juntas dos variables X y Y:

- **Positiva:** cuando X está por encima de su media, Y tiende a estarlo también (suben juntas).
- **Negativa:** cuando X sube, Y tiende a bajar.

- **Cero:** no hay relación lineal.
- **Varianza:**

$$\mathrm{Var}(X) = E[(X-\mu)^2] = E[X^2] - E[X]^2$$

- **Escala y traslación:**

$$\mathrm{Var}(aX+b) = a^2\,\mathrm{Var}(X)$$

- **Covarianza:**

$$\mathrm{Cov}(X,Y) = E[XY] - E[X]\,E[Y]$$

- **Varianza de una suma:**

$$\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} \mathrm{Var}(X_i) + 2\sum_{i<j} \mathrm{Cov}(X_i, X_j)$$

  Si las variables son independientes, todas las covarianzas son 0 y las varianzas se suman.

- **Varianza de una indicadora:**

$$\mathrm{Var}(\mathbb{1}_A) = P(A)\,(1-P(A))$$

- **Segundo momento de un conteo** ($X = \sum_i \mathbb{1}_{A_i}$):

$$E[X^2] = \sum_i P(A_i) + 2\sum_{i<j} P(A_i \cap A_j)$$

- **Ley de la varianza total:**

$$\mathrm{Var}(X) = E[\mathrm{Var}(X \mid Y)] + \mathrm{Var}(E[X \mid Y])$$

Distribuciones comunes

Para cada una se muestra $E[X]$ y $\mathrm{Var}(X)$.

- **Bernoulli(p):** Un solo intento, $X=1$ si hay éxito $X=0$ si no. $E[X]=p,\ \mathrm{Var}(X)=p(1-p)$
- **Binomial(n,p)** (éxitos en n intentos):

$$P(X=k) = \binom{n}{k} p^k (1-p)^{n-k}, \qquad E[X]=np, \qquad \mathrm{Var}(X)=np(1-p)$$

- **Geométrica(p)** (intento en que ocurre el primer éxito):

$$P(X=k) = (1-p)^{k-1}p, \qquad E[X]=\frac{1}{p}, \qquad \mathrm{Var}(X)=\frac{1-p}{p^2}$$

- **Binomial negativa(r,p)** (intento en que ocurre el r-ésimo éxito):

$$P(X=k) = \binom{k-1}{r-1} p^r (1-p)^{k-r}, \qquad E[X]=\frac{r}{p}, \qquad \mathrm{Var}(X)=\frac{r(1-p)}{p^2}$$

- **Hipergeométrica(N,K,n)** (éxitos en n extracciones sin reemplazo de una población de N con K éxitos):

$$P(X=k) = \frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}, \qquad E[X]=n\frac{K}{N}, \qquad \mathrm{Var}(X)=n\,\frac{K}{N}\,\frac{N-K}{N}\,\frac{N-n}{N-1}$$

- **Poisson(λ)** probabilidad de que ocurra k veces un evento en un intervalo fijo, siendo que el promedio de ocurrencias en ese intervalo es λ:

$$P(X=k) = e^{-\lambda}\frac{\lambda^k}{k!}, \qquad E[X]=\mathrm{Var}(X)=\lambda$$

  Ejemplo: llegan en promedio 3 correos por hora, y la probabilidad de recibir exactamente 5 en una hora es $e^{-3}\frac{3^5}{5!}$.

- **Uniforme continua en** $[a,b]$ (Todos los valores en el intervalo son igual de probables): $E[X]=\frac{a+b}{2},\ \mathrm{Var}(X)=\frac{(b-a)^2}{12}$
- **Exponencial(λ):** X es el tiempo de espera hasta que ocurre un evento que sucede en promedio cada $\frac{1}{\lambda}$ unidades de tiempo.

  $E[X]=\frac{1}{\lambda},\ \mathrm{Var}(X)=\frac{1}{\lambda^2}$

  Probabilidad de esperar a lo sumo un tiempo x:

$$P(X \le x) = 1 - e^{-\lambda x}$$

  Probabilidad de esperar más de un tiempo x:

$$P(X > x) = e^{-\lambda x}$$

  No tiene memoria: haber esperado ya un rato no cambia cuánto falta.

7.7 FFT

```
/*
FFT
- Multiplica un polinomio grado 'n' con otro grado 'm'.
- Complejidad, sea N la primer potencia de 2 mayor a n+m-1:
  temporal: O(N * log(N)), espacial: O(N).
*/

typedef double fd; // double para los datos (mas rapido) ; ld queda para los twiddles
typedef complex<fd> C;

void fft(vector<C>& a){
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<ld>> R(2, 1);
    static vector<C> rt(2, 1);
    for(static int k = 2; k < n; k *= 2){
        R.resize(n); rt.resize(n);
        auto x = polar(ld(1), acos(-1.0L) / k);
        forlr(i, k, 2*k-1) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    for0(i, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for0(i, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for(int k = 1; k < n; k *= 2)
        for(int i = 0; i < n; i += 2 * k) for0(j, k){
            auto x = (fd *)&rt[j+k], y = (fd *)&a[i+j+k];
            C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
}
```

7.8 Random

```
//Esta funcion genera numeros aleatorios hasta de 64 bits
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());

void solver(){
    long long x;
    x = rng();
    cout<<x<<endl;
}
```

```
// c[x] = sum a[i]*b[x-i]   (resultado exacto en enteros)
vl conv(const vl& a, const vl& b) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
    vector<C> in(n), out(n);
    copy(all(a), begin(in));
    for0(i, sz(b)) in[i].imag(b[i]);
    fft(in);
    for (C& x : in) x *= x;
    for0(i, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    for0(i, sz(res)) res[i] = floor(imag(out[i]) / (4 * n) + 0.5);
    return res;
}

// igual que conv pero modulo M (cualquier M, no solo primos NTT)
vl convMod(const vl& a, const vl& b, int M) {
    if (a.empty() || b.empty()) return {};
    vl res(sz(a) + sz(b) - 1);
    int B = 32 - __builtin_clz(sz(res)), n = 1 << B, cut = int(sqrt(M));
    vector<C> L(n), R(n), outs(n), outl(n);
    for0(i, sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
    for0(i, sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
    fft(L), fft(R);
    for0(i, n) {
        int j = -i & (n - 1);
        outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
        outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / C(0, 1);
    }
    fft(outl), fft(outs);
    for0(i, sz(res)) {
        ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i]) + .5);
        ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
        res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
    }
    return res;
}

void solver(){
    int n, m; cin>>n>>m;            // cantidad de coeficientes de A y de B
    vl a(n), b(m);
    for0(i, n) cin >> a[i];   // a[i] = coeficiente de x^i
    for0(i, m) cin >> b[i];
    vl c = conv(a, b);          // tamano n + m - 1
    for0(i, sz(c)) cout << c[i] << " \n"[i + 1 == sz(c)];
}
```

7.9 Composicion de funciones

```
/*
- f o g representa f(g(x))
- No es conmutativa: f o g != g o f
- Es asociativa: f o (g o h) = (f o g) o h
- Funcion identidad: I(x) = x
- Elemento neutro: f o I = I o f = f
- Funcion inversa: ~f es la inversa de f, se halla
  despejando x y swapeando x con y.
  Para poder hallarla se requiere que la funcion sea
  inyectiva (a cada elemento del dominio le
  corresponde un elemento diferente del codominio,
  es decir, si f(a) = f(b) entonces obligatoriamente
  a = b), de lo contrario se debe restringir el
  dominio para que se cumpla esa propiedad.
- Componer con la inversa: f o ~f = ~f o f = x
- La inversa de la composicion de 2 funciones es
  igual a la composicion de sus inversas en orden
  contrario: ~(f o g) = ~g o ~f
- Para realizar la composicion NO es necesario saber
  de antemano el valor de x
*/
```

7.10 Numeros practicos

```
/*
- Halla si un numero 'n' es practico.
- Complejidad: O(sqrt(n)) pero en la practica suele ser
    mejor.
  (Se puede mejorar con pollard rho).
- Un numero practico es un entero positivo 'n' tal que
    todo numero
  entero entre '1' y 'n' puede escribirse como suma de
      distintos
  divisores de 'n'.
Propiedades:
- El unico impar que es numero practico es el 1.
- Todas las potencias de '2' son practicas.
- Todos los numeros perfectos pares son practicos.
- Todo practico mayor que '2' es divisible por '4' o por
    '6'.
- El producto de dos numeros practicos es practico.
*/
bool isPractical(ll n){
    if(n == 1) return 1;
    if(n & 1) return 0;

    ll sigma = 1;                          // sigma de la
        parte ya procesada
    for(ll p = 2; p * p <= n && p <= sigma + 1; ++p){
        if(n % p) continue;
        ll pk = 1, term = 1;               // term = 1 + p
            + ... + p^a
        while(n % p == 0){
            n /= p;
            pk *= p;
            term += pk;
        }
        sigma *= term;
    }
    if(n > 1 && n > sigma + 1) return 0;
    return 1;
}

void solver(){
    ll n; cin>>n;
    cout<<(isPractical(n) ? "Yes" : "No")<<endl;
}
```

8 Number Theory

8.1 Binpow

```
// O(log(e))
// Calcula (b ^ e) % mod
ll binpow(ll b, ll e){
    b %= mod;
    ll ans = 1;
    while(e){
        if(e & 1) ans = (ans * b) % mod;
        b = (b * b) % mod;
        e >>= 1;
    }
    return ans;
}
```

8.2 Inverso modular

```
// (a * a^-1) % mod = 1
// Para poder hallar a^-1 se requiere que a y mod sean
    coprimos
// Complejidad: O(log(n))
ll mod_inverse(ll a, ll mod){
    ll i = mod, v = 0, d = 1;
    while(a > 0){
        ll t = i/a, x = a;
        a = i%x;
        i = x;
        x = d;
        d = v - t*x;
        v = x;
    }
    v %= mod;
    if(v < 0) v += mod;
    return v; // (a * v) % mod = 1
}
```

8.3 Descomposicion prima

```
/*
- Complejidad: O(sqrt(n))
- Factorizacion prima de un numero.
  n = p_1 ^ a_1 * p_2 ^ a_2 * ... * p_k ^ a_k
  donde p_k son los numeros primos que componen n y
  a_k es la cantidad de veces que esta dicho primo.
*/
vi factors(int n){
    vi f;
    for(int x = 2; x*x <= n; x++){
        while(n % x == 0){
            f.pb(x);
            n /= x;
        }
    }

    if(n > 1) f.pb(n);
    return f;
}
```

8.4 Sieve of Eratosthenes

```
// Encuentra todos los numeros primos entre 1 y n
// Complejidad: O(n * log(log(n)))
// Algoritmo: Criba de Eratostenes.

// Se usa limite como n
// Hasta este limite hay un poco mas de 6e5 primos
// Fomula de cantidad de primos aprox hasta n -> n / log
    (n)
const int limite = 10001000;

vb esPrimo(limite + 1, 1);
vi primos;

void sieve(){
    esPrimo[0] = esPrimo[1] = 0;
    for(ll p = 2; p*p <= limite; p++){
        if(esPrimo[p]){
            for(ll i = p * p; i <= limite; i += p)
                esPrimo[i] = 0;
        }
    }

    forlr(i,2, limite) if(esPrimo[i]) primos.pb(i);
}
```

8.5 Segmented sieve

```
// Halla los numeros primos en [L, R].
// isPrime[idx] dice si L + idx es primo
// Complejidad: O((R-L+1)*log(log(R)) + sqrt(R)*log(log(
    R))).
// Usar hasta R-L+1 aprox 1e7 y R aprox 1e12.

vb segmentedSieve(ll L, ll R){
    // generate all primes up to sqrt(R)
    ll lim = sqrt(R);
    vb mark(lim + 1);
    vl primes;
    for(ll i = 2; i <= lim; ++i){
        if(!mark[i]){
            primes.emplace_back(i);
            for(ll j = i * i; j <= lim; j += i)
                mark[j] = 1;
        }
    }

    vb isPrime(R - L + 1, 1);
    for(ll i : primes)
        for(ll j = max(i * i, (L + i - 1) / i * i); j <=
            R; j += i)
            isPrime[j - L] = 0;
    if(L == 1) isPrime[0] = 0;
    return isPrime;
}
```

8.6 Divisores de cada numero

```
// Complejidad: O(n * lg(n))
// Almacena en orden ascendente todos los divisores de
    los numeros del 1 al maxn
const int maxn = 2e5 + 2;

vi divs[maxn];

void pre(){
    forlr(i,2,maxn-1){
        for(int j = i; j < maxn; j += i) divs[j].pb(i);
    }
}
```

8.7 Miller Rabin

```
/*
- Verificar rapidamente si un numero n es primo.
- Complejidad por cada check_composite: log(n).
*/

using u64 = uint64_t;
using u128 = __uint128_t;

u64 binpow(u64 b, u64 e, u64 m){
    b %= m;
    u64 ans = 1;
    while(e){
        if(e & 1) ans = ((u128)b * ans) % m;
        b = ((u128)b * b) % m;
```

```
        e >>= 1;
    }
    return ans;
}

bool check_composite(u64 n, u64 a, u64 d, int s){
    u64 x = binpow(a, d, n);
    if(x == 1 || x == n - 1) return 0;
    for1(r,s-1){
        x = ((u128)x * x) % n;
        if(x == n - 1) return 0;
    }
    return 1;
};

// Version optimizada
bool MillerRabin(u64 n) { // returns true if n is prime,
    else returns false.
    if(n < 2) return 0;

    // Testear con los primeros primos para optimizar ya
        que el 88% de numeros son divisibles por al
        menos uno.
    // Agregar mas no optimiza casi nada.
    for(int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
        37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79,
        83, 89, 97}){
        if(n == (u64)a) return 1;
        if(!(n % a)) return 0;
    }
    for(u64 a : {193, 407521, 299210837}) if(n == a)
        return 1; // Primos especiales que se testean
        aparte

    int r = 0;
    u64 d = n - 1;
    while((d & 1) == 0){
        d >>= 1;
        r++;
    }

    for(u64 a : {2, 325, 9375, 28178, 450775, 9780504,
        1795265022}){
        if(check_composite(n, a, d, r)) return 0;
    }
    return 1;
}

// Version un poco mas lenta.
// Si n es un entero de 32 bits, solo testear con 2, 3,
    5 y 7.
bool MillerRabin(u64 n) { // returns true if n is prime,
    else returns false.
    if(n < 2) return 0;

    int r = 0;
    u64 d = n - 1;
    while((d & 1) == 0){
        d >>= 1;
        r++;
    }

    for(int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
        37}){
        if(n == a) return 1;
        if(check_composite(n, a, d, r)) return 0;
    }
    return 1;
}

void solver(){
    u64 n; cin>>n;
    cout<<(MillerRabin(n) ? "YES" : "NO")<<endl;
}
```

8.8 Pollard rho

```
/*
- Halla la factorizacion prima de n.
- Complejidad: O(n^(1/4) * log(n)).
- Para numeros grandes es mas eficiente que el algoritmo
    clasico en sqrt(n).
  Para numeros pequenos si es mejor usar el otro.
*/

using i128 = __int128_t;
mt19937_64 rng(chrono::steady_clock::now().
    time_since_epoch().count());

ll mult(ll a, ll b, ll mod){ return ((i128)a * b) % mod;
    }
ll f(ll x, ll c, ll mod){ return (mult(x, x, mod) + c) %
    mod; }

ll rho(ll n){
    ll x = rng() % n;
    ll c = (rng() % (n-1)) + 1;
    ll g = 1;
    ll q = 1;
    ll xs, y;

    int m = 128;
    int l = 1;
    while(g == 1){
        y = x;
        for(int i = 1; i < l; i++) x = f(x, c, n);
        int k = 0;
        while(k < l && g == 1){
            xs = x;
            for(int i = 0; i < m && i < l - k; i++){
                x = f(x, c, n);
                q = mult(q, abs(y - x), n);
            }
            g = gcd(q, n);
            k += m;
        }
        l *= 2;
    }
    if(g == n){
        do{
            xs = f(xs, c, n);
            g = gcd(abs(xs - y), n);
        } while (g == 1);
    }
    return g == n ? rho(n) : g;
}

void factorize(ll n, map<ll, int>& fact){
    if(n == 1) return;
    if(MillerRabin(n)){ // Primero testear si es primo
        fact[n]++;
        return;
    }
    ll g = rho(n);
    factorize(g, fact); factorize(n/g, fact);
}

void solver(){
    ll n; cin>>n; // Se puede optimizar testeando antes
        los primeros primos (aprox hasta 1000)
    map<ll, int> fact;
    factorize(n, fact); // Se guarda en fact la
        factorizacion prima de n
}
```

8.9 Funcion de Mobius

```
// La funcion de Mobius (mu) es una funcion fundamental
    en teoria de numeros.
// Se usa mucho en problemas de inclusion-exclusion
    sobre divisores o primos
// (por ejemplo, para contar pares coprimos, sumas sobre
    gcd, etc).
//
// Se define segun la factorizacion en primos de x:
/*
        |  1,      si x = 1
u(x) =  |  0,      si x tiene algun factor primo al
    cuadrado (no es squarefree)
        |  (-1)^k, si x es squarefree y tiene k factores
            primos distintos
*/
// sieve_mobius calcula mu[x] para todo x en [1, limite]
    en O(n log log n)
// usando una criba de Eratostenes modificada:
// - Por cada primo p, multiplica por -1 el mobius de
    todos sus multiplos
//   (equivale a agregar un factor primo distinto).
// - Luego anula (pone en 0) el mobius de los multiplos
    de p^2,
//   ya que esos x dejan de ser squarefree.

const int limite = 1E6;

vi mobius(limite + 1, 1);
vb esPrimo(limite + 1, 1);

void sieve_mobius(){
    mobius[1] = 1;
    for(ll p = 2; p <= limite; p++){
        if(esPrimo[p]){
            mobius[p] = -1;
            for(ll j = 2*p; j<= limite; j+=p){
                esPrimo[j] = 0;
                mobius[j] *= -1;
            }

            ll p2 = p * p;
            for(ll j = p2; j <= limite; j+=p2){
                mobius[j] = 0;
            }
        }
    }
}

// Una forma alternativa es usando la propiedad de
    inclusion-exclusion
// Note que el array esta vez tiene que inicializarse en
    0
// Esta forma es O(n log n)
vi mobius(limite + 1, 0);

void inclusion_mobius(){
    mobius[1] = -1;
    for1(i,limite){
        if(mobius[i]){
            mobius[i] = -mobius[i];
            for(int j = 2 * i; j <= limite; j += i){
                mobius[j] += mobius[i]; }
        }
    }
}
```

8.10 Funcion de Euler

```
// Halla phi(n), la cantidad de numeros entre 1 y n que
    son coprimos con n
// Complejidad: O(sqrt(n))
int phi(int n){
    int ans = n;
    for(int i = 2; i * i <= n; i++){
        if(n % i == 0){
            while(n % i == 0)
                n /= i;
```

```
            ans -= ans / i;
        }
    }
    if(n > 1) ans -= ans / n;
    return ans;
}

// Halla phi(n) para todos los numeros entre 1 y n
// Complejidad: O(n * log(log(n)))
void phi_1_to_n(int n){
    vi phi(n + 1);
    for0(i,n+1) phi[i] = i;

    forlr(i,2,n){
        if(phi[i] == i){
            for(int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

8.11 Mulpow

```
// Util si se requiere modularidad pero el modulo es muy grande
// como para multiplicar directamente.
// O(log(n))

const ll mod = (ll)1e18 + 3;

ll binmul(ll b, ll e){
    ll ans = 0;
    b %= mod; e %= mod;
    while(e){
        if(e & 1) ans = (ans + b) % mod;
        b = (b + b) % mod;
        e >>= 1;
    }
    return ans;
}
```

8.12 Euclides

```
// O(log(n)), siendo n el minimo entre a y b.
// En versiones recientes de C++ ya estan de forma nativa, en otras
// mas antiguas se puede llamar usando __gcd o __lcm.
ll gcd(ll a, ll b){
    ll temp;
    while(b != 0){
        temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

ll lcm(ll a, ll b){
    return (a / gcd(a, b)) * b;
}
```

8.13 Diofanticas

```
/*
- Complejidad: O(log(n)), siendo n el minimo entre a y b
- Ecuacion de la forma ax + by = c, donde a, b, c
  son constantes y hay que encontrar x, y.
- Cada valor debe ser entero.
- Solo tiene solucion si c es divisible por gcd(a, b)
- En este algoritmo se obtiene un par de soluciones x, y. Una
  vez se obtiene una solucion se pueden formar infinitar a partir
  de esta, asi: (x + (k*b)/gcd(a, b), y - (k*a)/gcd(a, b)), donde
  k es un entero.
*/
void extendedEuclid(ll a, ll b, ll &x, ll &y, ll &mcd){
  if(b == 0){
    x = 1;
    y = 0;
    mcd = a;
  }
  else{
    extendedEuclid(b, a % b, x, y, mcd);
    x = y;
    y = (mcd - a*x) / b;
  }
}

void solver(){
  ll a, b, x, y, mcd;
  a = 146;
  b = 388;
  extendedEuclid(a, b, x, y, mcd);
  cout<<mcd<<' '<<x<<' '<<y<<endl;
}
```

8.14 Chinese remainder theorem

```
using i128 = __int128_t;

/*
- Sirve para hallar x para un grupo de ecuaciones de la forma:
  x % m_1 = a_1 % m_1
  x % m_2 = a_2 % m_2
  ...
  x % m_n = a_n % m_n
  Donde para cada par i, j donde i != j, m_i, m_j es coprimo.
- Una vez se halla una solucion se pueden crear infinitas de la forma
  x + k*M, donde M es m_1 * m_2 * ... * m_n, y k es un entero cualquiera.
- Complejidad: O(n*log(n)).
*/

ll mulmod(ll a, ll b, ll mod){
    return (i128)a * b % mod;
}


ll chinese_reminder(int n, ll M, vl& a, vl& m){
    ll x = 0;
    // vl d(n), c(n);
    for0(i,n){
        ll d = M / m[i];
        x += mulmod(a[i], (mulmod(d, mod_inverse(d, m[i]), M)), M);
        x %= M;
        // En caso de ser necesarios los vectores d y c
        // d[i] = M/m[i];
        // c[i] = (d[i] * mod_inverse(d[i], m[i])) % M;
        // x = (x + mulmod(a[i], c[i], M)) % M;
    }

    return x % M;
}

void solver(){
    int n; cin>>n;
    vl a(n), m(n);
    ll M = 1;
    for0(i,n){
        cin>>a[i]>>m[i];
        M *= m[i];
    }
    ll x = chinese_reminder(n, M, a, m);
    cout<<x<<endl;
}
```

8.15 Fibonacci code

```
const int len = 51; // Para calcular hasta el n-esimo termino
vl fibo(len);

// Calcula Fibonacci. Tomando fibo[0] = 0, fibo[1] = 1
// Complejidad: O(n).
void calcFibo(){
    fibo[1] = 1;
    forlr(i,2,len-1) fibo[i] = fibo[i-2] + fibo[i-1];
}

/*
- Calcula el codigo de fibonacci de un numero n.
- Todo numero natural n puede ser representado como una secuencia unica
  de numeros de Fibonacci tal que no hay 2 numeros consecutivo iguales.
- El algoritmo itera cada numero de fibonacci f[i] con i desde len hasta
  2, revisa si este es menor o igual a n, en caso de que si asigna 1 en
  la posicion i-2 del codigo y resta el valor a n, y en caso de que no
  asigna 0 en la posicion i-2.
- Complejidad: aproximadamente O(log(n)).
*/
vb fiboCode(ll n){
    vb code(len - 1);
    forrl(i,2,len-1){
        if(n >= fibo[i]){
            code[i-2] = 1;
            n -= fibo[i];
        }
    }

    return code;
}

void solver(){
    calcFibo(); // Precalcular Fibonacci

    // Mostrar el codigo de fibonacci de los numeros desde 0 hasta 10
    for0(i,11){
        vb code = fiboCode(i);
        for0(i,12) cout<<code[i];
        cout<<endl;
    }
}
```

8.16 Teoria

Propiedades y teoremas

- En promedio los números tienen $n^{1/3}$ divisores.
- Sea $d(i)$ el número de divisores de i, la sumatoria de $d(i)$ para i entre 1 y n es aproximadamente $n \ln(n)$.
- Entre 1 y n hay aproximadamente $\pi(n) = \frac{n}{\ln(n)}$ números primos.
- La distancia promedio entre primos desde 1 hasta n es $\ln(n)$.

  Para todos los enteros hasta 10^9, la máxima distancia entre primos es 282. Hasta 10^{18} es 1442.

- **Teorema de Bertrand:** para todo $n \geq 2$ hay al menos un primo $n < p < 2n$.

Descomposición prima

Descomposición prima de un número: $n = p_1^{a_1} \cdot p_2^{a_2} \cdots p_k^{a_k}$

- **Número de divisores** de n:

$$\tau(n) = \prod_i (a_i + 1)$$

- **Suma de factores** de n:

$$\sigma(n) = \prod_i \frac{p_i^{a_i+1} - 1}{p_i - 1}$$

- **Producto de los factores** de n:

$$u(n) = n^{\tau(n)/2}$$

- Un número es **perfecto** si $n = \sigma(n) - n$, es decir, si la suma de sus factores entre 1 y $n-1$ es igual a sí mismo.
- Una función es **multiplicativa** si a, b coprimos implica $f(a \cdot b) = f(a) \cdot f(b)$.
- Las funciones $d(n)$ y $\sigma(n)$ son multiplicativas.

Función phi de Euler (totient)

$\phi(n)$ da la cantidad de números coprimos con n entre 1 y n.
Ej: $\phi(12) = 4$, ya que $1, 5, 7$ y 11 son coprimos con 12.

$$\phi(n) = \prod_i p_i^{a_i - 1}(p_i - 1)$$

- Si p es primo: $\phi(p) = p - 1$
- Si p es primo y $k \geq 1$: $\phi(p^k) = p^k - p^{k-1}$
- Si a y b son coprimos: $\phi(a \cdot b) = \phi(a) \cdot \phi(b)$ (es multiplicativa)
- Para cualquier par $\{a, b\}$ con $d = \gcd(a, b)$:

$$\phi(a \cdot b) = \phi(a) \cdot \phi(b) \cdot \frac{d}{\phi(d)}$$

- Para cualquier número n: $\sum_{d|n} \phi(d) = n$

  $d \mid n$ quiere decir que d es divisor de n

Exponenciación con phi:

- Para calcular $(a^{b^c}) \bmod m$:

  $m_1 = \phi(m)$, luego $(a^{(b^c \bmod m_1)}) \bmod m$.

- Para calcular $(a^{b^{c^d}}) \bmod m$:

  $m_1 = \phi(m)$, $m_2 = \phi(m_1)$, luego $(a^{(b^{(c^d \bmod m_2)} \bmod m_1)}) \bmod m$.

  Y así sucesivamente para cadenas de potencias.

- **Generalización:** $a^n \bmod m = a^{(n \bmod \phi(m))} \bmod m$

Teoremas de congruencias

- **Pequeño teorema de Fermat:** $x^{m-1} \bmod m = 1$, siendo m primo y x, m coprimos.

  Además: $x^k \bmod m = x^{(k \bmod (m-1))} \bmod m$

- **Teorema de Euler (generalización):** $x^{\phi(m)} \bmod m = 1$ cuando x, m son coprimos.
- **Inverso modular:** si existe, se calcula como $a^{-1} = a^{\phi(m)-1}$; si m es primo, $x^{-1} = x^{m-2}$.

Otros teoremas

- **Teorema de los números poligonales de Fermat:** todo número puede representarse como la suma de 3 números triangulares, 4 números cuadrados, 5 números pentagonales, y n números n-poligonales.
- **Ternas pitagóricas:** ternas (a, b, c) que satisfacen $a^2 + b^2 = c^2$, es decir, hay un triángulo rectángulo con lados a, b, c (ej: $3, 4, 5$).
  - Si (a, b, c) es una terna pitagórica, todas las ternas (ka, kb, kc) con $k > 1$ también lo son.
  - Una terna es **primitiva** si $\gcd(a, b, c) = 1$. Todas las ternas pitagóricas pueden generarse a partir de estas usando un factor k.
  - **Fórmula de Euclides:** genera todas las ternas primitivas de la forma
    $$(n^2 - m^2,\ 2nm,\ n^2 + m^2)$$
    donde $0 < m < n$, n y m son coprimos y al menos uno de los dos es par.
  - La terna pitagórica primitiva más pequeña es $(3, 4, 5)$.
- **Teorema de Wilson:** un número n es primo si y solo si $(n-1)! \bmod n = n - 1$.

Fibonacci

- Empieza con $F_0 = 0$, $F_1 = 1$.
- **Fórmula de Binet:**

$$F_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}}$$

  Sin embargo, es difícil de usar por el redondeo.

- **Con matrices:**

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

- **Identidad de Cassini:** $F_{n+1}F_{n-1} - F_n^2 = (-1)^n$
- **Regla de adición:** $F_{n+k} = F_k F_{n+1} + F_{k-1} F_n$

  Cuando $k = n$: $F_{2n} = F_n(F_{n+1} + F_{n-1})$

  Por lo tanto, para cualquier entero positivo k, F_{kn} es múltiplo de F_n, y si F_m es múltiplo de F_n, entonces m es múltiplo de n.

- **Identidad del GCD:** $\gcd(F_m, F_n) = F_{\gcd(m,n)}$
- Para cualquier número p, existe una secuencia de Fibonacci módulo p que se repite periódicamente, de longitud máxima p^2.
- **Teorema de Zeckendorf:** todo entero positivo tiene una única representación como suma de números de Fibonacci tal que ningún par de números es igual ni son dos consecutivos. A partir de esto se calcula el código de Fibonacci.

Conjeturas (no probadas)

- **Goldbach:** todo entero > 2 puede representarse como la suma de 2 números primos.
- **Twin prime:** hay un número infinito de pares $\{p, p+2\}$ donde ambos son primos.
- **Legendre:** siempre hay un primo entre n^2 y $(n+1)^2$.

---

9 Miscelanea

9.1 Compresion

```
typedef pair<pii, int> piii;
typedef vector<piii> viii;

// O(n*lg(n))

// Comprimir array
void compressArr(vi& a){
    vi comp = a;
    sort(all(comp));
    comp.erase(unique(all(comp)), comp.end());
    for0(i,sz(a)) a[i] = lb(all(comp), a[i]) - comp.
        begin();

    /* En caso de requerir hallar la compresion de algun
    valor k, sin que se necesite guardar en el array:
    k = lb(all(comp), k) - comp.begin();
    Usar lb o ub segun sea el caso.*/
}

// Comprimir queries para segtree
void compressQ(viii& a){
    vi comp;
    for0(i,sz(a)){
        comp.pb(a[i].fi.fi);
        comp.pb(a[i].fi.se);
    }
    sort(all(comp));
    comp.erase(unique(all(comp)), comp.end());
    for0(i,sz(a)){
```

```
        a[i].fi.fi = lb(all(comp), a[i].fi.fi) - comp.
            begin();
        a[i].fi.se = lb(all(comp), a[i].fi.se) - comp.
            begin();
    }
}
```

9.2 Biseccion

```
// Con avance r = m, l = m + 1 (Hallar el menor que
    cumple)
// La respuesta se guarda en cualquiera de l o r
while(l < r){
    int m = l + (r - l) / 2;
    if(can(m)) r = m;
    else l = m + 1;
}

// Con avance r = m - 1, l = m (Hallar el mayor que
    cumple)
// La respuesta se guarda en cualquiera de l o r
while(l < r){
    int m = l + (r - l + 1) / 2;
    if(can(m)) l = m;
    else r = m - 1;
}
```

9.3 Busqueda ternaria

```
int ternarySearch(vi& A, int i, int j, int s){
    int l, r;
    while(i <= j){
        l = i + (j-i)/3, r = j - (j-i)/3;
        if(A[l] == s) return l;
        if(A[r] == s) return r;
        if(A[l] > s)
            i = r+1;
        else if(A[r] > s) {
            i = l+1;
            j = r-1;
        }
        else
            j = l-1;
    }
    return -i -1;
}
```

9.4 Bits

```
// Operaciones de bits (x > 0)
// Cantidad de ceros a la derecha (lsb)
#define ictz(x) __builtin_ctz(x)
#define llctz(x) __builtin_ctzll(x)

// Cantidad de ceros a la izquierda
#define iclz(x) __builtin_clz(x) // (msb = 31 -
    __builtin_clz(x))
#define llclz(x) __builtin_clzll(x) // (msb = 63 -
    __builtin_clz(x))

// Cantidad de bits en 1
#define llpc(x) __builtin_popcountll(x)
#define ipc(x) __builtin_popcount(x)

// Some useful equations
a|b = a^b + a&b
a^(a&b) = (a|b)^b
(a&b)^(a|b) = a^b

a+b = a|b + a&b
a+b = a^b + 2(a&b)

a-b = (a^(a&b)) - ((a|b)^a)
a-b = ((a|b)^b) - ((a|b)^a)
a-b = (a^(a&b)) - (b^(a&b))
a-b = ((a|b)^b) - (b^(a&b))
```

9.5 Sliding window

```
// Para un sliding window de minimos. Complejidad O(n)
void sw(vi& a, int k){
    int n = sz(a); // Se asume que n >= k
    deque<pii> dq;

    // Insertar los primeros k-1
    for0(i,k-1){
        // Elimino todos los mayores o iguales que puse
            antes
        while(!dq.empty() && dq.back().fi >= a[i]) dq.
            pop_back();
        dq.pb({a[i], i});
    }

    forlr(i,k-1,n-1){
        if(i - dq.front().se == k) dq.pop_front(); //
            Elimino el elemento que sale de la ventana

        // Elimino todos los mayores o iguales al nuevo
            elemento
        while(!dq.empty() && dq.back().fi >= a[i]) dq.
            pop_back();
        dq.pb({a[i], i});

        // Elemento menor en esta ventana
        cout<<dq.front().fi<<endl;
    }
}
```

9.6 Game theory

Juegos imparciales

Notación: $v \to u$ significa que desde el estado v existe un movimiento que lleva al estado u.

Se asume que el juego cumple lo siguiente:

- Dos jugadores que se alternan los turnos.
- Los movimientos posibles dependen solo del estado, no de quién juega (juego *imparcial*).
- Información perfecta y sin azar.
- El juego es finito: tarde o temprano alguien no puede mover.
- Juego normal: **pierde quien no puede mover** (gana quien hace el último movimiento). No hay empates.

El juego se puede ver como un grafo dirigido acíclico donde los vértices son los estados y las aristas son los movimientos. Un vértice sin aristas de salida es un estado terminal.

- **Estado ganador:** el jugador que mueve puede forzar la victoria.

$$v \text{ es ganador} \iff \exists u \text{ con } v \to u \text{ y } u \text{ perdedor}$$

- **Estado perdedor:** el jugador que mueve pierde si el rival juega bien.

$$v \text{ es perdedor} \iff \forall u \text{ con } v \to u,\ u \text{ es ganador}$$

Tener en cuenta que un estado terminal es perdedor (no existe ningún u, así que la condición se cumple vacíamente). Con estas dos reglas se clasifican todos los estados empezando desde los terminales. Es una recursión: un estado es ganador si alguno de sus movimientos lleva a un estado perdedor, y perdedor en caso contrario.

Juegos de sustracción

Una sola pila de n fichas y un conjunto S de tamaño m con las cantidades que se pueden quitar en un turno.

- Primero se dice que $g_0 = 0$. Entonces para cada valor $1 \leq i \leq n$, se calcula si algún $g_{i-S_j} = 0$, en caso de que si $g_i = 1$ y se considera un estado ganador y en caso contrario $g_i = 0$ y se considera un esatdo perdedor.
- **Periodicidad:** si en un turno se pueden quitar a lo sumo k fichas, que un estado sea ganador o perdedor depende solo de si los k estados anteriores son ganadores o perdedores. Como solo hay 2^k secuencias posibles de largo k, la secuencia de estados termina siendo periódica.
- Si $S = \{1, 2, 3, ..., m\}$, gana el primer jugador si $n \bmod m > 0$ y pierde en caso contrario.

Nim

Hay n pilas y la pila i tiene x_i palitos. En cada turno se elige una pila no vacía y se quita cualquier cantidad positiva de palitos de ella. Gana el jugador que realiza el último movimiento.

- Sea s el xor del tamaño de todas las pilas, el jugador que empieza es ganador si $s > 0$ y perdedor en caso contrario. A este valor s se le dice nim-sum.
- **Cantidad de movimientos ganadores desde un estado ganador:** es la cantidad de pilas k tales que

$$x_k \oplus s < x_k$$

- Si además se permitiera **agregar** palitos a una pila (con reglas que mantengan el juego finito), el resultado del juego no cambia. Si un jugador agrega palitos, el rival simplemente deshace el aumento quitando exactamente lo mismo. Como el juego es finito, alguien tiene que terminar haciendo un movimiento normal de Nim.
- **Escalera:** si se dijera que para cada fila entre 2 y n, desde la fila i se pasa cualquier cantidad positiva de palitos a la fila $i-1$, y la fila 1 quita cualquier cantidad de palitos, y pierde el jugador que se queda sin movimientos, se halla unicamente el xor del tamaño de las filas con i **impar** $(1, 3, 5, ...)$. Si se plantea como un arbol rooteado donde todas las hojas tienen la misma paridad en su profunidad, cada nodo puede pasar palitos a alguno de sus hijos y las hojas quitan cualquier cantidad, se toma el xor de todos los nodos cuya profundidad tiene la misma paridad que el de las hojas.
- Dos estados de Nim con la misma nim-sum son equivalentes. En particular, cualquier Nim se puede reemplazar por una sola pila de tamaño s.

Nim misère

Es el mismo Nim pero **pierde quien quita el último palito**.

Se juega como Nim normal hasta que el siguiente movimiento dejaría todas las pilas con a lo sumo 1 palito. En ese momento se cambia la regla de paridad:

- Nim normal: dejar una cantidad **par** de pilas con 1 palito.
- Nim misère: dejar una cantidad **impar** de pilas con 1 palito.

Esto funciona porque el estado en el que cambia la estrategia siempre aparece en la partida, y es ganador: tiene exactamente una pila con más de un palito, así que su nim-sum es distinta de 0.

- Si alguna pila tiene $x_i \geq 2$: gana el primero si y solo si $s \neq 0$.
- Si todas las pilas cumplen $x_i \leq 1$: gana el primero si y solo si la cantidad de pilas con 1 palito es **par**.

Números de Grundy

El **número de Grundy** (o nimber, o valor de Grundy) de un estado v es:

$$g(v) = \text{mex}\{\, g(u) : v \to u \,\}$$

Si v no tiene movimientos, $g(v) = 0$.

Propiedades

- $g(v) = 0 \iff v$ es perdedor. Si $g(v) > 0$, el estado es ganador.
- Si $g(v) = 0$, todos los movimientos llevan a estados con Grundy **positivo**.
- Si $g(v) = x > 0$, desde v se puede llegar a estados con cada uno de los valores $0, 1, \ldots, x-1$ (y posiblemente a otros valores mayores que x).
- Una pila de Nim con k palitos tiene $g_k = k$.

Teorema de Sprague-Grundy

Todo juego imparcial, finito, de dos jugadores y con información perfecta (en juego normal) es equivalente a **una sola pila de Nim** de tamaño $g(v)$, donde:

$$g(v) = \text{mex}\{\, g(v_1), g(v_2), \ldots, g(v_k) \,\}$$

siendo $v_1, \ldots, v_k$ los estados alcanzables desde v.

Cuándo pensar en Grundy:

- El juego no es Nim directo, pero las reglas de movimiento dependen solo del estado.
- El juego está formado por componentes independientes.
- Clasificar los estados completos sería demasiado lento, pero cada componente es pequeño y se puede precalcular.
- No es Misère, ya que el teorema vale solo para juego normal.

Receta:

1. Definir el estado y los movimientos.
2. Calcular g de los estados pequeños (a mano o con fuerza bruta), empezando por los terminales.
3. Combinar los componentes con XOR.
4. Si el resultado es 0, el estado es perdedor; si no, es ganador.

Suma de juegos

Si el juego se compone de k subjuegos independientes y en cada turno el jugador **elige uno de ellos y mueve solo en ese**, el número de Grundy del juego completo es el XOR de los números de Grundy de los subjuegos:

$$g(G_1 + G_2 + \cdots + G_k) = g(G_1) \oplus g(G_2) \oplus \cdots \oplus g(G_k)$$

Cada subjuego se comporta como una pila de Nim cuyo tamaño es su Grundy, así que la estrategia es la misma de Nim: si el XOR es $s \neq 0$, se busca el subjuego i donde $g_i \oplus s < g_i$ y se mueve a un estado de ese subjuego con Grundy $g_i \oplus s$. Siempre existe, porque desde un estado con Grundy g_i se alcanzan todos los valores menores.

Variantes de la suma de juegos

Si cambia la regla de qué se puede mover en un turno, ya no se usa el XOR. Sean $g_1, \ldots, g_K$ los números de Grundy de la posición actual de cada subjuego:

- **Se eligen algunos subjuegos (al menos uno) y se mueve en todos los elegidos:** el estado es perdedor si y solo si $g_i = 0$ para todo i.
- **Se eligen algunos subjuegos (al menos uno, pero no todos) y se mueve en todos los elegidos:** el estado es perdedor si y solo si
$$g_1 = g_2 = \cdots = g_K$$

Gana el primero en ganar un componente

Si se tiene un juego que consiste en varios componentes independientes, donde el ganador global es el primero que logre ganar un componente (hace el movimiento que lleva a un estado sin movimientos validos):

- Si en algun componente el jugador que inicia puede llegar a un estado terminal en un solo movimiento, gana inmediatamente.
- Se recorta cada componente: se construye cada componente eliminando los estados terminales y los estados desde los cuales se puede llegar a un estado terminal en un movimiento. En cada componente recortado se hallan los numeros de grundy (los movimientos a estados eliminados no se tienen en cuenta).
- Se halla el xor de todos los componentes y si este es mayor a 0 gana el jugador que inicia, de lo contrario pierde.

Juegos que se parten en subjuegos

A veces un movimiento **divide** el juego en varias partes independientes. Entonces el valor de ese movimiento es el XOR de los números de Grundy de las partes, y el Grundy del estado es el mex de todos esos valores:

$$g(v) = \text{mex}\{\, a_{k,1} \oplus a_{k,2} \oplus \cdots \oplus a_{k,m_k} : k \text{ es un movimiento posible} \,\}$$

donde el movimiento k genera subjuegos con números de Grundy $a_{k,1}, \ldots, a_{k,m_k}$.

Juego de Grundy

Una pila de n palitos. En cada turno se parte una pila en **dos pilas no vacías de tamaños distintos**, y pierde quien no puede mover. Si $f(n)$ es el número de Grundy de una pila de n palitos:

$$f(n) = \text{mex}\{\, f(a) \oplus f(n-a) : 1 \leq a < n - a \,\}$$

Los casos base son $f(1) = f(2) = 0$ (no se pueden partir). Los primeros valores son:

$$f(1), f(2), \ldots, f(8) = 0, 0, 1, 0, 2, 1, 0, 2$$

Por ejemplo, para $n = 8$ las particiones son $1+7$, $2+6$ y $3+5$:

$$f(8) = \text{mex}\{f(1) \oplus f(7),\ f(2) \oplus f(6),\ f(3) \oplus f(5)\} = \text{mex}\{0, 1, 3\} = 2$$

Como $f(8) \neq 0$, gana el primero, y la jugada ganadora es partir en $1+7$ porque $f(1) \oplus f(7) = 0$.

Tener en cuenta que para este juego no se conoce un patrón general en los valores de Grundy, así que hay que calcular la tabla, por ende la solución más rapida es $O(n^2)$. Hasta 10^6 el valor más grande para el cual gana el segundo jugador es 1222.

Patrones en los números de Grundy

En muchos juegos que parecen difíciles de analizar, los números de Grundy resultan periódicos o con una forma fácil de describir. La forma de aprovecharlo:

1. Calcular $g(n)$ con la definición para valores pequeños de n.
2. Imprimir la tabla y buscar periodicidad, dependencia de $n \bmod k$, relación con potencias de 2, etc.
3. Verificar el patrón por inducción si hace falta.

En la gran mayoría de los casos el patrón que se observa es verdadero. Pero hay juegos muy simples (como el juego de Grundy de arriba) donde no se conoce ninguna regularidad.

9.7 Games on arbitrary graphs

```
#include "template.h"

/*
- Para juegos donde ambos juagadores juegan optimamente,
     en grafos
  dirigidos arbitrarios, se quiere saber cuales estados
       (vertices)
  son ganadores, perdedores o llevan a empate.
- En caso de que los movimientos no dependan solo del
    vertice sino
  tambien de cual jugador tiene el turno se debe tomar
       cada estado como
  una pareja {vertice, jugador}.
- Si un estado no tiene aristas de salida o si todas sus
     aristas llevan
  a estados ganadores, es perdedor.
- Si un estado tiene al menos una arista a un estado
    perdedor, es ganador.
- Si ninguna de estas se cumple, este estado lleva a un
    ciclo donde no va
  a haber un ganador.
- Complejidad: O(m).
- Si las transiciones de los estados son similares (ir a
     los lados en un
  laberinto, desde cada vertice u ir al vertice u + x,
       etc). Se puede
  evitar consruir el grafo para ahorrar memoria.
*/
const int maxn = 2e5 + 1;
```

```
vector<vi> graph(maxn);
vi indegree(maxn);
vb win(maxn), lose(maxn), vis(maxn);
void DFS(int u){
    vis[u] = 1;
    for(int v : graph[u]){
        if(vis[v]) continue;
        if(lose[u]) win[v] = 1;
        else if(--indegree[v] == 0) lose[v] = 1;
        else continue;
        DFS(v);
    }
}

void solver(){
    // Crear grafo
    int n, m; cin>>n>>m;
    for0(i,m){
        int u, v; cin>>u>>v;
        graph[v].pb(u); // Guardar las aristas alreves
        indegree[u]++;
    }

    // Procesar
    for1(i,n) if(!indegree[i]) lose[i] = 1;
    for1(i,n) if(lose[i]) DFS(i);

    // Resultados
    for1(i,n){
        if(win[i]) cout<<i<<" es ganador"<<endl;
        if(lose[i]) cout<<i<<" es perdedor"<<endl;
        else cout<<i<<" lleva a empate"<<endl;
    }
}

int main(){
    ios_base::sync_with_stdio(0);cin.tie(0);cout.tie(0);
    // freopen("name.in", "r", stdin);
    // freopen("name.out", "w", stdout);
    int t = 1;
    // cin>>t;
    while(t--){
        solver();
    }

    return 0;
}
```

9.8 Arbol cartesiano

```
/*
Teniendo un array a, armar su arbol cartesiano, es decir, un
arbol donde la raiz es el mayor elemento y para cada nodo, sus 2
hijos son menores. El hijo izquierdo es un nodo menor que se
encuentra a la izquierda en el array y el hijo derecho es un nodo
menor que se encuentra a la derecha en el array.
Proceso:
- Crear una pila
- Recorrer el array en orden i -> [1, n]
- Sacar de la pila todos los elementos menores a a[i]
- El ultimo elemento que se saca es el hijo izquierdo de i
- Si la pila no esta vacia, i es el hijo derecho del tope
- Se inserta i en la pila
Complejidad: O(n)
Notas:
- Si en vez de arbol donde cada hijo es menor, se quiere uno
  donde cada hijo sea mayor, en lugar de sacar de la pila todos
  los elementos menores que a[i] en la pila, se sacan todos los
  mayores.
- En caso de que el array tenga numeros repetidos, la raiz del
  arbol sera el indice de la primera ocurrencia del mayor
  elemento del array.
*/
void cartesianTree(int n, vi& a, vi& sl, vi& sr){
    stack<int> st;
    for0(i,n){
        int last = -1;
        while(!st.empty() && a[i] > a[st.top()]){
            last = st.top();
            st.pop();
        }

        if(last != -1) sl[i] = last;
        if(!st.empty()) sr[st.top()] = i;

        st.push(i);
    }
}
```

9.9 Combinaciones longitud k

```
/*
- Dado un conjunto de n elementos (ordenados y sin repeticiones), se quieren
  generar sus combinaciones de tamano k, en orden ascendente.
- La primera combinacion esta compuesta por los elementos [0, k)
- A partir de ahi, se llama next_combination, la cual da la siguiente combinacion y
  retorna True si la hay, o False en caso de que ya se haya llegado a la ultima.
- Complejiadad: O(k) para hallar la siguiente combinacion, O(k * binom{n, k}) para
  hallarlas todas.
*/

bool next_combination(vi& a, int n){
    int k = sz(a);
    forn0(i,k){
        if(a[i] < n - k + i + 1){
            a[i]++;
            forlr(j, i+1, k-1) a[j] = a[j - 1] + 1;
            return 1;
        }
    }
    return 0;
}
```

9.10 Josefo

```
/*
- Hay n personas en 1 circulo. Se empiezan a eliminar de 1 en 1 hasta obtener un ganador.
  Iniciando con la persona 1, se empieza a contar desde 1 hasta p y se elimina a esa persona.
  Luego se cuenta nuevamente iniciando en la persona a la derecha del anterior eliminado.
  Ejemplo: con n = 5 y p = 3, el primer eliminado es el numero 3, el ganador es 4.
*/

/*
Si solo se quiere saber el ganador
- Complejidad: O(n)
- La idea detras de la solucion es que se calcula la respuesta desde n = 1, hasta llegar al
  n del parametro. Al iniciar con un ciruclo de n personas, y eliminar 1, ahora tenemos el
  mismo problema pero con una posicion inicial diferente y con 1 persona menos, asi que por
  eso se reduce el modulo y se cambia la posicion inicial.
*/
int josefo(int n, int p){
    int ans = 0;
    forlr(i,2,n) ans = (ans + p) % i;
    return ans + 1; // Suma para circulo de 1 a n
}

/*
Si se quiere saber el orden en que se van eliminando
- Complejidad: O(n * log(n))
*/
vi josefo(int n, int p){
    indexed_set nums;
    for1(i,n) nums.insert(i);
    vi ans;
    int cur = (p-1) % n;
    while(sz(nums) > 1){
        int num = *nums.find_by_order(cur);
        ans.pb(num);
        nums.erase(num);
        n--;
        cur = (cur + p - 1) % n;
    }
    ans.pb(*nums.begin()); // Ganador
}
```

9.11 Hashing XOR

```
/* El hashing map sirve para almacenar elementos en un conjunto especifico
Consiste en leer un numero a para almacenar cierto dato en un vector, entonces
al numero que tenemos en la posicion a de nuestro vector le hacemos xor con un
numero r random, asegurando de esta manera que el valor que tendra sera unico */

/* El ejercicio que se encuentra en este archivo consiste en t casos de prueba.
Nos dan n ciudades y todas estan conectadas de la forma i con i+1, para i < n,
y para i = n se dice que esta conectada con la ciudad 1. Entonces nos dan m
pares de amigos que pertenecen respectivamente a la ciudad a y a la ciudad b.
Entonces se nos pregunta cual es la minima cantidad de caminos que puede tener
la ciudad de modo que cada par de amigos este conectado.

Para resolver este ejercicio usamos hashing xor de la siguiente manera:
Se leen los numero a y b y al valor que tenia en cada posicion se le hace
un XOR con un numero random (de 64 bits para evitar colisiones).
Posteriormente, queremos encontrar un numero fixAns que sea la mayor cantidad de
numeros consecutivos iguales, de modo que podamos decir que puedo conectar todas
las ciudades usando n - fixAns caminos. Debido a que nuestro vector tiene guardados
los numeros a los que les aplicamos XOR, podemos elegir un numero arbitrario save
que guarde en dicha posicion dentro de un mapa las veces que encontre save. Save va
a cambiar segun el numero que estamos evaluando del vector y solo volvera a ser el
mismo numero que en una iteracion anterior cuando cruce por un numero en el vector
```

```
que sea diferente de 0, o cuando halla pasado por el
    mayor numero a o b que lei
al inicio del ejercicio.*/

/* Link del ejercicio: https://codeforces.com/problemset
    /problem/1996/G
Para una explicacion mas detallada de la logica detras
    de esta colucion, dirigirse
a la editorial, la cual se puede encontrar en el link
    adjunto*/

// "rng" genera un numero random de 64 bits
mt19937_64 rng(chrono::steady_clock::now().
    time_since_epoch().count());

void solver(){
    int n, m; cin>>n>>m;
    vl D(n);
    while(m--){
        int a, b; cin>>a>>b;
        ll r = rng();
        D[a-1] ^= r;
        D[b-1] ^= r;
    }

    unordered_map<ll, int> conjuntos;
    int fixAns = 0;
    ll save = 0;
    for(ll diag : D) fixAns = max(fixAns, conjuntos[save
        ^=diag]++);

    cout<<n-fixAns-1<<endl;
}
```

9.12 Python

```
# Es importante tener en cuenta que python es
    aproximadamente 40 veces mas lento que C++

def main ():
    # Leer una linea completa, separar los elementos por
        los espacios y almacenarlos en una
    # lista (permitiendo repetidos)
    read = input()
    numbers = read.split()
    numbers = list(map(int, numbers)) # 'map' transforma
        cada elemento de numbers en un int

    #Leer un entero
    n = int(input())

    # Leer un numero flotante
    d = float(input())

    # Leer una cadena
    s = input()

    # Leer hasta fin de archivo
    def func():
        while(True):
            try:
                n = int(input()) # Cambiar por input()
                    si no me dan solo un numero
                # Procesar lo que sea necesario
            except EOFError:
                break


    # Lectura y escritura rapida
    import sys
    input = sys.stdin.readline # Reemplaza input() por
        una version mucho mas rapida (poner afuera de
        todas las funciones)
    # readline NO quita el '\n' del final. int() y float
        () lo ignoran, pero con cadenas hay que
        limpiarlo
    n = int(input())
    numbers = list(map(int, input().split()))
    s = input().strip() # strip() quita espacios y el
        salto de linea

    # Con readline, al llegar a fin de archivo devuelve
        '' (NO lanza EOFError)
    # Leer hasta fin de archivo:
    for line in sys.stdin:
        pass # Procesar line

    # Leer TODA la entrada de una vez y separarla por
        espacios y saltos de linea
    data = sys.stdin.read().split()
    n = int(data[0]) # n es un token
    numbers = list(map(int, data[1:n + 1])) # lista con
        los n tokens siguientes [1, n+1)


    # Escribir: muchos print() son lentos. Acumular y
        escribir una sola vez
    out = []
    out.append(str(n)) # Cada elemento es una linea de
        salida
    sys.stdout.write("\n".join(out) + "\n") # Imprime
        todo, cada elemento en una linea diferente

    print(*numbers) # Imprime los elementos separados
        por espacio
    print("\n".join(map(str, numbers))) # Un elemento
        por linea


    # Tamano de una cadena
    x = len(s)

    # Agregar un caracter a una cadena
    # Las cadenas son inmutables asi que al agregarle un
        caracter se crea una cadena nueva, lo cual es
        costoso
    # Es mejor manejarlas con listas de char

    # Negar un bool
    cond = True
    not cond # Esto seria False

    # Potencia de un numero
    ans = base ** exp
    ans = pow(base, exp, mod) # Exponenciacion binaria
    inv = pow(num, -1, mod) # Calcula el inverso modular
        de num, siempre que num y mod sean coprimos (
        Desde Python 3.8)

    # Ciclo for
    for i in range (0, 5): # Esto iria hasta el 4
    for auto in autos: # autos seria una lista

    # Condicionales
    if(cond1):
    elif(cond2):
    else:

    # Minimo y maximo
    mina = min(mina, number)
    maxa = max(maxa, number)

    # Para usar decimales precisos
    from decimal import Decimal, getcontext

    getcontext().prec = 70 # Cantidad de digitos
        decimales a usar
    ans = Decimal("0.0") # Inicializar un numero decimal
    val += Decimal("1") # Para constantes usar Decimal
        tambien
    val += Decimal("1") / r # Se puede dividir por un
        entero sin problema
    print(f"{ans:.6f}") # Imprimir con 6 decimales


    # Convertir un numero base x (guardado como cadena)
        en un entero base 10
    n10 = int(n2, x)

    # Pasar un numero de base 10 a base 2 (se convierte
        a cadena) (falla con negativos)
    ans2 = bin(ans10)[2:] # Para cambiar de base 10 a
        cualquier otra base (que no sea base 2) toca a
        mano

    # Evaluar una string que es una expresion matematica
        (tambien puede tener parentesis)
    result = eval(expresion) # Toma ^ como XOR, ** como
        potencia, / da float, // es division parte
        entera

    # Metodos de listas (list)
    # Resaltar que una sola lista puede contener varios
        tipos de datos, incluida otra lista
    lista = [1, "dos", [3, "cuatro"], 5.0, True] #
        Declarar
    exp = [val] * n # Crear una lista de n elementos
        iguales a val
    n = lista[i] # Sirve para acceder al elemento i de
        una lista
    lista.append(6) # Agregar un elemento
    lista.remove("dos") # Eliminar la primera aparicion
        de "dos", ValueError si no esta
    lista.sort() # Ordenar (todos los elementos tienen
        que ser comparables entre si (ej: int con str
        falla), sino da TypeError)
    print(lista) # Imprimir toda la lista
    lista.extend([7, 8])  # Agrega 7 y 8 a la lista
    lista.insert(1, 'a') # Inserto 'a' en la posicion 1,
         el resto de elementos corren un espacio a la
        derecha
    n = lista.pop() # Elimina el ultimo elemento de la
        lista y se lo asigna a 'n'
    n = lista.pop(3) # Elimina el elemento en la
        posicion 3 de la lista y se lo asigna a 'n'
    lista.clear() # Elimina todos los elementos de la
        lista, quedando: lista = []
    lista.reverse() # Invierte los elementos de la lista
    indice = lista.index("dos") # Devuelve el indice de
        la primera aparicion de "dos" (lanza ValueError
         si no esta)
    indice2 = lista.index(x, start) # Comienza a buscar
        desde el indice start
    indice3 = lista.index(x, start, end) # Devuelve la
        primera aparicion de x en el rango [start, end)
    conteo = lista.count(x) # Devuelve el numero de
        veces que aparece 'x' en la lista


    # Metodos deque
    from collections import deque
    # list.pop(0) y list.insert(0, x) son O(n). Para
        colas (BFS) usar deque
    d = deque() # Declarar vacio
    d = deque([1, 2, 3]) # Declarar desde una lista
    d.append(4) # Agrega a la derecha: [1, 2, 3, 4]
    d.appendleft(0) # Agrega a la izquierda: [0, 1, 2,
        3, 4]
    x = d.pop() # Elimina y retorna el de la derecha
    x = d.popleft() # Elimina y retorna el de la
        izquierda
    # pop() y popleft() lanzan IndexError si el deque
        esta vacio
    x = d[0] # Primer elemento (O(1))
    x = d[-1] # Ultimo elemento (O(1)). Acceder a
        posiciones del medio es O(n)
    d.extend([5, 6]) # Agrega varios a la derecha
    d.extendleft([-1, -2]) # Agrega a la izquierda uno
        por uno, asi que queda en orden inverso: [-2,
```

```
    -1, ...]
d.rotate(1) # Rota 1 posicion a la derecha. Con -1
    rota a la izquierda
d.reverse() # Invierte el deque
d.remove(5) # Elimina la primera aparicion (
    ValueError si no esta)
n = d.count(5) # Cuenta cuantas veces aparece
n = len(d) # Tamano
d.clear() # Vacia el deque
if d: # Un deque vacio es False, uno con elementos
    es True
    pass
d = deque(maxlen=3) # Con maxlen, al agregar de mas
    se elimina del extremo opuesto


# heapq (cola de prioridad). Es un min-heap
    implementado sobre una lista normal
import heapq
h = [] # Declarar vacio
heapq.heappush(h, 5) # Agregar un elemento. O(log n)
heapq.heappush(h, 2)
x = heapq.heappop(h) # Elimina y retorna el MENOR. O
    (log n). IndexError si esta vacio
x = h[0] # Ver el menor sin eliminarlo. O(1)
heapq.heapify(lista) # Convierte una lista en heap,
    en el mismo lugar. O(n)
x = heapq.heappushpop(h, 3) # Agrega 3 y luego
    elimina y retorna el menor (puede ser el mismo
    3) (mas eficiente que por separado)
x = heapq.heapreplace(h, 3) # Elimina y retorna el
    menor y luego agrega 3
mayores = heapq.nlargest(3, lista) # Los 3 mayores,
    de mayor a menor
menores = heapq.nsmallest(3, lista) # Los 3 menores,
     de menor a mayor
# Imprimir la lista del heap NO la muestra ordenada,
    solo se garantiza que h[0] es el menor
# Con tuplas se compara el primer elemento y, si
    empatan, el segundo, y asi
heapq.heappush(h, (dist, nodo)) # Prioridad por
    distancia
dist, nodo = heapq.heappop(h)

# Metodos de sets
# No tiene elementos repetidos, no esta ordenado
s = {1, 2, 3} # Declarar
s.add(4) # Agregar un elemento
s.remove(4) # Eliminar un elemento. Si el elemento
    no esta lanza un error
s.discard(4) # Tambien elimina un elemento pero si
    no esta no genera error
x = s.pop() # Elimina y retorna un elemento
    arbitrario
s.clear() # Vacia el conjunto
s1 = {1, 2}
s2 = {2, 3}
s3 = {4, 5, 6}

# Metodos de operaciones de conjuntos
# Union de conjuntos
s4 = s1.union(s2, s3)
s4 = s1 | s2 | s3

# Interseccion de conjuntos
s4 = s1.intersection(s2, s3)
s4 = s1 & s2 & s3

# Retorna lo que esta en el primer conjunto pero no
    en los demas
s4 = s1.difference(s2, s3)
s4 = s1 - s2 - s3

# Elementos que estan en un numero impar de
    conjuntos
s4 = s1.symmetric_difference(s2).
    symmetric_difference(s3)
s4 = (s1 ^ s2) ^ s3

# Metodos para relaciones de conjuntos

s1.issubset(s2) # Devuelve True si s1 es subconjunto
    de s2
s1.issuperset(s2) # Devuelve True si s1 es
    superconjunto de s2
s1.isdisjoint(s2) # Devuelve True si no tienen
    elementos en comun

# Metodos de actualizacion

# Actualiza s1 con la union entre s1 y s2
s1.update(s2)
s1 |= s2

# Actualiza s1 con la interseccion entre s1 y s2
s1.intersection_update(s2)
s1 &= s2

# Actualiza s1 eliminando los elementos que estan en
    s2
s1.difference_update(s2)
s1 -= s2

# Actualiza s1 con la diferencia simetrica entre s1
    y s2
s1.symmetric_difference_update(s2)
s1 ^= s2


# Diccionarios
dic = {'a' : 1, 'b' : 2} # Declarar
n = dic.get('a') # Acceder al value de la key 'a',
    sino esta devuelve None
n = dic.get('a', 0) # Si la key 'a' esta, devuelve
    su value, sino devuelve 0
claves = list(dic.keys()) # Retorna una lista con
    todas las keys
valores = list(dic.values()) # Retorna una lista con
    todos los values
parejas = list(dic.items()) # Retorna una lista de
    tuplas con todos los pares key-value
dic.update({'c' : 3, 'd' : 4}) # Insertar elementos
    en dic
dic.update(dic2) # Actualizar dic con los elementos
    de dic2
a = dic.pop('a') # Elimina y retorna el value cuya
    key es 'a', sino esta genera error
a = dic.pop('a', 0) # Elimina y retorna el value
    cuya key es 'a', sino esta devuelve 0
a = dic.popitem() # Elimina y devuelve el ultimo par
     key-value que fue insertado
dic.clear() # Vacia el diccionario
valor = dic.setdefault('a') # Si esta esa key,
    devuelve su value
valor = dic.setdefault('a', 2) # Si esta esa key,
    devuelve su valor e ignora el 2
value = dic.setdefault('e', 5) # Si no esta esa key,
     la agrega con ese value y retorna el value
keys = ['a', 'b', 'c']
dic = dict.fromkeys(keys, 0) # Crea un nuevo
    diccionario con claves del iterable y todas con
     el mismo valor


# bisect (busqueda binaria). La lista DEBE estar
    ordenada. O(log n)
from bisect import bisect_left, bisect_right
a = [1, 2, 4, 4, 4, 7]
i = bisect_left(a, 4) # 2: primera posicion con
    elemento >= 4 (retorna len(a) si no hay ninguno
    )
i = bisect_right(a, 4) # 5: primera posicion con
    elemento > 4 (justo despues de la ultima
    aparicion)
cantidad = bisect_right(a, 4) - bisect_left(a, 4) #
    3: cuantas veces aparece el 4
i = bisect_left(a, 4, lo, hi) # Buscar solo en el
    rango [lo, hi)

main()
```

10 C++

10.1 Bitset

```
/*
- Cuando se requiere un vector de bool, un bitset logra
    la misma funcion
  pero mucho mas eficiente ya que se pueden usar
      operaciones de bits.
- Internamente se hacen estas operaciones en bloques de
    64 (o 32,
  dependiendo de la arquitectura), asi que la
      complejidad de lo que sea
  que se este haciendo se ve reducida en un vector de
      64. Por ejemplo en
  un knapsack donde solo quiero saber si puedo lograr
      cierto valor, la
  complejidad pasaria de ser O(n^2) a ser O(n^2 / 64)
*/

bitset<MAXN> b; // MAXN debe ser una constante
b.test(pos), b[pos] // Retorna si el bit pos es 1
b[pos1] = 1, b[pos2] = 0;
b.set() // Pone todos los bits en 1
b.reset() // Pone todos los bits en 0
b.flip() // Invierte todos los bits
b.flip(pos) // Invierte el valor del bit
b.any() // Pregunta si hay algun bit en 1
b.none() // Pregunta si todos son 0
b.all() // Pregunta si todos son 1
b.count() // Cuenta la cantidad de bits en 1
string s = b.to_string();
llu x = b.to_tllong(); // Lo convierte en un entero (
    solo si cabe)
sz(b); // Retorna la longitud

int i = b._Find_first(); // Retorna el indice del primer
     bit en 1
i = b._Find_next(i); // Retorna el indice del primer bit
     en 1 DESPUES de i
// Si no existe, retorna size

// Operaciones de bits
b1 | b2;
b1 ^ b2;
b1 & b2;
b <<= cnt; // Modifica cada bit i como bit i - cnt (
    Desplazamiento a la derecha)
b |= (b << add); // Hace que para bit i en 1, el bit i +
     add tambien sea 1
```

10.2 Enteros de 128 bits

```
// Tomado del template de Jiangly
using u128 = unsigned __int128;
using i128 = __int128;

// - Solo lo soportan algunos compiladores
// - Solo disponible en arquitecturas de 64 bits (como
    x86_64)
// - MSVC (Microsoft Visual C++) no lo soporta
    directamente
```

```
// - En CF se puede desde C++ 20 (la 17 es arquitectura
    de 32 bits)

// Convertir en cadena para imprimir
i128 x;
string s = "";
while(x > 0){
    s += '0' + (x % 10);
    x /= 10;
}
reverse(s.begin(), s.end());
```

10.3 Imprimir decimales

```
cout<<fixed<< setprecision(cantidad_decimales)<<ans<<
    endl;
```

10.4 Deque

```
deque<int> dq;
dq.push_front(1);
dq.push_back(10);
deque<int>::iterator it = dq.begin();
it++;
dq.insert(it, 5); // O(n)
it = dq.end();

int n = dq.front(); //1
int m = dq.back(); //10
int x = dq.at(1); //5

dq.pop_front();
dq.pop_back();
it = dq.begin();
dq.erase(it); // O(n)

int len = dq.size();

dq.clear();
```

10.5 Indexed set

```
// Set indexado
// Sin elementos repetidos
// Se puede simular elementos repetidos usando pii

#include<ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
using indexed_set = tree<int, null_type, less<int>,
    rb_tree_tag, tree_order_statistics_node_update>;
// Para ordenar por MAX cambiar less<int> por greater<
    int>
// Para multiset cambiar less<int> por less_equal<int>

indexed_set nombre; // Declaracion
nombre.insert(valor); // Insertar
nombre.size() // Numero de elementos
nombre.find_by_order(3) // Iterador al 4to elemento mas
    pequeno
nombre.order_of_key(6) // Cuenta la cantidad de
    elementos menores a 6
nombre.erase(valor) // Si lo encuentra lo elimina, sino
    no hace nada
auto it = nombre.find(valor) // Devuelve un iterador a
    ese elemento. Si no esta, devuelve un iterador al
    final
nombre.count(valor) // Retorna un entero, 1 si valor
    esta, 0 si no esta
nombre.swap(nombre2); // Swapea 2 indexed set en O(1).
    El swap normal (swap(nombre, nombre2)) loa hace en
    o(n)
nombre.empty()
nombre.clear()
auto it = nombre.begin()
auto it = nombre.end()
auto it = nombre.lower_bound(5) // Retorna un iterador
    al primer elemento mayor o igual a 5
auto it = nombre.upper_bound(5) // Retorna un iterador
    al primer elemento mayor a 5
```

10.6 Listas

```
/* A partir de estas funciones se pueden simular listas
    ascendentes,
   circulares, enlazadas y doblemente enlazadas */

list <int>::iterator next(list <int> &lista, list <int
    >::iterator i){
    i++;
    if(i == lista.end())
        i = lista.begin();
    return i;
}

list <int>::iterator prev(list <int> &lista, list <int
    >::iterator i){
    if (i == lista.begin())
        i = lista.end();
    i--;
    return i;
}

list <int> lista;
lista.push_back(1);
int len = lista.size();
list<int>::iterator it1 = lista.begin();
list<int>::iterator it2 = lista.end();
lista.erase(it1);
list<int>::iterator s1 = next(lista, s1);
list<int>::iterator s2 = prev(lista, s1);
```

10.7 Map

```
// Como acceder a los elementos de un mapa
map<int, int> T;

// Recorrer por valor o referencia
T[key]; // Accede al valor de key
for(const auto& par : T){
    key = par.fi;
    valor = par.se;
}

// Por puntero o iterador
auto it = T.find(key);
if(it != T.end()){
    key = it->fi;
    valor = ir->se
}

// lower_bound y upper_bound en set y map
auto it = T.lower_bound(key);
auto it = T.upper_bound(key);
```

10.8 Metodos de ordenamiento

```
// En este caso se ordena revisando cual de los second
    es menor y si son iguales
// se revisa cual de los first es menor
struct comparator {
    bool operator () (pii a, pii b){
        if(a.se == b.se) return a.fi < b.fi;
        return a.se < b.se;
    }
};

// Usar en:
sort(all(vec), comparator());
pqueue<pii, vector<pii>, comparator> pq;
set<pii, comparator> mySet;
```

10.9 Permutaciones

```
// Entrego una permutacion y me devuelve la siguiente o
    la anterior.
// Cuando ya no hay mas, devuelve false.
// Complejidad por llamado O(n)
next_permutation(all(perm)); // Si quiero verlas todas,
    ordenar ascendente
prev_permutation(all(perm)); // Si quiero verlas todas,
    ordenar descendente

// Complejidad: O(n*n!)
do{
    // Process
} while(next_permutation(all(perm)));
```

11 Strings

11.1 Hashing

```
// Algunas notas:
// - La probabilidad de que 2 cadenas tengan el mismo
    hashing es de n / m
//   siendo n la longitud de la cadena mas larga
// - Si tengo un conjunto de hashes de k cadenas con
    longitud de maximo n,
//   la probabilidad de que 2 cadenas tengan igual hash
    es de (k^2 * n) / m
//   Por ende, si k y n son de alrededor de 1e5 y m
    alrededor de 1e9, es
//   casi seguro que habra colisiones. Una forma de
    solucionarlo puede ser
//   un primo m mas largo, como 2^61 - 1
    (2305843009213693951) y usar
//   enteros de 128 bits para las multiplicaciones,
    aunque ahora el hashing
//   y las queries serian un poco mas lentas. (Para
    hacerlo puedo simplemente
//   poner using ll = __int128; y cambiar el modulo).
// - Al usar doble modulo se estan reduciendo esas
    probabilidades considerablemente

// Hashing con doble modulo
// En caso de necesitar otro modulo
// 1000234999, 1000567999, 1000111997, 1000777121,
    999727999, 1070777777
const int MOD[] = { 1001864327, 1001265673 }, N = 1e6 +
    10;
const pii BASE(257, 367), ZERO(0, 0), ONE(1, 1);
inline int add(int a, int b, int mod) { return a+b >=
    mod ? a+b-mod : a+b; }
inline int sbt(int a, int b, int mod) { return a-b < 0 ?
    a-b+mod : a-b; }
inline int mul(int a, int b, int mod) { return ll(a) * b
    % mod;}
inline ll operator ! (const pii a) { return (ll(a.fi) <<
    32) | a.se; }
inline pii operator + (const pii& a, const pii& b) {
  return {add(a.fi, b.fi, MOD[0]), add(a.se, b.se, MOD
      [1])};}
inline pii operator - (const pii& a, const pii& b) {
  return {sbt(a.fi, b.fi, MOD[0]), sbt(a.se, b.se, MOD
      [1])};}
inline pii operator * (const pii& a, const pii& b) {
  return {mul(a.fi, b.fi, MOD[0]), mul(a.se, b.se, MOD
      [1])};}

pii base[N]{ONE};
void prepare() { for1(i, N-1) base[i] = base[i-1] * BASE
```

```
    ; }
template <class type>
struct hashing {    /// HACELEEE PREPAREEEE en el main!!!
  vii ha;        // ha[i] = t[i]*p0 + t[i+1]*p1 + t[i+2]*
      p2 + ..
  hashing(type &t): ha(sz(t)+1, ZERO){ // O(n)
    for(int i = sz(t) - 1; i >= 0; --i) ha[i] = ha[i+1]
        * BASE + pii{t[i], t[i]};
  }
  pii query(int l, int r){ return ha[l] - ha[r+1] * base
      [r-l+1]; } // [l,r], O(1)
};
```

11.2 Minimal Rotation

```
void solve(){
    string s; cin>>s;
    s += s;
    int n = sz(s);
    vi fail(n, -1);

    int minRot=0;
    for1(i,n-1){
        char c = s[i];
        int j = fail[i-minRot-1];

        while(j != -1 && c != s[minRot+j+1]){
            if(c < s[minRot+j+1]) minRot = i-j-1;
            j = fail[j];
        }

        if(c != s[minRot+j+1]){
            if(c < s[minRot]) minRot = i;
            fail[i-minRot] = -1;
        }
        else{
            fail[i-minRot] = j+1;
        }
    }

    cout<<s.substr(minRot, n/2)<<endl;
}
```

11.3 String Matching

```
/* Algunas notas:
Este algortimo sirve para encontrar todos los prefijos
    en un array
algo de caja negra, haga un pull request de expandir
    esta info lol
aun tratando de entender el KMP completamente, by felipe
    sanchez
*/

void solve(){
    string s, t;
    cin >> s >> t;
    int m = sz(t);

    s = t + "#" + s;
    // This make the Pattern Search KMP array
    int n = sz(s);
    vi lps(n);
    for1(i,n-1){
        int j = lps[i-1];
        while(j > 0 && s[i] != s[j]) j = lps[j-1];
        if(s[i] == s[j]) j++;
        lps[i] = j;
    }

/* Esto serviria si buscaramos un patron de string t en
    s.
    int ans=0;
    for(int i = m; i < n; i++){
        ans += (lps[i] == m);
    }
    cout<< ans << "\n";
*/
    return;
}
```

11.4 Trie

```
/*
- Construye un automata finito determinista que permite
    buscar
  patrones en un conjunto de palabras.
- Complejidad O(m*k), siendo 'm' la suma de longitudes
    de todas las
  cadenas y 'k' el numero de caracteres en el alfabeto.
*/

const int K = 26; // Cantidad de caracteres

// Vertices del trie
struct Vertex{
    int next[K]; // Vertice al que se llegaa siguiendo
        el caracter i, -1 si ninguno
    bool output = 0; // 1 si el vertice corresponde a un
        array en el diccionario

    Vertex(){
        fill(begin(next), end(next), -1);
    }
};

vector<Vertex> trie(1); // Se guarda el trie como un
    array de Vertex

// Agregar string al trie
// Complejidad: O(sz(s))
void add_string(string const& s){
    int v = 0; // Se inicia en el nodo raiz
    for(char ch : s){ // Se recorre el trie con la
        palabra a agregar
        int c = ch - 'a'; // Se toma 'a' como primer
            caracter del alfabeto
        if(trie[v].next[c] == -1){ // Si no hay un
            vertice correspondiendo al siguiente
            caracter, se agrega
            trie[v].next[c] = sz(trie);
            trie.emplace_back();
        }
        v = trie[v].next[c]; // Se avanza al siguiente
            vertice
    }
    trie[v].output = 1; // Marca la flag del vertice de
        salida
}
```

12 Geometria

12.1 Geo Struct

```
////////////////////////////////////////////////
// TEMPLATE DE GEOMETRIA 2D (C++17)
//
//  1. Basicos ............ EPS, sgn, Point/Vector,
    distancias, orient
//  2. Rectas y segmentos .. proyeccion, distancias,
    interseccion de rectas y segmentos, struct Segment
//  3. Circulos ............ struct Circle,
    intersecciones, tangentes, areas, circulo minimo
//  4. Triangulos ......... struct Triangle, puntos
    notables, colision de triangulos
//  5. Poligonos ........... area, punto en poligono,
    convex hull, semiplanos, Minkowski, etc.
//  6. Esfera .............. distancia sobre una esfera
    (haversine)
//
// Convencion: positivo = sentido antihorario (CCW).
    Funciones con "template<T>" sirven con
// ld (flotante) y ll (entero). Las demas trabajan solo
    con pt (= Point<ld>).
////////////////////////////////////////////////

////////////////////////////////////////////////
// 1. BASICOS
////////////////////////////////////////////////

const ld PI = acosl(-1);

// Tolerancia: 1e-9 para flotantes, 0 para enteros (asi
    el mismo codigo sirve para ambos)
template<typename T>
constexpr T EPS = is_floating_point_v<T> ? 1e-9 : 0;

// Signo de x respetando la tolerancia: devuelve 1, 0 o
    -1
template<typename T>
int sgn(T x) {
    return (x > EPS<T>) - (x < -EPS<T>);
}

template<typename T = ld>
struct Point {
    T x, y;

    Point() : x(0), y(0) {}
    Point(T _x, T _y) : x(_x), y(_y) {}

    // Operadores aritmeticos
    Point operator+(const Point& p) const { return Point
        (x + p.x, y + p.y); }
    Point operator-(const Point& p) const { return Point
        (x - p.x, y - p.y); }
    Point operator*(T d) const { return Point(x * d, y *
        d); }
    Point operator/(T d) const { return Point(x / d, y /
        d); }

    Point& operator+=(const Point& p) { x += p.x; y += p
        .y; return *this; }
    Point& operator-=(const Point& p) { x -= p.x; y -= p
        .y; return *this; }
    Point& operator*=(T d) { x *= d; y *= d; return *
        this; }
    Point& operator/=(T d) { x /= d; y /= d; return *
        this; }

    // Orden EXACTO (por x y luego y): seguro para sort,
        set y map
    bool operator<(const Point& p) const { return tie(x,
        y) < tie(p.x, p.y); }

    // Igualdad con tolerancia (EPS) en flotantes
    bool operator==(const Point& p) const { return sgn(x
        - p.x) == 0 && sgn(y - p.y) == 0; }
    bool operator!=(const Point& p) const { return !(*
        this == p); }

    // Producto punto: a . b = |a||b|cos(theta)       (
        atajo: a * b)
    T dot(const Point& p) const { return x * p.x + y * p
        .y; }
    T operator*(const Point& p) const { return dot(p); }

    // Producto cruz 2D: a x b = |a||b|sin(theta)     (
        atajo: a % b)
    T cross(const Point& p) const { return x * p.y - y *
        p.x; }
    T operator%(const Point& p) const { return cross(p);
        }

    // Norma al cuadrado (exacta en enteros) y longitud
```

```
    T sq() const { return x * x + y * y; }
    ld len() const { return sqrtl((ld)sq()); }

    // Vector unitario (misma direccion, largo 1)
    Point<ld> unit() const {
        ld l = len();
        return Point<ld>(x / l, y / l);
    }

    // Vector perpendicular (rotado 90 grados
        antihorario)
    Point perp() const { return Point(-y, x); }

    // Angulo polar del vector, en [-PI, PI]
    ld angle() const { return atan2l(y, x); }

    // Rotacion antihoraria por theta radianes
    Point<ld> rotate(ld theta) const {
        return Point<ld>(x * cosl(theta) - y * sinl(
            theta),
                         x * sinl(theta) + y * cosl(
                             theta));
    }

    friend istream& operator>>(istream& is, Point& p) {
        return is >> p.x >> p.y; }
    friend ostream& operator<<(ostream& os, const Point&
         p) { return os << p.x << " " << p.y; }
};

template<typename T = ld>
using Vector = Point<T>;

using pt = Point<ld>;
using ptll = Point<ll>;
using vec = Vector<ld>;
using vecll = Vector<ll>;

// Distancia al cuadrado entre dos puntos (exacta con
    enteros)
template<typename T>
T dist2(const Point<T>& a, const Point<T>& b) {
    return (a - b).sq();
}

// Distancia euclidiana entre dos puntos
template<typename T>
ld dist(const Point<T>& a, const Point<T>& b) {
    return (a - b).len();
}

// Orientacion de c respecto a la recta dirigida a->b: (
    b - a) x (c - a)
//   > 0: c a la izquierda (giro CCW)   < 0: a la
    derecha (CW)   = 0: colineales
template<typename T>
T orient(const Point<T>& a, const Point<T>& b, const
    Point<T>& c) {
    return (b - a).cross(c - a);
}

// Angulo entre dos vectores, en [0, PI]
ld angle_between(const pt& a, const pt& b) {
    return atan2l(fabsl(a.cross(b)), a.dot(b));
}

// acos protegido contra errores numericos (argumento
    fuera de [-1, 1])
ld safe_acos(ld x) {
    return acosl(max((ld)-1, min((ld)1, x)));
}

////////////////////////////////////////////
// 2. RECTAS Y SEGMENTOS
//    Un segmento se representa con Segment (extremos a
    y b). Cuando una funcion habla de una
//    RECTA tambien recibe un Segment: la recta infinita
     que pasa por sus dos extremos.
////////////////////////////////////////////

// Declaraciones adelantadas (las definiciones estan mas
    abajo)
template<typename T> struct Segment;
template<typename T> bool on_segment(const Point<T>& p,
    const Segment<T>& s);
template<typename T> bool segments_intersect(const
    Segment<T>& s, const Segment<T>& t);
template<typename T> vector<pt>
    segment_intersection_points(const Segment<T>& s,
    const Segment<T>& t);
template<typename T> ld dist_point_segment(const Point<T
    >& p, const Segment<T>& s);

template<typename T = ld>
struct Segment {
    Point<T> a, b;   // extremo inicial y final

    Segment() : a(), b() {}
    Segment(Point<T> _a, Point<T> _b) : a(_a), b(_b) {}
    Segment(T x1, T y1, T x2, T y2) : a(x1, y1), b(x2,
        y2) {}

    // Vector direccion a -> b
    Vector<T> dir() const { return b - a; }

    // Longitud (y su cuadrado, exacto con enteros)
    ld length() const { return dist(a, b); }
    T length2() const { return dist2(a, b); }

    // Punto medio
    Point<ld> midpoint() const {
        return Point<ld>((ld)(a.x + b.x) / 2, (ld)(a.y +
            b.y) / 2);
    }

    // Lado en que cae p respecto a la recta a->b: 1
        izquierda, -1 derecha, 0 sobre la recta
    int side(const Point<T>& p) const { return sgn(
        orient(a, b, p)); }

    // Atajos a las funciones de abajo (se pueden usar
        de cualquiera de las dos formas)
    bool contains(const Point<T>& p) const { return
        on_segment(p, *this); }
    bool intersects(const Segment& s) const { return
        segments_intersect(*this, s); }
    vector<pt> intersection_points(const Segment& s)
        const { return segment_intersection_points(*
        this, s); }
    ld dist_to(const Point<T>& p) const { return
        dist_point_segment(p, *this); }

    friend istream& operator>>(istream& is, Segment& s)
        { return is >> s.a >> s.b; }
    friend ostream& operator<<(ostream& os, const
        Segment& s) { return os << "[" << s.a << "] ->
        [" << s.b << "]"; }
};

using seg = Segment<ld>;
using segll = Segment<ll>;

// Proyeccion del vector v sobre el vector u
pt proj_vec(const pt& v, const pt& u) {
    return u * (v.dot(u) / u.sq());
}

// Proyeccion del punto p sobre la recta l (pie de la
    perpendicular)
pt proj_point_line(const pt& p, const seg& l) {
    pt ab = l.dir();
    return l.a + ab * ((p - l.a).dot(ab) / ab.sq());
}

// Reflexion (simetrico) del punto p respecto a la recta
     l
pt reflect_point_line(const pt& p, const seg& l) {
    return proj_point_line(p, l) * 2 - p;
}

// Distancia de p a la recta infinita l
ld dist_point_line(const pt& p, const seg& l) {
    return fabsl(orient(l.a, l.b, p)) / l.length();
}

// Distancia de p al segmento cerrado s (funciona aunque
    s sea un punto)
template<typename T>
ld dist_point_segment(const Point<T>& p, const Segment<T
    >& s) {
    pt q(p.x, p.y), a(s.a.x, s.a.y), b(s.b.x, s.b.y);
    pt ab = b - a;
    ld t = ab.dot(q - a);
    if (t <= 0) return dist(q, a);              // el
        punto mas cercano es el extremo a
    if (t >= ab.sq()) return dist(q, b);        // el
        punto mas cercano es el extremo b
    return dist_point_line(q, seg(a, b));       // cae
        dentro del segmento
}

// Verifica si p pertenece al segmento cerrado s (
    colineal y entre los extremos)
template<typename T>
bool on_segment(const Point<T>& p, const Segment<T>& s)
    {
    return sgn(orient(s.a, s.b, p)) == 0 && sgn((s.a - p
        ).dot(s.b - p)) <= 0;
}

// Verifica si los segmentos cerrados s y t tienen al
    menos un punto en comun
template<typename T>
bool segments_intersect(const Segment<T>& s, const
    Segment<T>& t) {
    int o1 = sgn(orient(s.a, s.b, t.a));
    int o2 = sgn(orient(s.a, s.b, t.b));
    int o3 = sgn(orient(t.a, t.b, s.a));
    int o4 = sgn(orient(t.a, t.b, s.b));

    // Cruce propio: los extremos de cada segmento
        quedan a lados opuestos del otro
    if (o1 * o2 < 0 && o3 * o4 < 0) return true;

    // Casos de borde: un extremo toca al otro segmento
        (incluye colineales)
    return on_segment(t.a, s) || on_segment(t.b, s) ||
           on_segment(s.a, t) || on_segment(s.b, t);
}

// Puntos de interseccion de los segmentos s y t:
//   0 puntos (no se tocan), 1 punto, o 2 puntos (
    segmentos colineales solapados: extremos del solape
    )
template<typename T>
vector<pt> segment_intersection_points(const Segment<T>&
     s, const Segment<T>& t) {
    pt a(s.a.x, s.a.y), b(s.b.x, s.b.y), c(t.a.x, t.a.y)
        , d(t.b.x, t.b.y);
    seg ab(a, b), cd(c, d);

    ld oa = orient(c, d, a), ob = orient(c, d, b);
    ld oc = orient(a, b, c), od = orient(a, b, d);

    // Cruce propio: un unico punto, se calcula por
        proporcion de las distancias con signo
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        {
        return {(a * ob - b * oa) / (ob - oa)};
    }
```

```
    // Caso degenerado: recolectar extremos que caen
        sobre el otro segmento
    vector<pt> res;
    if (on_segment(c, ab)) res.push_back(c);
    if (on_segment(d, ab)) res.push_back(d);
    if (on_segment(a, cd)) res.push_back(a);
    if (on_segment(b, cd)) res.push_back(b);
    sort(res.begin(), res.end());
    res.erase(unique(res.begin(), res.end()), res.end())
        ;
    return res;
}

// Interseccion de las RECTAS l1 y l2. Devuelve false si
     son paralelas; si no, guarda el punto en res
bool intersect_lines(const seg& l1, const seg& l2, pt&
    res) {
    ld cp = l1.dir().cross(l2.dir());
    if (sgn(cp) == 0) return false;
    ld t = (l2.a - l1.a).cross(l2.dir()) / cp;
    res = l1.a + l1.dir() * t;
    return true;
}

// Distancia minima entre dos segmentos (0 si se tocan)
ld dist_segment_segment(const seg& s, const seg& t) {
    if (s.intersects(t)) return 0;
    // Si no se cruzan, la distancia minima siempre
        ocurre en algun extremo
    return min(min(s.dist_to(t.a), s.dist_to(t.b)),
               min(t.dist_to(s.a), t.dist_to(s.b)));
}

////////////////////////////////////////////
// 3. CIRCULOS
////////////////////////////////////////////

struct Circle {
    pt o;   // centro
    ld r;   // radio

    Circle() : o(pt(0, 0)), r(0) {}
    Circle(pt _o, ld _r) : o(_o), r(_r) {}
    Circle(ld x, ld y, ld _r) : o(pt(x, y)), r(_r) {}

    // Posicion de un punto: 1 (dentro), 0 (en el borde)
        , -1 (fuera)
    int contains(const pt& p) const { return sgn(r -
        dist(o, p)); }

    ld area() const { return PI * r * r; }
                                    // pi * r^2
    ld perimeter() const { return 2 * PI * r; }
                                    // 2 * pi * r
    ld sector_area(ld theta) const { return 0.5 * r * r
        * theta; }          // sector de angulo theta
    ld segment_area(ld theta) const { return 0.5 * r * r
         * (theta - sinl(theta)); } // segmento
        circular
};

// Interseccion RECTA l - circulo: devuelve 0, 1 o 2
    puntos
vector<pt> intersect_circle_line(const Circle& c, const
    seg& l) {
    pt p = proj_point_line(c.o, l);             // pie
        de la perpendicular desde el centro
    ld h2 = c.r * c.r - (p - c.o).sq();         // (
        semicuerda)^2
    if (sgn(h2) < 0) return {};                 // la
        recta no toca el circulo
    if (sgn(h2) == 0) return {p};               //
        tangente
    pt u = l.dir().unit() * sqrtl(h2);
    return {p - u, p + u};
}

// Interseccion SEGMENTO - circulo: devuelve 0, 1 o 2
    puntos (solo los que caen dentro del segmento)
vector<pt> intersect_circle_segment(const Circle& c,
    const seg& s) {
    vector<pt> res;
    for (auto& p : intersect_circle_line(c, s)) {
        if (s.contains(p)) res.push_back(p);
    }
    return res;
}

// Interseccion circulo - circulo: devuelve 0, 1 o 2
    puntos (vacio si son concentricos)
vector<pt> intersect_circle_circle(const Circle& c1,
    const Circle& c2) {
    pt d = c2.o - c1.o;
    ld D = d.len();
    if (sgn(D) == 0) return {};
                                        // concentricos
    if (sgn(D - c1.r - c2.r) > 0) return {};
                        // muy lejos
    if (sgn(D - fabsl(c1.r - c2.r)) < 0) return {};
                // uno dentro del otro

    ld a = (c1.r * c1.r - c2.r * c2.r + D * D) / (2 * D)
        ;   // distancia de c1.o a la cuerda comun
    ld h2 = c1.r * c1.r - a * a;
    pt mid = c1.o + d * (a / D);
                                    // punto medio de la
         cuerda comun
    if (sgn(h2) <= 0) return {mid};
                                   // tangentes

    pt off = d.perp() / D * sqrtl(h2);
    return {mid + off, mid - off};
}

// Area de la interseccion (lente) de dos circulos
ld circle_circle_intersection_area(const Circle& c1,
    const Circle& c2) {
    ld d = dist(c1.o, c2.o);
    if (d >= c1.r + c2.r) return 0;
                                   // disjuntos
    if (d <= fabsl(c1.r - c2.r)) {
                                    // uno contiene al
        otro
        ld r = min(c1.r, c2.r);
        return PI * r * r;
    }
    ld a = safe_acos((d * d + c1.r * c1.r - c2.r * c2.r)
        / (2 * d * c1.r));  // semiangulo en c1
    ld b = safe_acos((d * d + c2.r * c2.r - c1.r * c1.r)
        / (2 * d * c2.r));  // semiangulo en c2
    return c1.r * c1.r * (a - sinl(2 * a) / 2) + c2.r *
        c2.r * (b - sinl(2 * b) / 2);
}

// Tangentes comunes a dos circulos. Cada tangente es un
     segmento {a = punto de contacto en c1, b = punto
    de contacto en c2}.
//   inner = false: tangentes EXTERNAS (hasta 2)
    inner = true: tangentes INTERNAS (hasta 2)
vector<seg> tangents_circle_circle(const Circle& c1,
    const Circle& c2, bool inner = false) {
    ld r2 = inner ? -c2.r : c2.r;       // truco: radio
        negativo da las tangentes internas
    pt d = c2.o - c1.o;
    ld dr = c1.r - r2;
    ld d2 = d.sq();
    ld h2 = d2 - dr * dr;
    if (sgn(d2) == 0 || sgn(h2) < 0) return {};

    vector<seg> res;
    for (ld s : {-1, 1}) {
        pt v = (d * dr + d.perp() * sqrtl(max((ld)0, h2)
            ) * s) / d2;
        res.push_back(seg(c1.o + v * c1.r, c2.o + v * r2
            ));
    }
    if (sgn(h2) == 0) res.pop_back();   // las dos
        tangentes coinciden
    return res;
}

// Puntos de contacto de las tangentes desde un punto p
    al circulo c (0 si p esta dentro, 1 si en el borde,
     2 si fuera)
vector<pt> tangents_point_circle(const pt& p, const
    Circle& c) {
    vector<pt> res;
    for (auto& t : tangents_circle_circle(Circle(p, 0),
        c)) res.push_back(t.b);
    return res;
}

// Circulo circunscrito del triangulo abc (pasa por los
    3 vertices). Requiere puntos no colineales
Circle circumcircle(const pt& a, const pt& b, const pt&
    c) {
    pt u = b - a, v = c - a;
    pt center = a + (u * v.sq() - v * u.sq()).perp() / u
        .cross(v) / 2;
    return Circle(center, dist(center, a));
}

// Circulo inscrito del triangulo abc (tangente a los 3
    lados)
Circle incircle(const pt& a, const pt& b, const pt& c) {
    ld la = dist(b, c), lb = dist(a, c), lc = dist(a, b)
        ;   // longitud del lado opuesto a cada vertice
    ld perimeter = la + lb + lc;
    pt center = (a * la + b * lb + c * lc) / perimeter;
            // promedio ponderado por lados
    ld radius = fabsl(orient(a, b, c)) / perimeter;
                // r = area / semiperimetro
    return Circle(center, radius);
}

// Area de la interseccion entre el circulo c y un
    poligono simple (cualquier orientacion)
ld circle_polygon_intersection_area(const Circle& c,
    const vector<pt>& poly) {
    // Angulo orientado entre dos vectores
    auto arg = [](const pt& p, const pt& q) { return
        atan2l(p.cross(q), p.dot(q)); };

    // Area con signo de (triangulo O-p-q) & circulo
        centrado en el origen
    auto tri = [&](pt p, pt q) -> ld {
        ld half_r2 = c.r * c.r / 2;
        pt d = q - p;
        if (sgn(d.sq()) == 0) return 0;
                                // arista degenerada

        // Se resuelve |p + d*t|^2 = r^2 para hallar
            donde la arista entra/sale del circulo
        ld a = d.dot(p) / d.sq();
        ld b = (p.sq() - c.r * c.r) / d.sq();
        ld det = a * a - b;
        if (det <= 0) return arg(p, q) * half_r2;
                    // la recta no corta: solo sector

        ld s = max((ld)0, -a - sqrtl(det));
                                // parametro de entrada
        ld t = min((ld)1, -a + sqrtl(det));
                                // parametro de salida
        if (t < 0 || 1 <= s) return arg(p, q) * half_r2;
                // corta fuera del segmento: solo
```

```
            sector
        pt u = p + d * s, v = p + d * t;
                                // tramo interior =
            triangulo; extremos = sectores
        return arg(p, u) * half_r2 + u.cross(v) / 2 +
            arg(v, q) * half_r2;
    };
    ld total = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        total += tri(poly[i] - c.o, poly[(i + 1) % n] -
            c.o);
    }
    return fabsl(total);
}

// Circulo minimo que contiene todos los puntos (Welzl
    iterativo, esperado O(n))
Circle min_enclosing_circle(vector<pt> ps) {
    mt19937 rng(712367);
    shuffle(ps.begin(), ps.end(), rng);      // el orden
        aleatorio garantiza el O(n) esperado
    const ld slack = 1 + 1e-8;               //
        tolerancia relativa
    Circle c(ps[0], 0);
    int n = ps.size();
    for (int i = 0; i < n; i++) {
        if (dist(c.o, ps[i]) <= c.r * slack) continue;
        c = Circle(ps[i], 0);
                                                // ps
            [i] debe estar en el borde
        for (int j = 0; j < i; j++) {
            if (dist(c.o, ps[j]) <= c.r * slack)
                continue;
            c = Circle((ps[i] + ps[j]) / 2, dist(ps[i],
                ps[j]) / 2); // ps[i], ps[j] en el
                borde
            for (int k = 0; k < j; k++) {
                if (dist(c.o, ps[k]) <= c.r * slack)
                    continue;
                c = circumcircle(ps[i], ps[j], ps[k]);
                                // los tres en el
                    borde
            }
        }
    }
    return c;
}

//////////////////////////////////////////
// 4. TRIANGULOS
//////////////////////////////////////////

template<typename T = ld>
struct Triangle {
    Point<T> a, b, c;

    Triangle() : a(), b(), c() {}
    Triangle(Point<T> _a, Point<T> _b, Point<T> _c) : a(
        _a), b(_b), c(_c) {}

    // Area con signo: > 0 si a->b->c es antihorario, <
        0 si es horario
    ld signed_area() const { return (ld)orient(a, b, c)
        / 2; }

    // Area (siempre >= 0)
    ld area() const { return fabsl(signed_area()); }

    // Perimetro
    ld perimeter() const { return dist(a, b) + dist(b, c
        ) + dist(c, a); }

    // Posicion del punto p: 1 (interior estricto), 0 (
        en el borde o vertice), -1 (exterior)
    int contains(const Point<T>& p) const {
        int s1 = sgn(orient(a, b, p));
        int s2 = sgn(orient(b, c, p));
        int s3 = sgn(orient(c, a, p));

        bool has_neg = (s1 < 0 || s2 < 0 || s3 < 0);
        bool has_pos = (s1 > 0 || s2 > 0 || s3 > 0);
        if (has_neg && has_pos) return -1;
            // lados distintos de alguna arista: afuera
        if (s1 == 0 || s2 == 0 || s3 == 0) return 0;
            // sobre una arista
        return 1;
    }

    // Baricentro: promedio de los vertices
    Point<ld> centroid() const {
        return Point<ld>((ld)(a.x + b.x + c.x) / 3, (ld)
            (a.y + b.y + c.y) / 3);
    }

    // Circuncentro: centro del circulo que pasa por los
        3 vertices
    Point<ld> circumcenter() const {
        return circumcircle(pt(a.x, a.y), pt(b.x, b.y),
            pt(c.x, c.y)).o;
    }

    // Incentro: centro del circulo inscrito
    Point<ld> incenter() const {
        return incircle(pt(a.x, a.y), pt(b.x, b.y), pt(c
            .x, c.y)).o;
    }

    // Ortocentro: interseccion de las alturas (recta de
        Euler: H = 3G - 2O)
    Point<ld> orthocenter() const {
        return centroid() * 3 - circumcenter() * 2;
    }

    friend istream& operator>>(istream& is, Triangle& t)
        { return is >> t.a >> t.b >> t.c; }
    friend ostream& operator<<(ostream& os, const
        Triangle& t) {
        return os << "[" << t.a << "] [" << t.b << "] ["
            << t.c << "]";
    }
};

using tri = Triangle<ld>;
using trill = Triangle<ll>;

// Verifica si dos triangulos comparten al menos un
    punto (choque de bordes o uno dentro del otro)
template<typename T>
bool triangles_intersect(const Triangle<T>& t1, const
    Triangle<T>& t2) {
    Point<T> p1[3] = {t1.a, t1.b, t1.c};
    Point<T> p2[3] = {t2.a, t2.b, t2.c};

    // 1. Alguna arista de t1 toca alguna arista de t2
        (9 pares)
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            Segment<T> e1(p1[i], p1[(i + 1) % 3]);
            Segment<T> e2(p2[j], p2[(j + 1) % 3]);
            if (segments_intersect(e1, e2)) return true;
        }
    }

    // 2. Sin tocarse los bordes, uno puede estar
        completamente dentro del otro
    return t1.contains(t2.a) >= 0 || t2.contains(t1.a)
        >= 0;
}

//////////////////////////////////////////
// 5. POLIGONOS
//    (un poligono es un vector<Point<T>> con los
    vertices en orden)
//////////////////////////////////////////

// DOBLE del area con signo (formula del cordon /
    shoelace). > 0 si el poligono es CCW.
// Se devuelve el doble para que sea exacto con enteros:
     area = abs(polygon_area2(p)) / 2
template<typename T>
T polygon_area2(const vector<Point<T>>& poly) {
    T s = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++) s += poly[i].cross(poly
        [(i + 1) % n]);
    return s;
}

// Centroide (centro de masa) de un poligono simple
pt polygon_centroid(const vector<pt>& poly) {
    pt c;
    ld area6 = 0;   // 3 * (doble del area)
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        pt a = poly[i], b = poly[(i + 1) % n];
        ld w = a.cross(b);
        c = c + (a + b) * w;
        area6 += w;
    }
    return c / (3 * area6);
}

// Posicion de p respecto a un poligono simple: 1 (
    dentro), 0 (en el borde), -1 (fuera). O(n)
template<typename T>
int point_in_polygon(const Point<T>& p, const vector<
    Point<T>>& poly) {
    int n = poly.size();
    bool inside = false;
    for (int i = 0; i < n; i++) {
        Segment<T> edge(poly[i], poly[(i + 1) % n]);
        if (edge.contains(p)) return 0;
        // Rayo horizontal desde p: se cuenta cada
            arista que cruza hacia la derecha de p
        if (((p.y < edge.a.y) - (p.y < edge.b.y)) *
            orient(p, edge.a, edge.b) > 0) inside = !
            inside;
    }
    return inside ? 1 : -1;
}

// Envolvente convexa (Andrew / monotone chain), O(n log
    n).
// Resultado en sentido antihorario, sin puntos
    colineales ni duplicados.
template<typename T>
vector<Point<T>> convex_hull(vector<Point<T>> pts) {
    sort(pts.begin(), pts.end());
    pts.erase(unique(pts.begin(), pts.end()), pts.end())
        ;
    int n = pts.size(), k = 0;
    if (n < 3) return pts;

    vector<Point<T>> h(2 * n);
    for (int i = 0; i < n; i++) {
        // cadena inferior
        while (k >= 2 && sgn(orient(h[k - 2], h[k - 1],
            pts[i])) <= 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
```

```
        // cadena superior
        while (k >= t && sgn(orient(h[k - 2], h[k - 1],
            pts[i])) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}

// Diametro de un poligono convexo CCW (rotating
    calipers): devuelve la distancia^2 maxima entre
    vertices
template<typename T>
T convex_diameter_sq(const vector<Point<T>>& hull) {
    int n = hull.size();
    int j = (n < 2) ? 0 : 1;
    T best = 0;
    for (int i = 0; i < j; i++) {        // j crece
        mientras recorremos: el limite se reevalua
        for (;; j = (j + 1) % n) {
            best = max(best, (hull[i] - hull[j]).sq());
            // Avanzar j mientras el siguiente vertice
                se aleje de i
            if (sgn((hull[(j + 1) % n] - hull[j]).cross(
                hull[i + 1] - hull[i])) >= 0) break;
        }
    }
    return best;
}

// Par de puntos mas cercano (barrido con set), O(n log
    n). Requiere n >= 2
template<typename T>
pair<Point<T>, Point<T>> closest_pair(vector<Point<T>>
    pts) {
    sort(pts.begin(), pts.end(), [](const Point<T>& a,
        const Point<T>& b) { return a.y < b.y; });

    set<Point<T>> active;                    // puntos
        dentro de la franja vertical actual, ordenados
        por x
    ld best = 4e18;                          // mejor
        distancia^2 encontrada
    pair<Point<T>, Point<T>> ans;
    int left = 0;

    for (auto& p : pts) {
        Point<T> d((T)(sqrtl(best) + 1), 0);         //
            radio de busqueda
        while (pts[left].y <= p.y - d.x) {           //
            sacar puntos demasiado abajo
            active.erase(pts[left++]);
        }
        auto lo = active.lower_bound(p - d);
        auto hi = active.upper_bound(p + d);
        for (; lo != hi; ++lo) {
            ld e = (*lo - p).sq();
            if (e < best) { best = e; ans = {*lo, p}; }
        }
        active.insert(p);
    }
    return ans;
}

// Ordena puntos/vectores por angulo polar en [0, 2*PI)
    respecto al origen (sin usar atan2)
template<typename T>
void polar_sort(vector<Point<T>>& pts) {
    // half = 0 para el semiplano superior (angulo en
        [0, PI)), 1 para el inferior
    auto half = [](const Point<T>& p) { return sgn(p.y)
        < 0 || (sgn(p.y) == 0 && sgn(p.x) < 0); };
    sort(pts.begin(), pts.end(), [&](const Point<T>& a,
        const Point<T>& b) {
        if (half(a) != half(b)) return half(a) < half(b)
            ;
        return sgn(a.cross(b)) > 0;      // misma mitad:
            a va antes si b esta a su izquierda
    });
}

// Recorta un poligono CONVEXO conservando solo la parte
    a la izquierda de la recta l (dirigida a->b)
vector<pt> cut_convex_polygon(const vector<pt>& poly,
    const seg& l) {
    vector<pt> res;
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        pt c = poly[i], d = poly[(i + 1) % n];
        ld sc = orient(l.a, l.b, c), sd = orient(l.a, l.
            b, d);
        if (sgn(sc) >= 0) res.push_back(c);
                                                 // c se queda
        if (sgn(sc) * sgn(sd) < 0) res.push_back(c + (d
            - c) * (sc / (sc - sd)));  // la arista
            cruza la recta
    }
    return res;
}

// Semiplano: region a la IZQUIERDA de la recta dirigida
    a->b
struct HalfPlane {
    pt p, d;     // un punto de la recta y su direccion
    ld ang;      // angulo de la direccion (para ordenar)

    HalfPlane(const pt& a, const pt& b) : p(a), d(b - a)
        , ang(atan2l(d.y, d.x)) {}
    HalfPlane(const seg& l) : HalfPlane(l.a, l.b) {}
        // izquierda de la recta que va de l.a a l.b

    // true si q queda estrictamente fuera (a la derecha
        )
    bool out(const pt& q) const { return sgn(d.cross(q -
        p)) < 0; }
};

// Punto donde se cortan las rectas de dos semiplanos
pt halfplane_inter(const HalfPlane& s, const HalfPlane&
    t) {
    return s.p + s.d * ((t.p - s.p).cross(t.d) / s.d.
        cross(t.d));
}

// Interseccion de semiplanos, O(n log n). Devuelve los
    vertices del poligono resultante (CCW),
// o vacio si la interseccion es vacia. Si puede ser no
    acotada, agrega 4 semiplanos de una caja enorme.
vector<pt> half_plane_intersection(vector<HalfPlane> H)
    {
    sort(H.begin(), H.end(), [](const HalfPlane& a,
        const HalfPlane& b) { return a.ang < b.ang; });
    deque<HalfPlane> dq;

    for (auto& h : H) {
        // Quitar semiplanos del final/inicio cuyo
            vertice ya quedo fuera del nuevo semiplano
        while (dq.size() > 1 && h.out(halfplane_inter(dq
            .back(), dq[dq.size() - 2]))) dq.pop_back()
            ;
        while (dq.size() > 1 && h.out(halfplane_inter(dq
            [0], dq[1]))) dq.pop_front();

        // Rectas paralelas
        if (!dq.empty() && sgn(h.d.cross(dq.back().d))
            == 0) {
            if (h.d.dot(dq.back().d) < 0) return {};
                                 // direcciones opuestas:
                interseccion vacia
            if (h.out(dq.back().p)) dq.pop_back();
                                 // mismo sentido: nos
                quedamos con el mas restrictivo
            else continue;
        }
        dq.push_back(h);
    }

    // Limpieza final entre el inicio y el final del
        deque
    while (dq.size() > 2 && dq[0].out(halfplane_inter(dq
        .back(), dq[dq.size() - 2]))) dq.pop_back();
    while (dq.size() > 2 && dq.back().out(
        halfplane_inter(dq[0], dq[1]))) dq.pop_front();
    if (dq.size() < 3) return {};

    vector<pt> res;
    for (size_t i = 0; i < dq.size(); i++) {
        res.push_back(halfplane_inter(dq[i], dq[(i + 1)
            % dq.size()]));
    }
    return res;
}

// Suma de Minkowski de dos poligonos convexos CCW: {a +
    b | a en A, b en B}
template<typename T>
vector<Point<T>> minkowski_sum(vector<Point<T>> A,
    vector<Point<T>> B) {
    // Rotar cada poligono para que empiece en su
        vertice mas bajo (y luego mas a la izquierda)
    auto reorder = [](vector<Point<T>>& v) {
        auto it = min_element(v.begin(), v.end(), [](
            const Point<T>& a, const Point<T>& b) {
            return tie(a.y, a.x) < tie(b.y, b.x);
        });
        rotate(v.begin(), it, v.end());
    };
    reorder(A);
    reorder(B);
    A.push_back(A[0]); A.push_back(A[1]);   //
        centinelas para no salirse del rango
    B.push_back(B[0]); B.push_back(B[1]);

    vector<Point<T>> res;
    size_t i = 0, j = 0;
    while (i < A.size() - 2 || j < B.size() - 2) {
        res.push_back(A[i] + B[j]);
        int c = sgn((A[i + 1] - A[i]).cross(B[j + 1] - B
            [j]));  // cual arista tiene menor angulo
        if (c >= 0 && i < A.size() - 2) i++;
        if (c <= 0 && j < B.size() - 2) j++;
    }
    return res;
}

// Teorema de Pick para poligono con vertices enteros: A
    = I + B/2 - 1
// Devuelve {I = puntos enteros interiores, B = puntos
    enteros en el borde}
pair<ll, ll> pick_theorem(const vector<ptll>& poly) {
    ll boundary = 0;
    int n = poly.size();
    for (int i = 0; i < n; i++) {
        ptll d = poly[(i + 1) % n] - poly[i];
        boundary += __gcd(llabs(d.x), llabs(d.y));
            // puntos enteros en cada arista
    }
    ll interior = (llabs(polygon_area2(poly)) - boundary
        + 2) / 2;
    return {interior, boundary};
}

////////////////////////////////////////////
// 6. ESFERA
////////////////////////////////////////////
```

```
// Distancia sobre la superficie de una esfera de radio
    R (haversine).
// Latitud y longitud en RADIANES (grados * PI / 180).
ld sphere_distance(ld lat1, ld lon1, ld lat2, ld lon2,
    ld R = 1) {
    ld s_lat = sinl((lat2 - lat1) / 2);
    ld s_lon = sinl((lon2 - lon1) / 2);
    ld a = s_lat * s_lat + cosl(lat1) * cosl(lat2) *
        s_lon * s_lon;
    return 2 * R * asinl(min((ld)1, sqrtl(a)));
}
```

12.2 Convex Hull

```
/*
- Complejidad: O(n*log(n))
- Retorna los puntos que hacen parte del convex hull (no
  necesariamente ordenados).
- Si se permite colinealidad es posible que se repitan
    algunos puntos.
- Si todas las coordenadas son enteras se pueden usar
    enteros y eliminar
  los epsilon.
*/

const ld eps = 1e-9;
struct pt{
    ld x, y;
    pt(){}
    pt(ld x, ld y): x(x), y(y){}
    bool operator<(pt p)const{ // for sort, convex hull/
        set/map
        return x < p.x - eps || (abs(x - p.x) <= eps &&
            y < p.y - eps); }
    pt operator-(pt p){return pt(x - p.x, y - p.y);}
    ld operator%(pt p){return x * p.y - y * p.x;}
    ld side(pt p, pt q){return (q - p) % (*this - p);}
};
// CCW order, excludes collinear points
// Change .side(r[sz(r)-2], p[i]) > 0 to include
    collinear
vector<pt> chull(vector<pt>& p){
    if(sz(p) < 3) return p;
    vector<pt> r;
    sort(all(p));  // first x, then y
    for0(i, sz(p)){ // lower hull
        while(sz(r) > 1 && r.back().side(r[sz(r)-2], p[i
            ]) > 0) r.pop_back();
        r.pb(p[i]);
    }
    r.pop_back();
    int k = sz(r);
    forrl(i, 0, sz(p)-1){ // upper hull
        while(sz(r) > k+1 && r.back().side(r[sz(r)-2], p
            [i]) > 0) r.pop_back();
        r.pb(p[i]);
    }
    r.pop_back();
    return r;
}
```

12.3 Intersection Points

```
#include "../../template.h"

const ll INFL = 1e18;
const int INF = 1e9;

struct Event {
    int x, type, y1, y2;
    bool operator<(const Event& other) const {
        if (x != other.x) return x < other.x;
        return type < other.type; // 0: start, 1: query,
            2: end
    }
};

// FenWick Tree Implementation
const int MAX_Y = 2000010;
int bit[MAX_Y];

void add(int idx, int val) {
    for (; idx < MAX_Y; idx += idx & -idx) bit[idx] +=
        val;
}

int query(int idx) {
    int sum = 0;
    for (; idx > 0; idx -= idx & -idx) sum += bit[idx];
    return sum;
}

int query(int l, int r) {
    return query(r) - query(l - 1);
}

void solver(){
    int n;
    cin>>n;

    vector<Event> events;
    int XSTART=0; // start of a horizontal segment
    int YLINE=1; // vertical segment
    int XEND=2;  // end of a horizontal segment
    for0(i, n) {
        int x1, y1, x2, y2;
        cin>>x1>>y1>>x2>>y2;
        if(x1 > x2) swap(x1, x2);
        if(y1 > y2) swap(y1, y2);

        // Desplazamiento (offset) para manejar
            coordenadas negativas en el BIT
        int offset = 1e6 + 5;

        if (y1 == y2)
        { // horizontal segment, query at start and end
            of the segment
            int y = y1 + offset;
            events.pb({x1, XSTART, y, y});
            events.pb({x2, XEND, y, y});
        }
        else if (x1 == x2)
        { //vertical segment
            int x = x1;
            int y_low = y1 + offset;
            int y_high = y2 + offset;
            events.pb({x, YLINE, y_low, y_high});
        }
    }

    sort(all(events));

    // Cuenta cuantos segmentos horizontales estan
        activos en cada punto de la linea vertical
    ll ans = 0;
    for (const auto& e : events) {
        if (e.type == XSTART) {
            add(e.y1, 1);
        } else if (e.type == YLINE) {
            ans += query(e.y1, e.y2);
        } else if (e.type == XEND) {
            add(e.y1, -1);
        }
    }

    cout<<ans<<"\n";
}
```

12.4 Line Segment Intersection

```
#include "../../template.h"

struct Point {
    int x, y;

    Point operator-(const Point& a) const {
        return {x - a.x, y - a.y};
    }

    long long cross(const Point& other) const {
        return (1LL * x * other.y) - (1LL * y * other.x)
            ;
    }
};

struct Segment {
    Point p1, p2;

    /**
     * Verifica si un punto 'x' que es colineal con el
        segmento (a, b)
     * se encuentra fisicamente dentro del rectangulo
        delimitado por los extremos del segmento.
     */
    bool PointOnSegment(const Point& p) const {
        return min(p1.x, p2.x) <= p.x && p.x <= max(p1.x
            , p2.x) &&
               min(p1.y, p2.y) <= p.y && p.y <= max(p1.y
                   , p2.y);
    }
};

long long cruz(Point p1, Point p2) {
    return p1.cross(p2);
}

int sign(long long x) {
    return (x > 0) - (x < 0);
}

/**
 * Determina la orientacion del triplete de puntos (a, b
    , c):
 *   0  -> Los puntos son colineales.
 *   1  -> Giro en sentido antihorario (Counter-
    Clockwise).
 *  -1  -> Giro en sentido horario (Clockwise).
 */
int orientation(Point a, Point b, Point c) {
    return sign(cruz(c - a, b - a));
}


void solver() {
    int n;
    cin >> n;
    Point p1, p2, p3, p4;
    while (n--) {
        cin >> p1.x >> p1.y >> p2.x >> p2.y >> p3.x >>
            p3.y >> p4.x >> p4.y;
        Segment s1{p1, p2}, s2{p3, p4};

        // Evalua la orientacion relativa de los puntos
            de un segmento respecto al otro.
        int a = orientation(p1, p2, p3);
        int b = orientation(p1, p2, p4);
        int c = orientation(p3, p4, p1);
        int d = orientation(p3, p4, p2);

        // Verifica si los segmentos se intersectan:
        // 1. Interseccion general: los extremos del
            segmento 2 quedan a lados opuestos del
            segmento 1 (a != b)
        //    y viceversa (c != d).
```

```
        // 2. Casos especiales (colinealidad): un
             extremo de un segmento descansa sobre el
             otro segmento.
        if ((a != b && c != d) ||
            (a == 0 && s1.PointOnSegment(p3)) ||
            (b == 0 && s1.PointOnSegment(p4)) ||
            (c == 0 && s2.PointOnSegment(p1)) ||
            (d == 0 && s2.PointOnSegment(p2))) {
            cout << "YES\n";
        } else {
            cout << "NO\n";
        }
    }
}
```

12.5 Segment Intersection

```
/*
  Retorna si el punto (x, y) esta dentro (o en el borde)
       de la caja definida por los puntos
  (x0, y0) y (x1, y1)
*/
bool pointInBox(ld x, ld y, ld x0, ld y0, ld x1, ld y1){
    return min(x0, x1) <= x && x <= max(x0, x1) &&
           min(y0, y1) <= y && y <= max(y0, y1);
}

/*
  Retorna el producto cruz del segmento que va desde {x1
       , y1} hasta {x3, y3} con el segmento que
  va de {x1, y1} hasta {x2, y2}.
*/
int direction(int x1, int y1, int x2, int y2, int x3,
    int y3){
    return (x3 - x1) * (y2 - y1) - (y3 - y1) * (x2 - x1)
        ;
}

/*
  Como saber si dos segmentos de recta [{x1, y1}, {x2,
       y2}] y [{x3, y3}, {x4, y4}] se intersectan
  (no halla el punto exacto de interseccion).
  Tambien funciona para segmentos paralelos, donde en
       caso de que compartan puntos infinitos, se
  toma como que si se intersectan.
*/
bool segmentIntersection(int x1, int y1, int x2, int y2,
     int x3, int y3, int x4, int y4){
    int d1 = direction(x3, y3, x4, y4, x1, y1);
    int d2 = direction(x3, y3, x4, y4, x2, y2);
    int d3 = direction(x1, y1, x2, y2, x3, y3);
    int d4 = direction(x1, y1, x2, y2, x4, y4);

    bool b1 = (d1 > 0 && d2 < 0) || (d1 < 0 && d2 > 0);
    bool b2 = (d3 > 0 && d4 < 0) || (d3 < 0 && d4 > 0);
    if(b1 && b2) return 1;

    if(d1 == 0 && pointInBox(x1, y1, x3, y3, x4, y4))
        return 1;
    if(d2 == 0 && pointInBox(x2, y2, x3, y3, x4, y4))
        return 1;
    if(d3 == 0 && pointInBox(x3, y3, x1, y1, x2, y2))
        return 1;
    if(d4 == 0 && pointInBox(x4, y4, x1, y1, x2, y2))
        return 1;

    return 0;
}

void solver(){
    int x1, y1, x2, y2, x3, y3, x4, y4; cin>>x1>>y1>>x2
        >>y2>>x3>>y3>>x4>>y4;
    cout<<segmentIntersection(x1, y1, x2, y2, x3, y3, x4
        , y4)<<endl;
}
```

12.6 Point in Polygon

```
struct P {
    ll x, y;

    P operator-(P o) {
        return {x - o.x, y - o.y};
    }
};

using i128 = __int128_t;

i128 cross(P a, P b) {
    return (i128)a.x * b.y - (i128)a.y * b.x;
}

bool onSegment(P a, P b, P c) {
    if(cross(b - a, c - a) != 0) return false;

    return min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x)
        &&
           min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y);
}

string pointInPolygon(vector<P>& poly, P p) {
    int n = poly.size();
    bool inside = false;

    for0(i, n) {
        P a = poly[i];
        P b = poly[(i + 1) % n];

        if(onSegment(a, b, p))
            return "BOUNDARY";

        if((a.y > p.y) != (b.y > p.y)) {
            i128 cr = cross(b - a, p - a);
            if((b.y > a.y && cr > 0) ||
               (b.y < a.y && cr < 0)) {
                inside = !inside;
            }
        }
    }

    return inside ? "INSIDE" : "OUTSIDE";
}
```

12.7 Polygons Area

```
struct Point {
    double x, y;

    double cross(const Point& other) const {
        return x * other.y - y * other.x;
    }
};

/**
 * Calcula el area de un poligono simple ordenado
      mediante la formula de
 * Gauss / Shoelace:
 *                          | n-1
 *    Area(P) = 1/2 * | SUM (Pi x P((i+1)%n)
 *                          | i=0
 */
double polygonArea(vector<Point>& poly) {
    int n = poly.size();
    if (n < 3) return 0.0; // Un poligono debe tener al
         menos 3 vertices

    double cross_sum = 0.0;
    for (int i = 0; i < n; ++i) {
        cross_sum += poly[i].cross(poly[(i + 1) % n]);
    }
    // abs porque dependiendo si esta ordenado clockwise
         o anticlockwise
    // retornaria positivo o negativo
    return abs(cross_sum) / 2.0;
}
```

12.8 Li Chao Tree

```
// LiChao<T, MAX>: T = ll o ld, MAX = true para maximo,
    false para minimo.
// - addLine(m, b)              recta completa
                    O(log n)
// - addLine(x1,y1,x2,y2)       recta por 2 puntos (solo
    flotante)
// - addSeg(m, b, xl, xr)       recta restringida a [xl,
    xr]  O(log^2 n)
// - addSegP(x1,y1,x2,y2)       segmento por 2 puntos (
    solo flotante)
// - query(x) devuelve -1 si no hay recta/segmento que
    cubra x, o si x no
//   esta en xs. Asume que las respuestas reales nunca
    son -1.
// - Con ll cuidar que |m*x + b| quepa en ll. Con ld se
    usa eps = 1e-9.

template<class T, bool MX = false> struct LiChao{
    struct Line{
        T m, b;
        Line(): m(0), b(numeric_limits<T>::max()){}
        Line(T m, T b): m(m), b(b){}
        T operator()(T x)const{ return m * x + b; }
    };
    vector<T> xs;
    vector<Line> t;
    int n;

    static bool lt(T a, T b){
        if constexpr(is_floating_point<T>::value) return
            a < b - 1e-9;
        else return a < b;
    }

    LiChao(vector<T> v): xs(v){
        sort(all(xs));
        xs.erase(unique(all(xs)), xs.end());
        n = sz(xs);
        t.assign(4 * max(n, 1), Line());
    }
    LiChao(ll l, ll r): xs(r - l + 1){
        iota(all(xs), (T)l);
        n = sz(xs);
        t.assign(4 * max(n, 1), Line());
    }
    LiChao(int m) : LiChao(0, m) {}

    void addLine(T m, T b){
        if(MX){ m = -m; b = -b; }
        if(n) insert(1, 0, n - 1, Line(m, b));
    }
    void addLine(T x1, T y1, T x2, T y2){
        static_assert(is_floating_point<T>::value, "
            addLine(2 puntos) requiere T flotante");
        if(x1 == x2) return;                    //
            recta vertical: no representable
        T m = (y2 - y1) / (x2 - x1);
        addLine(m, y1 - m * x1);
    }

    void addSeg(T m, T b, T xl, T xr){
        if(MX){ m = -m; b = -b; }
        int ia = lower_bound(all(xs), xl) - xs.begin();
        int ib = upper_bound(all(xs), xr) - xs.begin() -
            1;
        if(ia <= ib) addRange(1, 0, n - 1, ia, ib, Line(
```

```
            m, b));
    }
    void addSegP(T x1, T y1, T x2, T y2){
        static_assert(is_floating_point<T>::value, "
            addSeg(2 puntos) requiere T flotante");
        if(x1 > x2){ swap(x1, x2); swap(y1, y2); }
        if(x1 == x2){ addSeg(0, MX ? max(y1, y2) : min(
            y1, y2), x1, x1); return; }
        T m = (y2 - y1) / (x2 - x1);
        addSeg(m, y1 - m * x1, x1, x2);
    }

    T query(T x){
        int i = lower_bound(all(xs), x) - xs.begin();
        if(i >= n || xs[i] != x) return -1;
        int v = 1, l = 0, r = n - 1;
        T res = numeric_limits<T>::max();
        while(true){
            res = min(res, t[v](x));
            if(l == r) break;
            int m = (l + r) / 2;
            if(i <= m){ v = 2 * v; r = m; }
            else{ v = 2 * v + 1; l = m + 1; }
        }
        // si el problema permite negativos, devolver un
            par <bool, T>
        if(res >= numeric_limits<T>::max() / 2) return
            -1;
        return MX ? -res : res;
    }

private:
    void insert(int v, int l, int r, Line nw){
        int m = (l + r) / 2;
        bool lef = lt(nw(xs[l]), t[v](xs[l]));
        bool mid = lt(nw(xs[m]), t[v](xs[m]));
        if(mid) swap(t[v], nw);
        if(l == r) return;
        if(lef != mid) insert(2 * v, l, m, nw);
        else insert(2 * v + 1, m + 1, r, nw);
    }
    void addRange(int v, int l, int r, int a, int b,
        Line nw){
        if(b < l || r < a) return;
        if(a <= l && r <= b){ insert(v, l, r, nw);
            return; }
        int m = (l + r) / 2;
        addRange(2 * v, l, m, a, b, nw);
        addRange(2 * v + 1, m + 1, r, a, b, nw);
    }
};
```

12.9 Gauss (UNTESTED)

```
// NO TESTED in a real problem yet

////////////////////////////////////////
// ELIMINACION DE GAUSS-JORDAN
//
// 1. Gauss modular (mod primo, por defecto 1e9+7):
    sistemas, rango, det, inversa, espacio nulo.
// 2. Gauss mod 2 con bitset: sistemas XOR (~64x mas
    rapido que vector<bool>).
// 3. Gauss real (long double, pivoteo parcial). Acepta
    entradas long long.
//
// Convencion de status:  1 = solucion unica | 0 =
    infinitas | -1 = sin solucion
//
// Notas:
//  - Las entradas pueden ser negativas o >= mod: se
    normalizan internamente.
//  - El modulo DEBE ser primo (se usan inversos
    modulares). Con 1e9+7 / 998244353 no hay problema.
//  - No de soluciones (si existen) = mod^(m - rank)  (
    XOR: 2^(m - rank)).
//  - Complejidad: O(n * m * min(n, m)).
////////////////////////////////////////

const ll MOD = 1e9 + 7;
const ld EPS = 1e-9;

inline ll norm(ll a, ll m = MOD) { a %= m; return a < 0
    ? a + m : a; }
inline ll mulmod(ll a, ll b, ll m = MOD) { return (ll)((
    __int128)a * b % m); }

// Inverso modular con Euclides extendido iterativo.
    Devuelve -1 si no existe.
ll mod_inv(ll a, ll m = MOD) {
    a = norm(a, m);
    ll b = m, x0 = 1, x1 = 0;
    while (b) {
        ll q = a / b;
        ll t = a - q * b; a = b; b = t;
        t = x0 - q * x1; x0 = x1; x1 = t;
    }
    if (a != 1) return -1;
    return norm(x0, m);
}

ll binpow(ll b, ll e, ll m = MOD) {
    b = norm(b, m);
    ll r = 1 % m;
    while (e > 0) {
        if (e & 1) r = mulmod(r, b, m);
        b = mulmod(b, b, m);
        e >>= 1;
    }
    return r;
}

// Requiere matriz n*m
struct Matrix {
    int n, m;
    vector<vl> M;

    Matrix() : n(0), m(0) {}
    Matrix(int n) : n(n), m(n), M(n, vl(n, 0)) {}
    Matrix(int n, int m) : n(n), m(m), M(n, vl(m, 0)) {}
    Matrix(const vector<vl>& mat) : n(sz(mat)), m(sz(mat
        ) ? sz(mat[0]) : 0), M(mat) {}

    vl& operator[](int i) { return M[i]; }
    const vl& operator[](int i) const { return M[i]; }

    static Matrix identity(int n) {
        Matrix I(n);
        for (int i = 0; i < n; i++) I.M[i][i] = 1;
        return I;
    }

    Matrix add(const Matrix& o, ll mod = MOD) const {
        Matrix r(n, m);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) r.M[i][j] = norm
                (M[i][j] + o.M[i][j], mod);
        return r;
    }

    // O(n * k * m)
    Matrix mul(const Matrix& o, ll mod = MOD) const {
        Matrix r(n, o.m);
        for (int i = 0; i < n; i++)
            for (int k = 0; k < m; k++) {
                ll a = norm(M[i][k], mod);
                if (!a) continue;
                for (int j = 0; j < o.m; j++) {
                    ll v = r.M[i][j] + mulmod(a, norm(o.
                        M[k][j], mod), mod);
                    if (v >= mod) v -= mod;
                    r.M[i][j] = v;
                }
            }
        return r;
    }

    vl mul(const vl& v, ll mod = MOD) const {
        vl r(n, 0);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++)
                r[i] = norm(r[i] + mulmod(norm(M[i][j],
                    mod), norm(v[j], mod), mod), mod);
        return r;
    }

    Matrix operator+(const Matrix& o) const { return add
        (o); }
    Matrix operator*(const Matrix& o) const { return mul
        (o); }
    vl operator*(const vl& v) const { return mul(v); }

    // Matriz cuadrada elevada a exp, O(log(exp) * n^3)
    Matrix binpow(ll exp, ll mod = MOD) const {
        Matrix res = identity(n), base = *this;
        while (exp > 0) {
            if (exp & 1) res = res.mul(base, mod);
            base = base.mul(base, mod);
            exp >>= 1;
        }
        return res;
    }

    Matrix transpose() const {
        Matrix T(m, n);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) T.M[j][i] = M[i
                ][j];
        return T;
    }

    friend ostream& operator<<(ostream& os, const Matrix
        & a) {
        os << "\nMatrix (" << a.n << "x" << a.m << "):\n
            ";
        for (int i = 0; i < a.n; i++) {
            for (int j = 0; j < a.m; j++) os << a.M[i][j
                ] << " ";
            os << "\n";
        }
        return os << "================\n";
    }

    // Declarados aqui, definidos al final del archivo
    ll det(ll mod = MOD) const;
    int get_rank(ll mod = MOD) const;
    pair<bool, Matrix> inv(ll mod = MOD) const;
};

////////////////////////////////////////
// 1. GAUSS MODULAR
////////////////////////////////////////

// Nucleo: forma escalonada reducida (RREF) in-place,
    pivoteando solo en las primeras `lim` columnas.
// Las operaciones afectan a TODAS las columnas (permite
    matriz aumentada).
// where[col] = fila del pivote de esa columna (o -1 si
    es libre). Devuelve el rango.
int rref_mod(Matrix& a, int lim, vi& where, ll mod = MOD
    ) {
    int n = a.n, m = a.m;
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++) a.M[i][j] = norm(a.M
            [i][j], mod);

    where.assign(lim, -1);
    int row = 0;
    for (int col = 0; col < lim && row < n; col++) {
        int sel = -1;
        for (int i = row; i < n; i++)
            if (a.M[i][col]) { sel = i; break; }
        if (sel == -1) continue;

        swap(a.M[sel], a.M[row]);
        where[col] = row;

        ll inv = mod_inv(a.M[row][col], mod);
        for (int j = col; j < m; j++) a.M[row][j] =
            mulmod(a.M[row][j], inv, mod);

        for (int i = 0; i < n; i++) {
            if (i == row) continue;
            ll f = a.M[i][col];
            if (!f) continue;
            for (int j = col; j < m; j++) {
                ll v = a.M[i][j] - mulmod(f, a.M[row][j
                    ], mod);
                if (v < 0) v += mod;
                a.M[i][j] = v;
            }
        }
        row++;
    }
    return row;
}

struct SolMod {
    int status;          // 1 unica, 0 infinitas, -1 sin
        solucion
    int rank;            // rango de A
    vl x;                // solucion particular (
        variables libres = 0); vacio si no hay solucion
    vector<vl> basis;    // base del espacio nulo (m -
        rank vectores). Solucion general = x +
        combinacion
};

// Resuelve A * x = b (mod mod). A es n x m, b de tamano
    n.
SolMod solve_mod(const Matrix& A, const vl& b, ll mod =
    MOD) {
    int n = A.n, m = A.m;
    Matrix aug(n, m + 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) aug.M[i][j] = A.M[i
            ][j];
        aug.M[i][m] = b[i];
    }
    vi where;
    int rank = rref_mod(aug, m, where, mod);

    SolMod res;
    res.rank = rank;

    // Filas >= rank son 0 en las primeras m columnas:
        si su termino es != 0, inconsistente
    for (int i = rank; i < n; i++)
        if (aug.M[i][m] != 0) { res.status = -1; return
            res; }

    res.x.assign(m, 0);
    for (int c = 0; c < m; c++)
        if (where[c] != -1) res.x[c] = aug.M[where[c]][m
            ];

    for (int f = 0; f < m; f++) {
        if (where[f] != -1) continue;
        vl v(m, 0);
        v[f] = 1;
        for (int c = 0; c < m; c++)
            if (where[c] != -1) v[c] = norm(-aug.M[where
                [c]][f], mod);
        res.basis.push_back(v);
    }

    res.status = (rank == m) ? 1 : 0;
    return res;
}

// Devuelve directamente la solucion particular (vector
    vacio si no hay solucion)
vl solve_vector(const Matrix& A, const vl& b, ll mod =
    MOD) {
    SolMod r = solve_mod(A, b, mod);
    return r.status == -1 ? vl() : r.x;
}

// Determinante de matriz cuadrada, O(n^3)
ll determinant(Matrix a, ll mod = MOD) {
    int n = a.n;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) a.M[i][j] = norm(a.M
            [i][j], mod);

    ll det = 1 % mod;
    for (int i = 0; i < n; i++) {
        int sel = -1;
        for (int j = i; j < n; j++)
            if (a.M[j][i]) { sel = j; break; }
        if (sel == -1) return 0;
        if (sel != i) { swap(a.M[sel], a.M[i]); det =
            norm(-det, mod); }

        det = mulmod(det, a.M[i][i], mod);
        ll inv = mod_inv(a.M[i][i], mod);
        for (int j = i + 1; j < n; j++) {
            ll f = mulmod(a.M[j][i], inv, mod);
            if (!f) continue;
            for (int c = i; c < n; c++) {
                ll v = a.M[j][c] - mulmod(f, a.M[i][c],
                    mod);
                if (v < 0) v += mod;
                a.M[j][c] = v;
            }
        }
    }
    return det;
}

// Rango de una matriz n x m
int rank_matrix(Matrix a, ll mod = MOD) {
    vi where;
    return rref_mod(a, a.m, where, mod);
}

// Matriz inversa. Retorna {existe, inversa}
pair<bool, Matrix> inverse_matrix(const Matrix& a, ll
    mod = MOD) {
    int n = a.n;
    Matrix aug(n, 2 * n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) aug.M[i][j] = a.M[i
            ][j];
        aug.M[i][n + i] = 1;
    }
    vi where;
    if (rref_mod(aug, n, where, mod) < n) return {false,
        Matrix()};
    Matrix res(n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) res.M[i][j] = aug.M[
            i][n + j];
    return {true, res};
}

inline ll Matrix::det(ll mod) const { return determinant
    (*this, mod); }
inline int Matrix::get_rank(ll mod) const { return
    rank_matrix(*this, mod); }
inline pair<bool, Matrix> Matrix::inv(ll mod) const {
    return inverse_matrix(*this, mod); }

//////////////////////////////////////////
// 2. GAUSS MOD 2 CON BITSET
// Cada ecuacion es un bitset: bits 0..m-1 =
    coeficientes, bit m = termino independiente.
// Requiere m < XN.
//////////////////////////////////////////
const int XN = 2005;
typedef bitset<XN> bs;

// Devuelve status. rank por referencia. ans = solucion
    particular (libres = 0).
int gauss_xor(vector<bs> a, int m, bs& ans, int& rank) {
    int n = sz(a);
    vi where(m, -1);
    int row = 0;
    for (int col = 0; col < m && row < n; col++) {
        int sel = -1;
        for (int i = row; i < n; i++)
            if (a[i][col]) { sel = i; break; }
        if (sel == -1) continue;
        swap(a[sel], a[row]);
        where[col] = row;
        for (int i = 0; i < n; i++)
            if (i != row && a[i][col]) a[i] ^= a[row];
        row++;
    }
    rank = row;
    ans.reset();
    for (int i = row; i < n; i++)
        if (a[i][m]) return -1;
    for (int c = 0; c < m; c++)
        if (where[c] != -1 && a[where[c]][m]) ans[c] =
            1;
    return row == m ? 1 : 0;
}

// A (n x m de 0/1) * x = b (mod 2). Devuelve la
    solucion como vb (vacio si no hay).
vb solve_vector_xor(const vector<vb>& A, const vb& b) {
    int n = sz(A), m = n ? sz(A[0]) : 0;
    vector<bs> aug(n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) aug[i][j] = A[i][j];
        aug[i][m] = b[i];
    }
    bs ans; int rank;
    if (gauss_xor(aug, m, ans, rank) == -1) return {};
    vb res(m);
    for (int j = 0; j < m; j++) res[j] = ans[j];
    return res;
}

//////////////////////////////////////////
// 3. GAUSS REAL (long double, pivoteo parcial).
    Entradas long long o ld.
//////////////////////////////////////////

// a: matriz aumentada n x (m+1). Devuelve status; rank
    por referencia.
int gauss_real(vector<vector<ld>> a, vector<ld>& ans,
    int& rank) {
    int n = sz(a);
    if (n == 0) { ans.clear(); rank = 0; return 0; }
    int m = sz(a[0]) - 1;
    vi where(m, -1);
```

```
    int row = 0;
    for (int col = 0; col < m && row < n; col++) {
        int sel = row;
        for (int i = row + 1; i < n; i++)
            if (fabsl(a[i][col]) > fabsl(a[sel][col]))
                sel = i;
        if (fabsl(a[sel][col]) < EPS) continue;

        swap(a[sel], a[row]);
        where[col] = row;

        ld piv = a[row][col];
        for (int j = col; j <= m; j++) a[row][j] /= piv;
        for (int i = 0; i < n; i++) {
            if (i == row) continue;
            ld f = a[i][col];
            if (fabsl(f) < EPS) continue;
            for (int j = col; j <= m; j++) a[i][j] -= f
                * a[row][j];
        }
        row++;
    }
    rank = row;
    for (int i = row; i < n; i++)
        if (fabsl(a[i][m]) > EPS) { ans.clear(); return
            -1; }

    ans.assign(m, 0);
    for (int c = 0; c < m; c++)
        if (where[c] != -1) ans[c] = a[where[c]][m];
    return row == m ? 1 : 0;
}

// A * x = b con A, b en long long. Devuelve {status, x}
pair<int, vector<ld>> solve_real(const vector<vl>& A,
    const vl& b) {
    int n = sz(A), m = n ? sz(A[0]) : 0;
    vector<vector<ld>> aug(n, vector<ld>(m + 1));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) aug[i][j] = A[i][j];
        aug[i][m] = b[i];
    }
    vector<ld> ans; int rank;
    int st = gauss_real(aug, ans, rank);
    return {st, ans};
}

// Determinante real con pivoteo parcial, O(n^3)
ld determinant_real(vector<vector<ld>> a) {
    int n = sz(a);
    ld det = 1;
    for (int i = 0; i < n; i++) {
        int sel = i;
        for (int j = i + 1; j < n; j++)
            if (fabsl(a[j][i]) > fabsl(a[sel][i])) sel =
                j;
        if (fabsl(a[sel][i]) < EPS) return 0;
        if (sel != i) { swap(a[sel], a[i]); det = -det;
            }
        det *= a[i][i];
        for (int j = i + 1; j < n; j++) {
            ld f = a[j][i] / a[i][i];
            for (int c = i; c < n; c++) a[j][c] -= f * a
                [i][c];
        }
    }
    return det;
}

//////////////////////////////////////////

// EJEMPLO DE USO
//////////////////////////////////////////
void solver() {
    // --- Modular:  2x + y = 5,  x + 3y = 10 ---
    Matrix A({{2, 1}, {1, 3}});
    vl b = {5, 10};

    SolMod r = solve_mod(A, b);
    cout << "status=" << r.status << " rank=" << r.rank
        << " x: ";
    for (ll v : r.x) cout << v << " ";
    cout << endl;                          // status=1 rank
        =2 x: 1 3
    cout << "Det: " << A.det() << " Rank: " << A.
        get_rank() << endl;
    auto [ok, Ainv] = A.inv();
    if (ok) cout << Ainv;

    // --- XOR:  x1 ^ x2 = 1,  x1 = 0 ---
    vector<vb> Ax = {{1, 1}, {1, 0}};
    vb bx = {1, 0};
    vb sx = solve_vector_xor(Ax, bx);
    cout << "XOR: ";
    for (bool v : sx) cout << v << " ";
    cout << endl;                          // 0 1

    // --- Real (entradas long long) ---
    auto [st, xr] = solve_real({{2, 1}, {1, 3}}, {5,
        10});
    cout << "Real (status " << st << "): ";
    for (ld v : xr) cout << fixed << setprecision(2) <<
        v << " ";
    cout << endl;                          // 1.00 3.00
}
```